

Fredrik Hammarberg

Optimising Weak Reference Processing in the JVM Z Garbage Collector

*Reference Pipeline Optimisations and an Exploratory
Weak-Field Mechanism for the JVM*



UPPSALA
UNIVERSITET

Abstract

Reference processing is a non-trivial source of garbage collection overhead in workloads that maintain many weakly reachable objects. In the Z Garbage Collector (ZGC), all discovered weak references follow the same pipeline regardless of whether they are associated with a `ReferenceQueue`, incurring unnecessary enqueue-path work for references that were never registered with a queue. This thesis investigates whether targeted modifications to ZGC's reference-processing pipeline can reduce this overhead, and whether representing weak semantics directly in object fields rather than through separate `WeakReference` objects can reduce overhead further.

Three orthogonal optimisations for queue-less `WeakReference` processing were implemented in an OpenJDK research fork: a skip-enqueue fast path that routes queue-less references to a separate discovered list, a dynamic-array representation that replaces the intrusive linked list used during reference discovery, and a specialised clear path that removes unnecessary per-reference operations. In addition, an exploratory weak-field mechanism was implemented through a Java field annotation and corresponding ZGC VM support, enabling a representation-level comparison under the same experimental setup.

The implementations were evaluated using controlled microbenchmarks executed on a super-computer cluster, with 250 repeated runs per variant under CPU-isolated, exclusive-node conditions. Two benchmark designs were used: a single-object stress benchmark that concentrates the entire weak-reference load into one GC cycle, and a multi-object benchmark that releases references gradually across five GC cycles. All eight combinations of the three `WeakReference` optimisations and the weak-field variant were compared against an unmodified baseline.

The results show that the combination of the specialised clear path and the dynamic array is the most effective configuration for queue-less `WeakReference` processing. In the single-object benchmark, this combination reduces median non-strong reference-processing time by approximately 81 % and median major-collection time by approximately 8 %. In the multi-object benchmark, the non-strong subphase reduction is approximately 57 %, while end-to-end major-collection gains remain small. The skip-enqueue separation on its own provides negligible improvement, indicating that the gains are driven by improved traversal locality and reduced per-reference work rather than by the list split alone. Dynamic-array variants also incur higher auxiliary GC memory than the linked-list baseline.

The weak-field mechanism achieves a lower major-collection time than all `WeakReference` variants, with reductions of approximately 40 % in the single-object benchmark and 28 % in the multi-object benchmark. It does not, however, achieve the lowest non-strong processing times; those remain lower for the best `WeakReference` pipeline configurations. The advantage of weak fields comes primarily from reduced heap occupancy: by eliminating roughly 914 MB of wrapper objects in the single-object benchmark, weak fields reduce the total volume of GC work across all phases of the cycle. Together, the results suggest that representation choices can have at least as large an impact on total GC cost as pipeline-level optimisations.

Acknowledgments

The performance evaluations were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725.

I want to express my gratitude to my supervisor for his general support throughout this thesis as well as my subject reviewer for his academic guidance and overall feedback.

Additionally, I want to thank my colleagues at Oracle for their insights and feedback on my work throughout the development process.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

Acknowledgments	iii
1 Introduction	1
1.1 Motivation	2
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Scope and Delimitations	3
2 Background	4
2.1 Garbage Collection in the JVM	4
2.1.1 Garbage Collection Phases	5
2.1.2 Garbage Collection Barriers	5
2.1.3 Generational Collection	6
2.2 Java Reference Types	6
2.2.1 The Reference Object Lifecycle	7
2.2.2 WeakReference and ReferenceQueue	8
2.3 The JVM Execution Model	9
2.4 The Z Garbage Collector	9
2.4.1 Coloured Pointers and Barriers	10
2.4.2 Concurrent Phases and Parallel Workers	10
2.4.3 Access Decorators	11
2.5 Reference Processing Pipeline in ZGC	11
2.5.1 Discovery	12
2.5.2 Processing	13
2.5.3 Enqueue	14
3 Design and Implementation	15
3.1 Skipping Enqueue Path	15
3.1.1 Implementation of Skipping Enqueue Path	15
3.2 Dynamic Array for Discovered References	16
3.2.1 Implementation of Dynamic Array for Discovered References	17
3.3 Optimised Clear Path	17
3.3.1 Implementation of Optimised Clear Path	19
3.4 Weak Fields	19
3.4.1 Implementation of Weak Fields	20
4 Evaluation	25
4.1 Optimisation Variants	25

4.2	Performance Evaluation	25
4.2.1	Benchmark Design	26
4.2.2	Execution Environment	27
4.2.3	Measurement Methodology	27
4.3	Presentation of Results	29
5	Analysis of Results	43
5.1	Non-Strong Reference Processing	43
5.1.1	Effect of the Clear Path and Dynamic Array	43
5.1.2	Effect of the Skip-Enqueue Separation Alone	44
5.1.3	Comparison with Weak Fields	44
5.2	Overall GC Collection Times	45
5.2.1	Major Collection	45
5.2.2	Old Generation	46
5.2.3	Old Phase: Concurrent Mark	46
5.2.4	Young Generation (Promote All)	47
5.3	Memory Behaviour	47
5.3.1	Auxiliary Memory Usage and the Dynamic Array	48
5.3.2	Java Heap Usage	48
5.3.3	Process memFootprint	48
6	Discussion	50
6.1	Superadditive Interaction of the Clear Path and Dynamic Array	50
6.2	Why the Skip-Enqueue Separation Alone Provides Minimal Benefit	51
6.3	Dynamic Array as an Enabler of the Skip-Enqueue Benefit	51
6.4	The Concurrent-Mark Regression	52
6.5	Memory Overhead of the Pre-Fetched Array Entries	52
6.6	Structural Advantage of Weak Fields	53
6.7	Processing Mechanics of Weak Fields	54
6.8	Limitations	54
6.9	Future Work	55
7	Related Work	57
7.1	Java and the JVM	57
7.2	Go	58
7.3	C++ and .NET	58
7.4	Lua and Table-Level Weak Semantics	58
7.5	Python	59
7.6	Synthesis	59
8	Conclusion	61
	References	62
	Appendix A: Usage of Generative Artificial Intelligence Tools	65

A.1	Use During Implementation	65
A.2	Use for Tests and Scripts	65
A.3	Use During Writing	65
Appendix B:	Source Code	66
B.1	Dynamic-array Data Structure	66
B.2	Reference Processor	69
B.2.1	Interface	69
B.2.2	Implementation Excerpts	72
B.3	Weak Fields	77
B.3.1	Annotation	77
B.3.2	Data Structure	78
B.3.3	Reference Processor	82
B.3.4	Marking Closure	87
B.3.5	Compiler and Interpreter Support	88
Appendix C:	Per-Variant Patch Details	95
C.1	all	95
C.2	clear_path_dyn	95
C.3	clear_path_only	96
C.4	clear_path_sep	96
C.5	dyn_only	97
C.6	sep_dyn	97
C.7	sep_only	98
C.8	weak_fields	98
Appendix D:	Summary Statistics	100
D.1	Major Collection Time	100
D.2	Old-Generation Collection Time	100
D.3	Old Phase: Concurrent Mark	101
D.4	Non-Strong Processing Time	101
D.5	Old Phase: Concurrent Relocate	102
D.6	Young Generation (Promote All)	102
D.7	Young Phase: Concurrent Mark	103
D.8	Young Phase: Concurrent Relocate	103
D.9	Concurrent Mark Follow Time	104
D.10	Auxiliary GC Memory	104
D.11	Java Heap	105
D.12	Total Process RSS	105
Appendix E:	Additional Benchmark Plots	107

List of Tables

Table 4.1:	Optimisation variants	26
Table 4.2:	Patch sizes per variant	26
Table 4.3:	Cluster node hardware specification	28
Table 4.4:	JVM options used in all benchmark runs	28
Table 4.5:	Continuous monitoring metrics in the measurement pipeline .	29
Table 4.6:	GC log metrics in the measurement pipeline	30
Table C.1:	File changes in all	95
Table C.2:	File changes in <code>clear_path_dyn</code>	96
Table C.3:	File changes in <code>clear_path_only</code>	96
Table C.4:	File changes in <code>clear_path_sep</code>	97
Table C.5:	File changes in <code>dyn_only</code>	97
Table C.6:	File changes in <code>sep_dyn</code>	97
Table C.7:	File changes in <code>sep_only</code>	98
Table C.8:	File changes in <code>weak_fields</code>	98
Table D.1:	Major Collection summary statistics	100
Table D.2:	Old Generation summary statistics	101
Table D.3:	Old Phase: Concurrent Mark summary statistics	101
Table D.4:	Non-Strong Processing summary statistics	102
Table D.5:	Old Phase: Concurrent Relocate summary statistics	102
Table D.6:	Young Generation (Promote All) summary statistics	103
Table D.7:	Young Phase: Concurrent Mark summary statistics	103
Table D.8:	Young Phase: Concurrent Relocate summary statistics	104
Table D.9:	Concurrent Mark Follow summary statistics	104
Table D.10:	Auxiliary GC memory summary statistics	105
Table D.11:	Java heap summary statistics	105
Table D.12:	Total process RSS summary statistics	105

List of Figures

Figure 2.1:	Reference lifecycle states	8
Figure 2.2:	ZGC reference-processing pipeline	12
Figure 2.3:	Cross-generational reference case	13
Figure 4.1:	Major Collection timing (single-object)	31
Figure 4.2:	Major Collection timing (multi-object)	32
Figure 4.3:	Old Generation timing (single-object)	33
Figure 4.4:	Old Generation timing (multi-object)	34
Figure 4.5:	Old Phase: Concurrent Mark timing (single-object)	35
Figure 4.6:	Old Phase: Concurrent Mark timing (multi-object)	36
Figure 4.7:	Old Phase: Concurrent Process Non-Strong timing (single-object)	37
Figure 4.8:	Old Phase: Concurrent Process Non-Strong timing (multi-object)	38
Figure 4.9:	Young Generation (Promote All) timing (single-object)	39
Figure 4.10:	Young Generation (Promote All) timing (multi-object)	40
Figure 4.11:	Memory median percentage differences (single-object)	41
Figure 4.12:	Memory median percentage differences (multi-object)	42
Figure E.1:	Old Phase: Concurrent Relocate timing (single-object)	108
Figure E.2:	Old Phase: Concurrent Relocate timing (multi-object)	109
Figure E.3:	Young Phase: Concurrent Mark timing (single-object)	110
Figure E.4:	Young Phase: Concurrent Mark timing (multi-object)	111
Figure E.5:	Young Phase: Concurrent Relocate timing (single-object) ..	112
Figure E.6:	Young Phase: Concurrent Relocate timing (multi-object) ...	113
Figure E.7:	Memory per-run maximum distributions (single-object)	114
Figure E.8:	Memory per-run maximum distributions (multi-object)	115

List of Listings

Listing 2.1:	WeakReference constructors	8
Listing 3.1:	Queue-less weak discovery split	16
Listing 3.2:	Dynamic-array weak processing	18
Listing 3.3:	Optimised-clear-path fast inactive check	22
Listing 3.4:	Weak-field annotation	22
Listing 3.5:	Weak-field discovery	23
Listing 3.6:	Weak-field processing	24
Listing B.1:	zWeakRefArray .hpp: Dynamic-array data structure ..	66
Listing B.2:	zReferenceProcessor .hpp: Reference processor interface (all variant)	69
Listing B.3:	zReferenceProcessor .cpp: Implementation excerpts (all variant)	72
Listing B.4:	weak .java: Weak-field annotation	77
Listing B.5:	zWeakFieldArray .hpp: Weak-field data structure	78
Listing B.6:	zReferenceProcessor .hpp and excerpts (weak_fields variant)	82
Listing B.7:	zMark .cpp: Weak-field discovery during marking	87
Listing B.8:	Compiler and interpreter support for weak fields	88

1. Introduction

In managed runtimes, garbage collection (GC) is a central part of application performance: GC pauses affect latency, while concurrent GC work competes with application threads for processor time and memory bandwidth [1, 2]. OpenJDK’s Z Garbage Collector (ZGC) is designed for this regime, combining millisecond-scale pauses with a predominantly concurrent architecture intended to keep pause times largely independent of heap size [3]. Since its introduction as an experimental collector, ZGC has matured into a production-ready, generational collector, making it a suitable platform for studying the overhead of individual components in low-latency garbage collection [4, 5].

One aspect of ZGC that has received comparatively little optimisation attention is the processing of *weak references*. In Java, the class `WeakReference` provides weak semantics, allowing applications to retain references to objects without preventing those objects from being collected [6, 7]. When the garbage collector determines that an object is no longer strongly reachable, being reachable only through a `WeakReference`, it clears the reference and, if the reference is associated with a `ReferenceQueue`, enqueues it for application-level processing [8, 9]. This enqueueing step serves as a garbage collector callback, notifying the application that the `WeakReference` has been cleared. Weak semantics, both with and without such callbacks, are used in a range of programming patterns and data structures, for example the Java library `WeakHashMap` [10].

The current ZGC reference-processing pipeline treats all weak references uniformly: discovered references are linked into an intrusive linked list, processed, and then handed to a Java-level daemon thread (the `ReferenceHandler`) for enqueueing [11, 8]. This design imposes disproportionate overhead on weak references that are not registered with a `ReferenceQueue`. Such references are used solely for weak semantics and therefore do not require the callback mechanism provided by the queue. Nevertheless, the `WeakReference` object still needs to go through the entire processing pipeline, even though the application has no need for enqueueing.

This inefficiency has previously been identified in the OpenJDK bug tracker [12], which notes the unnecessary traversal of the pending list for references without a `ReferenceQueue`. However, as of April 2026, no implementation addressing this issue has been integrated into the JDK. Building on that observation, this thesis investigates whether targeted modifications to ZGC’s reference-processing pipeline can reduce reference-processing overhead. It explores three orthogonal optimisation

approaches for `WeakReference`-based processing: skipping the enqueue path for references without queues, replacing the intrusive linked list with a dynamic array, and optimising the codepath for clearing references. In addition, it examines a weak-field mechanism as an alternative implementation of weak semantics that avoids `WeakReference` objects entirely.

1.1 Motivation

Weak references, unlike regular references, require explicit processing by the JVM garbage collector [11, 13]. Each discovered weak reference must be individually evaluated, its referent field cleared if the referent is unreachable, and the reference potentially enqueued. This per-reference processing cost scales linearly with the number of weak references, making it a bottleneck in applications that allocate many weak references.

The problem is amplified by a mismatch between the API design and its implementation. Java’s `WeakReference` API provides an optional `ReferenceQueue` parameter [7], allowing applications to choose whether they want the callback mechanism or not. However, ZGC’s reference processor does not take advantage of this distinction, treating all weak references as if they require enqueueing. This results in unnecessary work for references that do not use a `ReferenceQueue`. At the same time, weak semantics are currently implemented through separate `WeakReference` objects to support this callback mechanism. Applications therefore also pay the allocation and processing cost of those objects even when they only need weak semantics.

ZGC’s concurrent architecture makes both of these inefficiencies particularly relevant. Because ZGC processes references concurrently with the application [3, 11], any reduction in per-reference overhead translates into better application performance. This motivates two complementary lines of investigation: optimising the existing `WeakReference` pipeline for the queue-less case, and exploring whether weak semantics expressed through annotated object fields can reduce overhead further by avoiding `WeakReference` objects altogether.

1.2 Problem Statement

The current ZGC reference processing pipeline does not distinguish between weak references with and without a `ReferenceQueue` during its phases [11]. All cleared references are threaded onto a pending list, transferred to the `ReferenceHandler` thread, and iterated regardless of whether they will actually be enqueued or not [8]. Additionally, the intrusive linked-list data structure used for discovered references exhibits poor cache locality [14]. Beyond these pipeline inefficiencies, representing weak semantics through separate

`WeakReference` objects introduces object and heap overhead that may be avoidable if weak semantics is instead expressed directly through annotated object fields.

1.3 Research Questions

This thesis addresses the following research questions:

- RQ1** How do modifications to ZGC’s processing of `WeakReference` objects without a `ReferenceQueue` affect the wall-clock time of reference processing, the total GC cycle time, the GC memory footprint, and Java heap usage in select workloads?
- RQ2** How does expressing weak semantics through annotated object fields affect reference processing time, GC cycle time, GC memory footprint, and Java heap usage compared to the optimised `WeakReference`-based approach of RQ1?

1.4 Scope and Delimitations

This thesis focuses on the reference processing pipeline within ZGC, specifically targeting weak references without a `ReferenceQueue`. The primary implementation work concerns `WeakReference`-based processing, with weak fields treated as an extension evaluated in the same benchmarking framework. Although the optimisation approaches are described in general terms, the implementation and evaluation remain ZGC-specific. Furthermore, some of the optimisations, such as the optimised clear path, may have broader applicability beyond weak references without a `ReferenceQueue`. However, those broader applications fall outside the scope of this thesis and are not evaluated.

The correctness of the optimisations is only verified through the standard OpenJDK test suite and manual inspection. Performance measurements are conducted on a single hardware platform under controlled conditions and therefore may not generalise to all environments.

2. Background

To answer the research questions posed in Section 1.3, it is necessary to understand key concepts of the Java Virtual Machine’s (JVM) garbage collection mechanisms, the design and implementation of the Z Garbage Collector (ZGC), the Java reference types and their processing pipeline within ZGC, and general principles of garbage collection.

2.1 Garbage Collection in the JVM

The JVM provides automatic memory management through garbage collection (GC). Rather than requiring the programmer to explicitly allocate and free memory as in languages such as C or C++, the JVM tracks which objects are reachable from the application’s root set and reclaims the memory of objects that are no longer reachable. The root set consists of all data that is directly accessible to the application, such as local variables on the stack, static fields, and active threads.

The JVM ships with several garbage collector implementations, each designed with different trade-offs between throughput, latency, and memory footprint [1]. The collectors are:

- **Serial GC** A single-threaded collector that performs all garbage collection work using one thread, avoiding inter-thread communication overhead. It is best suited to single-processor systems and small heaps.
- **Parallel GC** A throughput-oriented generational collector that uses multiple GC threads to speed up collection on multiprocessor systems. It is intended for medium to large data sets.
- **G1 GC (Garbage-First)** A mostly concurrent, region-based collector designed to scale from small to large multiprocessor machines. It targets predictable pause times with high probability while maintaining high throughput and is the default collector on most platforms [1].
- **ZGC (Z Garbage Collector)** A scalable, low-latency collector that performs expensive GC work concurrently with application threads. It targets pause times of only a few milliseconds (largely independent of heap size) and supports heap sizes from 8 MB to 16 TB [3, 4].

This thesis focuses on ZGC, as its low-latency design and concurrent reference processing pipeline can particularly benefit from optimisations that reduce the amount of work done during reference processing.

2.1.1 Garbage Collection Phases

Garbage collection generally occurs multiple times (cycles) during the lifetime of a Java application. Each cycle consists of several distinct phases, each with specific responsibilities. The exact phases and their order can vary between collectors, but a common structure is as follows:

1. **Mark Phase** The collector identifies all live objects by traversing from the root set and marking all reachable objects. Dead objects are those that are not marked. This phase may be performed concurrently with the application or during a stop-the-world pause, depending on the collector design.
2. **Reference Processing Phase** After marking completes, the collector processes *Reference* objects discovered during the mark phase. This phase determines which references have dead referents and prepares them for enqueueing. Like marking, this phase may be concurrent or done during a pause.
3. **Sweep/Reclamation Phase** The collector reclaims memory occupied by dead objects. Some collectors use a sweep algorithm (scanning the heap and freeing unmarked objects) whilst others use evacuation (copying live objects to new memory regions).
4. **Compaction/Relocation Phase** Some collectors compact the heap to eliminate fragmentation, moving live objects to consolidate free space. Others relocate objects incrementally as part of the collection process.

The order and concurrency model of these phases varies significantly between collectors. Serial and Parallel GC typically perform all phases in stop-the-world pauses. G1 performs marking and sweeping mostly concurrently with short pauses for synchronisation. ZGC performs marking, reference processing, and relocation concurrently, using barriers to maintain consistency with concurrent application threads.

2.1.2 Garbage Collection Barriers

In JVM garbage collectors, a *barrier* is a small piece of code inserted around memory accesses so that the collector can maintain correctness while the application is running [2]. The term is slightly misleading: barriers are not global locks and do not stop threads. Instead, they are local checks or bookkeeping actions attached to individual loads and stores of object references [2, 3, 15].

Barriers are especially important in concurrent collectors because GC and application threads observe and modify heap state at the same time. Rather than pausing the application for long periods, the collector uses barriers to spread consistency work across ordinary execution [3, 2].

Two broad classes are common:

- **Load (read) barriers**, executed when a reference is read. These can validate or transform the loaded pointer before it is used by the application [3, 15].
- **Store (write) barriers**, executed when a reference field is overwritten. These record metadata needed by the collector, for example to preserve marking invariants or to track inter-generational pointers [2, 5].

2.1.3 Generational Collection

In current Oracle JDK releases, all four supported JVM garbage collectors (Serial, Parallel, G1, and Z) use *generational collection*, based on the *weak generational hypothesis*: most objects die young, and those that survive tend to remain live for a long time [16]. To exploit this hypothesis, the heap is divided into at least two generations:

- The **young generation**, where newly allocated objects reside. Young generation collections are frequent but fast, since most young objects are short-lived.
- The **old generation**, where objects that have survived multiple young collections are promoted to. Old generation collections are less frequent but more expensive since live objects are more plentiful here.

ZGC adopted generational collection starting with JDK 21 [5]. In generational ZGC, the young and old generations can be collected independently and therefore concurrently. This is relevant to reference processing because references in different generations are handled during different collection cycles.

2.2 Java Reference Types

In addition to ordinary (strong) references, Java provides *reference objects* in the `java.lang.ref` package [6, 8] that allow applications to reference objects without preventing their garbage collection. These reference types are:

1. **SoftReference** The referent field is cleared at the discretion of the GC, typically when memory is low. Commonly used for memory-sensitive caches.
2. **WeakReference** The referent field is cleared as soon as it becomes weakly reachable (i.e., no strong or soft references exist to it). Widely used in caching frameworks, canonical mappings, and listener registrations.
3. **FinalReference** An internal JVM type (not part of the public API) used to implement the `finalize()` mechanism. Objects with finalisers are wrapped in a `FinalReference` so that the GC can schedule finalisation before reclaiming the object. This scheduling is done via a `ReferenceQueue`, which means that `FinalReferences` always have an associated `ReferenceQueue`.

4. **PhantomReference** The referent is never accessible through the reference; `get()` always returns `null`. Used for post-mortem cleanup actions, often to avoid the pitfalls of finalisation. Phantom references must be registered with a `ReferenceQueue` to be useful.

All four types extend the abstract class `Reference<T>` (in the `java.lang.ref` package), which defines the key fields of the reference object and the API for retrieving the referent, clearing the referent field, and checking if it is enqueued.

2.2.1 The Reference Object Lifecycle

Each `Reference` object maintains several fields that track its state [8]:

- **referent field** The referenced object. The GC may clear this field (set it to `null`) when the referent becomes eligible for collection according to the reference's type.
- **queue field** The `ReferenceQueue` this reference is registered with. If the reference was not created with a queue, this field points to the sentinel object `ReferenceQueue.NULL_QUEUE`.
- **next field** Used as a link pointer when the reference is enqueued in a `ReferenceQueue`.
- **discovered field** A field with dual purpose: during GC marking, it serves as a link in the garbage collector's internal *discovered list*; after processing, it is reused as a link in the VM's *pending list* that feeds the `ReferenceHandler` thread. Due to its first use, the GC can detect whether a reference has already been discovered by checking if this field is `null`.

A reference object transitions through the following states during its lifetime [6, 8]:

1. **Active** The reference has been created and the referent is still reachable (or not yet processed).
2. **Pending** The GC has determined the referent is eligible for clearing and has added the reference to the VM's pending list. The `ReferenceHandler` thread will process it.
3. **Enqueued** The `ReferenceHandler` thread has moved the reference into its associated `ReferenceQueue`, where the application can retrieve it via `poll()` or `remove()`.
4. **Inactive** The reference has been dequeued by the application, or was created without a queue and has been cleared. It is no longer tracked by the GC or queuing infrastructure.

An important observation is that references created *without* a `ReferenceQueue` can transition directly from Active to Inactive, bypassing the Pending and Enqueued states entirely. However, as discussed in Section 2.5, the default

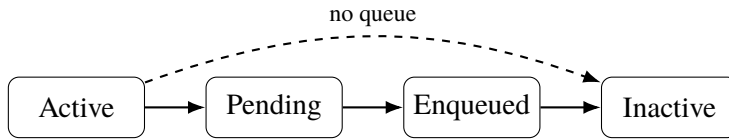


Figure 2.1. Lifecycle of Reference objects. References without a queue can bypass Pending and Enqueued.

implementation does not take advantage of this shortcut. The lifecycle and this direct shortcut path are illustrated in Figure 2.1.

2.2.2 WeakReference and ReferenceQueue

A `WeakReference` provides two constructors (with and without a `ReferenceQueue`), as shown in Listing 2.1 and implemented in OpenJDK [7].

```

1 // Without a queue - queue field set to NULL_QUEUE
2 public WeakReference(T referent) {
3     super(referent);
4 }
5
6 // With a queue
7 public WeakReference(T referent, ReferenceQueue<? super
8     T> q) {
9     super(referent, q);
10 }

```

Listing 2.1: *WeakReference* constructors

When a `WeakReference` is created with a `ReferenceQueue`, the application can be notified when the referent is collected. When created *without* a queue, the reference serves only as a conditional pointer, the application can call `get()` to check whether the referent is still alive, but receives no notification upon clearing.

The `ReferenceQueue` is implemented as a synchronized singly-linked list using the `next` field of the `Reference` objects. It provides `poll()` for non-blocking retrieval and `remove()` for blocking retrieval with an optional timeout [9]. This is the mechanism by which applications can react to the collection of referents, for example to clean up associated resources.

A key insight for this thesis is that Java explicitly supports queue-less weak references through the one-argument `WeakReference` constructor [7]. For workloads that only need liveness probing via `get()`, queue registration is unnecessary. In such cases, the VM still performs enqueue-path preparation work in the default pipeline, which motivated JDK-8029205 and the optimisation work in this thesis [12, 8, 11].

2.3 The JVM Execution Model

To understand how garbage-collector barriers are injected and why ZGC requires different components for each execution tier, it is helpful to examine HotSpot’s tiered execution model, which balances start-up latency against sustained throughput. Three execution tiers are relevant to this thesis:

1. **Interpreter.** At start-up, methods are first executed by the template interpreter, in which each bytecode is handled by generated assembly stubs. Interpretation provides the lowest throughput of the three tiers, but it has no Just In Time (JIT) compilation overhead and it gathers profiling data for later optimisation.
2. **C1 Compiler.** C1 is a fast JIT compiler that applies lightweight optimisations such as inlining, constant folding, and local simplifications. It compiles quickly, which makes it suitable for methods that are executed often enough to justify compilation but are not yet the hottest parts of the program.
3. **C2 Compiler.** C2 is the JVM’s aggressively optimising JIT compiler. It uses profiling data collected in earlier tiers to emit highly optimised native code for hot methods, applying transformations such as aggressive inlining, loop optimisations, and escape analysis.

In tiered compilation, methods move through these tiers as their execution frequency increases. A method typically starts in the interpreter, may then be compiled by C1, and can eventually be recompiled by C2 once sufficient profiling data has been collected.

Relevance to GC barriers.

Because a read barrier must intercept every load from the heap, and such loads occur in all three tiers, each tier requires its own barrier implementation. ZGC therefore ships three separate barrier components: `ZBarrierSetAssembler` generates barrier stubs embedded in the interpreter’s dispatch table, `ZBarrierSetC1` inserts barrier nodes during C1 intermediate representation construction, and `ZBarrierSetC2` integrates barriers into the C2 sea-of-nodes graph [15]. All three components obey the same decorator-driven barrier selection logic described in Section 2.4.3.

2.4 The Z Garbage Collector

ZGC is a concurrent, region-based, compacting garbage collector included in the JDK since JDK 11 (experimental) [3] and production-ready since JDK 15 [4]. Starting with JDK 21, ZGC supports generational collection, which is now the default mode [5]. The collector targets millisecond-scale pause times that remain bounded regardless of heap size, and supports heaps ranging from a few hundred megabytes to several terabytes [3]. Achieving these goals while performing the bulk of GC work concurrently is partly

realised by two closely related design decisions: encoding GC state directly in heap pointers, and using barriers to act on that state at every pointer access.

2.4.1 Coloured Pointers and Barriers

ZGC embeds metadata directly in the unused high bits of 64-bit heap pointers. These *colour bits* encode the state of a pointer relative to the current GC phase – for example, whether the pointed-to object has been remapped after relocation, or which generation it belongs to [3, 17]. Because the state is carried in the pointer itself, the GC can determine the status of a reference without loading the object’s header or consulting external tables.

To act on this encoded state, ZGC inserts barriers at every oop load and store. On a load, the barrier reads the colour bits of the retrieved pointer and checks whether they encode a *good* state for the current GC phase. If the pointer is stale – for example because the object has been relocated since the pointer was written – the barrier’s slow path applies the necessary fixup (remapping the pointer to the new address) before returning the address to the caller [3, 15]. Because these fixups happen lazily on access rather than eagerly during relocation, the collector can relocate objects concurrently without stalling the application.

Store barriers serve a complementary role: before overwriting a heap field, the barrier records the field’s previous value in a per-thread store-barrier buffer. This record is consumed by the marking phase to satisfy ZGC’s snapshot-at-the-beginning (SATB) invariant – ensuring that objects that were reachable at the start of a marking cycle are not missed – and by the remembered-set mechanism that tracks cross-generational pointers in the generational case [15].

2.4.2 Concurrent Phases and Parallel Workers

A ZGC collection cycle interleaves short stop-the-world pauses, used for root scanning and phase synchronisation, with longer concurrent phases in which marking, reference processing, and relocation proceed alongside running application threads [3, 11]. The load and store barriers described above are the mechanism that keeps the heap consistent during these concurrent phases: application threads that load a stale pointer automatically fix it up, and stores that would break the marking invariant are recorded for GC consumption.

Within each phase, ZGC distributes its work across multiple GC worker threads. To minimise contention, workers maintain their own thread-local data structures – such as per-worker discovered lists during reference discovery – and merge results into shared state only at well-defined synchronisation points [18, 11]. This design avoids the compare-and-swap contention that would arise from multiple threads updating a single shared resource.

2.4.3 Access Decorators

Throughout the HotSpot codebase, every load or store is annotated with a *decorator set*: a compile-time or runtime bitmask of type `DecoratorSet` (a 64-bit integer) whose set bits describe the semantics of the access [19]. The active `BarrierSet` implementation (e.g. `ZBarrierSet`) inspects this bitmask to decide which barrier variant to invoke, allowing a single generic `HeapAccess<decorators>::load()` call site to resolve to the correct low-level operation at both compile time (for template-specialised paths) and runtime (for dynamically dispatched paths).

The decorator flags are grouped by concern:

- **Reference strength** bits control how the barrier treats reachability. An access with `ON_WEAK_OOP_REF` tells ZGC that the field being read is the referent of a `WeakReference`: the barrier must return null if the referent has been collected, rather than reviving it. The default, when no strength decorator is specified, is `ON_STRONG_OOP_REF`.
- **Access location** bits indicate whether the access targets a heap object field or an off-heap native data structure. Some barrier actions are conditioned on `IN_HEAP`.
- **Barrier strength** bits control how strong barriers should be applied. `AS_RAW` bypasses GC barriers entirely and reads the raw memory value, which is necessary in certain internal GC paths. `AS_NO_KEEPLIVE` performs a correctly remapped access without keeping any objects alive, used in GC-internal scanning paths. `AS_NORMAL` (the default) invokes the full barrier.
- **Memory ordering** bits map to C++ memory-order semantics for the underlying atomic or plain load or store.

For ZGC, when `WeakReference.get()` is executed the referent field load carries `ON_WEAK_OOP_REF`. ZGC's barrier for this decorator checks the colour bits of the loaded pointer and if the pointer is already good (the referent is live and remapped or null), the address is returned directly. If not, the barrier's slow path is invoked. In the slow path, the liveness of the referent is checked and the barrier either returns a coloured null if the referent is no longer live, or a correctly coloured pointer if the referent is live [15, 19].

Store barriers are similarly decorator-driven. A normal store carries the decorator `ON_STRONG_OOP_REF` which triggers ZGC's store barrier, which records the old value in the store-barrier buffer (needed for the snapshot-at-the-beginning marking semantics and for maintaining remembered sets in the generational case).

2.5 Reference Processing Pipeline in ZGC

Reference processing is the mechanism by which the garbage collector handles `Reference` objects discovered during marking. The pipeline has three stages:

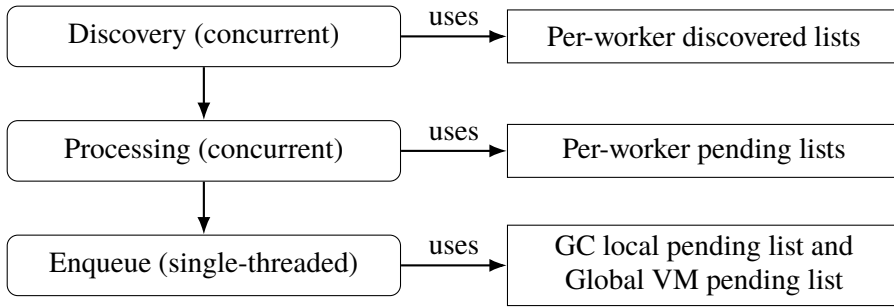


Figure 2.2. ZGC reference-processing pipeline and data flow from discovery to Java-level enqueue handling.

discovery, *processing*, and *enqueueing*. In ZGC, the first two stages run concurrently with the application during the marking phase, while the enqueue stage involves handing off references to the Java-level `ReferenceHandler` thread [11, 8]. An overview of this data flow is shown in Figure 2.2.

2.5.1 Discovery

During the concurrent marking phase, when ZGC encounters a `Reference` object (an instance of `SoftReference`, `WeakReference`, `FinalReference`, or `PhantomReference`), it evaluates whether the reference should be *discovered*, i.e. added to a list for later processing.

The discovery decision depends on several factors:

1. **Is the reference active?** Only active references are eligible. For non-`FinalReference` types, an inactive reference has a null referent field. For `FinalReference`, inactivity is indicated by a non-null next field.
2. **Is the referent still strongly reachable?** If the referent is already marked as strongly live, the reference does not need processing. The garbage collector checks this via its liveness information.
3. **Soft reference policy.** For `SoftReference` objects, the collector applies a policy (based on available memory and the reference’s last access time) to decide whether to keep the referent alive or clear the referent field [13, 20].
4. **Generational considerations.** In ZGC, only references residing in the old generation are discovered; young-generation references are always treated as strong [5, 11]. This is because reference processing introduces additional delay and uncertainty to the duration of the young-generation collection. By treating young references as strong, ZGC can complete young collections more predictably, which decreases the risk of the application running out of memory during a long young collection.

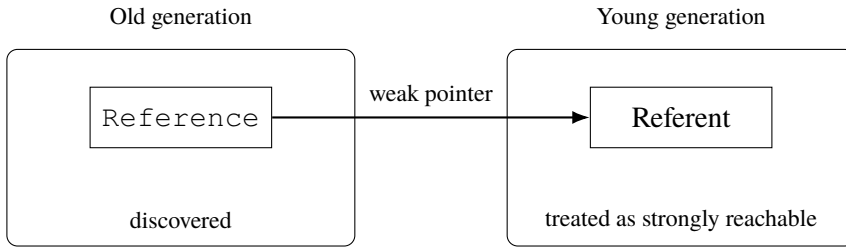


Figure 2.3. If an old-generation `Reference` points to a young referent, the young referent must remain alive during young collection.

Once a reference is selected for discovery, it is recorded on a *discovered list*. In ZGC, each GC worker thread maintains its own discovered-list head, so discovery is implemented as a cheap, non-atomic prepend to a per-worker list [11]. This avoids CAS contention found in shared processors where multiple threads update the same discovered head [13].

2.5.2 Processing

After marking completes, the discovered lists are processed. For each discovered reference, the collector determines the final fate of the referent:

1. If the referent field has become `null` (cleared by the application concurrently), the reference is dropped from the list.
2. If the referent is now strongly reachable (e.g. it was marked as alive after the reference's discovery), the reference is dropped.
3. If the referent is still unreachable according to the reference's strength, the referent field is **cleared** (set to `null`), and the reference is prepared for enqueueing.

In ZGC, clearing involves setting the `referent` field to a coloured null pointer via compare-and-swap (CAS) to ensure thread safety since application threads may be concurrently accessing the reference [15, 11].

After clearing, the reference is linked onto the thread-local pending list. This involves writing the reference's address into the `discovered` field of the previously processed reference. Once the thread has processed all references in its discovered list, the thread-local list is atomically prepended to ZGC's internal pending list [11].

Generational interactions are important during processing: when a `Refer-`
`ence` object is old but its referent is young, the referent is treated as strongly reachable and the referent field must not be cleared. Processing must therefore verify generation membership and, if appropriate, ensure young-generation liveness is preserved (by marking the young referent), as illustrated in Figure 2.3 [11, 21].

2.5.3 Enqueue

After a reference's referent field has been cleared, the reference must be made available to the application if it was created with a `ReferenceQueue`. This happens through the *pending list* mechanism which feeds the `ReferenceHandler` thread [11, 8]. The handoff process is as follows:

1. The GC has built an internal pending list by linking cleared references together via their `discovered` field.
2. Under the heap lock, the GC atomically prepends its internal pending list onto the global VM pending list.
3. The `ReferenceHandler` daemon thread is notified.
4. The `ReferenceHandler` thread wakes up, atomically retrieves the pending list, and iterates over it. For each reference, it checks whether `queue != NULL_QUEUE`. If the reference has a real queue, it calls `queue.enqueue(this)`; otherwise it simply advances to the next reference.

This handoff reveals an inefficiency: references created without a `ReferenceQueue` are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be skipped [8, 11]. For workloads where the majority of weak references lack queues, this results in a considerable amount of extra work. This behaviour has been reported in an OpenJDK enhancement issue [12]. Avoiding enqueue-path work for references with `NULL_QUEUE` is one of the primary optimisation targets explored in this thesis.

3. Design and Implementation

Three optimisations were implemented to address the inefficiencies in ZGC's processing of `WeakReference` objects without a `ReferenceQueue`. These optimisations are:

- Skipping the enqueue path for weak references without a `ReferenceQueue`, allowing them to be processed without being added to the pending list for the `ReferenceHandler` thread.
- Replacing the intrusive linked list used for discovered references with a dynamic array for weak references without a `ReferenceQueue`.
- Optimising the codepath for clearing references, to eliminate unnecessary operations for weak references without a `ReferenceQueue`.

In addition, an exploratory weak-field implementation was developed to evaluate an alternative way of expressing weak semantics that avoids the overhead of `WeakReference` objects entirely.

3.1 Skipping Enqueue Path

As described in Section 2.5.2, ZGC enqueues discovered references to the internal pending list regardless of whether they have a `ReferenceQueue`. As discussed in Section 2.5.3, this is inefficient for weak references without a queue. The reference-processing pipeline was therefore changed so that weak references without a `ReferenceQueue` can bypass the enqueue path entirely. The expected effect is less work for GC threads during reference processing, and therefore lower reference-processing time and GC cycle time.

3.1.1 Implementation of Skipping Enqueue Path

In order to skip enqueueing weak references without a `ReferenceQueue` the `ZReferenceProcessor` class was modified to check if the queue field of a weak reference being discovered is equal to the sentinel value `ReferenceQueue.NULL_QUEUE` before adding it to the discovered list. If the queue field is equal to `ReferenceQueue.NULL_QUEUE` (the weak reference was not created with a queue), it is added to a separate discovered list for weak references without a `ReferenceQueue` instead of being added to the regular discovered list. This separate discovered list is then processed independently by the GC threads and none of its weak references are enqueued to the pending

```

1 bool discover_reference(oop ref_obj, ReferenceType type)
2 {
3     zaddress ref = to_zaddress(ref_obj);
4
5     if (!should_discover(ref, type)) {
6         return false;
7     }
8
9     if (type == REF_WEAK && !has_reference_queue(ref)) {
10        weak_no_queue_list_per_worker.append(ref);
11    } else {
12        discovered_list_per_worker.append(ref);
13    }
14
15    mark_discovered(ref);
16    return true;
17 }

```

Listing 3.1: *Pseudocode of the null-queue fast path during discovery. Queue-less weak references are routed to a separate discovered list and processed by a no-enqueue pipeline (see Appendix B.2).*

list for the ReferenceHandler thread. Each GC thread maintains its own separate discovered list for weak references without a ReferenceQueue to avoid contention between threads. See Listing 3.1 for a pseudocode summary and Appendix B.2 for the actual implementation.

The discovered lists are separated rather than handled through conditional branches in a single processing loop so that two specialised reference-processing functions can be used: one for queue-backed references, which includes enqueue handling, and one for queue-less references, which omits it. This allows the queue-less path to avoid unnecessary per-reference checks during processing and keeps the no-enqueue path structurally simpler.

3.2 Dynamic Array for Discovered References

During processing, ZGC iterates over the discovered references. As described in Section 2.5.1, the baseline design stores them in an intrusive linked list [11]. Traversing such a list has poor cache locality [14], because successive references may be scattered throughout the heap. For weak references without a ReferenceQueue, the linked list was therefore replaced with a dynamic array. The expected benefit is better spatial locality during scanning, which should reduce time spent waiting on memory loads during processing and thereby lower both reference-processing time and GC cycle time.

3.2.1 Implementation of Dynamic Array for Discovered References

The implemented dynamic array, called `ZWeakRefArray`, stores discovered references in a contiguous layout rather than linking them via the `discovered` field. The array is allocated on the C heap and grows dynamically as needed.

Each GC worker thread maintains its own `ZWeakRefArray` instance to avoid contention. During discovery, when a reference should be added to a discovery list, its data is appended to the worker's array instead of being appended to a linked list. The reference's `discovered` field is still set (to a self-referencing pointer) to mark the reference as discovered, preventing duplicate discovery.

During processing, the array is iterated sequentially using index-based access. Because the elements are stored contiguously in memory, this iteration improves cache line utilisation compared to following pointers scattered across the heap [14]. After processing, the array is cleared for the next GC cycle but retains enough capacity to at least hold the same number of references that were processed but not made inactive. This avoids unnecessary reallocations in subsequent cycles when the number of discovered references is relatively stable.

See Listing 3.2 for a pseudocode summary and Appendices B.1 and B.2 for the actual implementation.

3.3 Optimised Clear Path

The standard reference processing path in ZGC uses general-purpose functions designed to handle all reference types uniformly. This generality introduces per-reference overhead that is unnecessary when processing only weak references without a `ReferenceQueue`, where the reference type is always `REF_WEAK` and enqueueing is never required.

The standard function for determining whether a reference's referent should be cleared performs several operations per reference [11, 15]:

1. A load barrier to read the referent from the reference object.
2. A virtual call to determine the reference type (soft, weak, final, or phantom).
3. A compare-and-swap (CAS) operation via a ZGC barrier to atomically set the referent field to a coloured null pointer.

For weak references without a `ReferenceQueue`, each of these operations can be simplified or eliminated:

1. When combined with the dynamic array optimisation, the referent field address and its value can be pre-loaded during discovery and stored in the array entry. This avoids redundant memory loads during processing. Pre-loading is safe because the only concurrent application-level operations

```

1 void discover_reference(oop ref_obj, ReferenceType type)
2 {
3     //...
4     ZWeakRefData data;
5     data.set_from_ref(ref);
6     worker_array.append(data)
7     //...
8 }
9
10 void process_worker_discovered_weak_refs_without_queue (
11     ZWeakRefArray& arr) {
12     size_t dropped = 0;
13     for (size_t i = 0; i < arr.length(); i++) {
14         const ZWeakRefData& data = arr.at(i);
15         data.mark_not_discovered();
16
17         if (try_make_inactive(data)) {
18             cleared_weak_no_queue_count++;
19         } else {
20             dropped++;
21         }
22     }
23     arr.clear_and_reserve(dropped);
24 }

```

Listing 3.2: *Pseudocode for the ZWeakRefArray-backed queue-less weak-reference path. Discovery appends pre-fetched fields and processing scans the array sequentially (see Appendices B.1 and B.2).*

on a `Reference`'s referent are clearing and enqueueing, both of which set the referent field to null rather than to a new non-null value [8, 11].

2. The reference type is known to be `REF_WEAK` at the call site, so the virtual call to determine the type can be eliminated.
3. The only concurrent modification to the referent field is an application thread setting it to null [8, 15, 11]. Because no thread can set the field to a new non-null value, the CAS operation for clearing the referent can be replaced with a simple store of a coloured null pointer.

These simplifications reduce both instruction count and memory-access overhead for each weak reference without a `ReferenceQueue` during processing, which should in turn reduce reference-processing time and GC cycle time.

3.3.1 Implementation of Optimised Clear Path

The implementation introduces a new function for determining whether a queue-less weak reference's referent should be cleared (see Appendix B.2). This function takes a data struct (either from the dynamic array or constructed on the fly from the linked list) containing the pre-fetched field addresses and values. This new function has similar semantics to the standard function and the ZGC barriers it uses, but it is specialised for the queue-less weak reference case and can therefore use more efficient operations. See Section 2.4 for more details on ZGC barriers.

The new function checks whether the referent is strongly live via metadata in ZGC's heap management. If the referent is not strongly live and resides in the old generation, its referent field is cleared using a direct pointer store rather than a CAS. This is safe under the constrained update model described above. If the referent is still live, either because it is strongly marked or because it resides in the young generation, the referent field is recoloured with the correct pointer metadata bits. That recolouring still has to use CAS so that a null value is not overwritten with a non-null value. Additionally, if the referent is young and the young generation is currently in its mark phase, the referent is marked to preserve cross-generational liveness [11, 15, 21].

The `discovered` field is also cleared using a direct store to the field address rather than through an atomic access function, since no other thread modifies this field after discovery in this processing path [11].

See Listing 3.3 for a pseudocode summary and Appendix B.2 for the actual implementation.

3.4 Weak Fields

The three core optimisations above operate within the existing `WeakReference` object model. However, that model has an inherent structural overhead: each weak relation requires a separate Java object with queue and list-link metadata, and the GC must still discover and process that object. To answer Research Question 2, an exploratory weak-field mechanism was implemented to test whether expressing weak semantics directly on object fields can reduce this overhead. The mechanism allows developers to annotate object fields as weak, indicating that the GC should treat those fields with weak semantics. The GC then applies weak semantic rules directly to the annotated field without requiring a separate `WeakReference` object. This is expected to slightly reduce reference-processing overhead and to reduce Java heap usage by eliminating the need for separate `WeakReference` objects. The reduction in Java heap usage should also reduce GC cycle time, since garbage-collection cost is broadly proportional to the number of live objects [2].

3.4.1 Implementation of Weak Fields

The implementation includes changes in three layers of the JVM: a Java-level annotation, GC-side field discovery and processing, and compiler/interpreter support for the annotation metadata.

Annotation

A new runtime-retained field annotation, `@java.lang.ref.weak`, was added to the `java.lang.ref` package. The annotation targets only fields and is retained at runtime so that the VM can inspect it during class loading. See Listing 3.4 for the annotation definition. During class-file parsing, the VM recognises the `@weak` annotation and sets a `weak` flag in the field's metadata.

Discovery

Weak-field discovery occurs during ZGC's old-generation marking phase. As the marking closure iterates over each object's reference fields, it checks whether the current field carries the `weak` flag in its metadata. If the field is annotated as `weak`, the marking closure diverts it to the weak-field discovery path instead of applying the normal strong mark barrier [11].

The discovery function first checks whether the field should be discovered at all. Fields that already point to null or to a strongly live object (identified by their ZGC pointer metadata bits) are skipped, since no clearing action will be needed for them. Fields in young-generation objects are also skipped, because young-generation weak-field processing is not supported by this implementation. If the field passes these checks, its address and current value are recorded as a `ZWeakFieldData` struct and appended to the GC worker thread's per-worker `ZWeakFieldArray`. See Listing 3.5 for a pseudocode summary and Appendix B.3.3 for the actual implementation.

Each GC worker thread maintains its own `ZWeakFieldArray` instance to avoid contention. The array is structurally identical to the `ZWeakRefArray` used for queue-less weak references: it stores entries contiguously on the C heap and grows dynamically (see Appendix B.3.2).

Processing

During the reference-processing phase, each GC worker iterates through its `ZWeakFieldArray` and applies a clearing function to each entry. The function uses the data recorded during discovery. If the field still contains the discovered value and the referent is no longer strongly live, the routine clears the field by storing a coloured null pointer. If the field still contains the discovered value but the referent remains live, it recolours the pointer with the correct metadata bits. If the field value has changed since discovery, no action is required, since only a strongly reachable referent could have been stored in the field after discovery. Apart from this extra check for post-discovery field updates, the function follows the same overall strategy as the optimised

clear path described in Section 3.3. After processing, the array is cleared but retains enough capacity to hold the entries that were not cleared, following the same capacity-retention strategy as `ZWeakRefArray`. See Listing 3.6 for a pseudocode summary and Appendix B.3.3 for the actual implementation.

Compiler and Interpreter Support

The `@weak` annotation metadata is also made available to the Just-In-Time compilers and interpreter. This is necessary to ensure that any loads or stores to a weak field outside GC processing are handled correctly with respect to weak semantics. The implementation adds a decorator to loads and stores that target a field carrying the `weak` flag in its metadata. ZGC then inserts barriers that apply the required semantics to all accesses with this decorator. Without the load decorator, a load from a weak field could return a non-null reference whose referent will be cleared by the GC. The store decorator is needed so that the GC does not record the previous value of the field, unlike in the default strong-reference case used to preserve the snapshot-at-the-beginning marking semantics. See Section 2.4.3 for more details on decorators and Appendix B.3.5 for the implementations.


```

1 bool try_make_inactive_fast(const ZWeakRefData& data) {
2     // if used with the dynamic array
3     zaddress referent = data.referent_addr;
4
5     // if used with the linked list
6     zaddress referent = load_barrier(data.
7         referent_field_addr);
8
9     // Young referents are implicitly kept alive by
10    generational rules.
11    if (is_young(referent)) {
12        if (young_mark_is_active()) {
13            mark_young_root(referent);
14        }
15        return false; // keep as active
16    }
17
18    // Old referent: clear only if not strongly live.
19    if (!is_strongly_live(referent)) {
20        *data.referent_field_addr = to_zpointer(zaddress
21            ::null);
22        return true; // became inactive
23    }
24
25    // Still live: recolour pointer metadata, but do not
26    // overwrite null.
27    zpointer observed = data.referent_field_value;
28    zpointer recoloured = remap_with_good_metadata(
29        referent);
30    cas_referent_if_non_null(data.referent_field_addr,
31        observed, recoloured);
32    return false;
33 }

```

Listing 3.3: Pseudocode for the specialised queue-less weak-reference inactive check. The old-generation clear path uses direct stores, while recolouring uses CAS to avoid null-to-non-null races (see Appendix B.2).

```

1 package java.lang.ref;
2
3 @Retention(RetentionPolicy.RUNTIME)
4 @Target(ElementType.FIELD)
5 public @interface weak {}

```

Listing 3.4: The `@weak` annotation. Applied to a field, it instructs the GC to treat the field value with weak-reference semantics (see Appendix B.3.1).

```

1 // During marking: the closure checks each reference
  field
2 void do_oop(oop* p) {
3     if (is_weak_annotated_field(p)) {
4         if (discover_weak_field(p)) {
5             return; // discovered, skip strong marking
6         }
7     }
8     // ... normal strong marking ...
9 }
10
11 bool discover_weak_field(oop* field) {
12     encountered_weak_fields_count++;
13
14     if (is_young(field)) {
15         return false; // young generation not supported
16     }
17
18     if (is_null(field) || is_strongly_live(field)) {
19         return false; // nothing to clear
20     }
21
22     discovered_weak_fields_count++;
23     ZWeakFieldData data(field);
24     worker_weak_field_array.append(data);
25     return true;
26 }

```

Listing 3.5: Pseudocode for weak-field discovery during marking. Annotated fields are diverted from the strong mark barrier and recorded in a per-worker array (see Appendix B.3.3).

```

1 void process_worker_discovered_weak_fields(
2     ZWeakFieldArray& weak_fields) {
3     size_t dropped = 0;
4     for (size_t i = 0; i < weak_fields.length(); i++) {
5         const ZWeakFieldData& data = weak_fields.at(i);
6         if (try_make_inactive_fast(data)) {
7             cleared_weak_fields_count++;
8         } else {
9             dropped++;
10        }
11    }
12    weak_fields.clear_and_reserve(dropped);
13 }

```

Listing 3.6: Pseudocode for weak-field processing. Each entry is passed to the clean barrier, which clears dead referents and recolours live ones (see Appendix B.3.3).

4. Evaluation

The implemented optimisations described in Chapter 3 were evaluated through custom microbenchmarks designed to synthetically target the reference processing pipeline. The benchmarks were executed on a supercomputer cluster under controlled conditions, and the resulting GC timing statistics and memory usage metrics were collected to enable comparison between the baseline and optimised variants.

4.1 Optimisation Variants

Nine optimisation variants were evaluated in total. Eight of them cover all combinations of the three `WeakReference` pipeline optimisations described in Chapter 3: the optimised clear path, skipping enqueue for queue-less references via separate discovered lists, and the dynamic array. Table 4.1 shows which optimisations each variant enables. The `none` variant serves as the unmodified baseline. The remaining variants enable a subset of the three optimisations, allowing their individual and combined effects to be isolated. The `all` variant enables all three simultaneously. The ninth and final variant, `weak_fields`, corresponds to the `@weak` field annotation described in Section 3.4.

Each variant is implemented as a set of patches under `patches/<variant>/. All variants except for dyn_only and weak_fields also include a shared base patch with infrastructure changes common to those optimisations. Table 4.2 lists the resulting diff sizes; full per-file breakdowns are available in Appendix C. A build script (scripts/build_configs.sh) automates applying the appropriate patches and building each JDK binary.`

4.2 Performance Evaluation

To evaluate the performance impact of the optimisation approaches, four microbenchmarks were used under controlled conditions: two `WeakReference`-based benchmarks and two weak-field counterparts with analogous workload structure. The benchmarks and their execution environment are described below.

Table 4.1. *Optimisation variants and their enabled optimisations. A tick indicates the optimisation is active; a dash indicates it is disabled.*

Variant	Optimised Clear Path	Skip Enqueue	Dynamic Array
none	—	—	—
dyn_only	—	—	✓
sep_only	—	✓	—
sep_dyn	—	✓	✓
clear_path_only	✓	—	—
clear_path_dyn	✓	—	✓
clear_path_sep	✓	✓	—
all	✓	✓	✓
weak_fields	—	—	—

Table 4.2. *Files changed and diff size per variant relative to upstream. Variants marked with † include the shared base patch (8 files, +39/−17 lines) in their totals.*

Variant	Modified	New	Ins.	Del.	Net
all†	10	1	+436	−65	+371
clear_path_dyn†	10	1	+448	−87	+361
clear_path_only†	10	0	+233	−44	+189
clear_path_sep†	10	0	+256	−43	+213
dyn_only	2	1	+210	−43	+167
sep_dyn†	10	1	+398	−58	+340
sep_only†	10	0	+213	−42	+171
weak_fields	27	2	+940	−257	+683

4.2.1 Benchmark Design

Two benchmark families were used to evaluate the optimisation approaches. The first family isolates reference-processing cost around a single shared reference, and the second is a more realistic multi-object workload that introduces greater allocation and liveness variability. Each family was implemented both with `WeakReference` objects and with weak fields. In order to enable `WeakReference`-to-weak-field comparison, both benchmark families use holder objects that either contain a `WeakReference` or an annotated weak field.

Single-Object Benchmark

The single-object benchmarks allocate 20 million holder objects, each containing either a `WeakReference` object or an annotated weak field that points to a single shared target object. The holder objects are allocated and stored in a randomised order using a Fisher-Yates shuffle [22] to avoid locality patterns.

None of the weak references are created with a `ReferenceQueue`. After a short pause, the single strong reference to the target object is released and a GC cycle is triggered via `System.gc()`. Because all weak references become clearable simultaneously in one GC cycle, this benchmark maximises the per-cycle reference-processing load and isolates the cost of the processing pipeline itself.

Multi-Object Benchmark

The multi-object benchmarks represent a slightly more realistic workload. They allocate 2 million objects, each containing a randomly sized byte array payload (between 509 and 1097 bytes). Each object has a strong reference and a corresponding holder object with either a `WeakReference` object or an annotated weak field. The allocation and storing order of both strong references and holder objects are randomised using separate Fisher-Yates shuffles [22] to avoid locality patterns.

Unlike the single-object benchmark, the strong references are not all released at once. Instead, the benchmark proceeds through five rounds of clearing. In each round, 20% of the strong references are nulled in a randomised order, making the corresponding objects eligible for garbage collection. After each round, `System.gc()` is called explicitly to trigger a collection cycle. This gradual release pattern means that weak references and weak fields are discovered and processed across multiple GC cycles rather than in a single burst. Furthermore, it tests how the pipeline performs when the referents have different sizes and lifetimes, which may affect locality and traversal cost.

4.2.2 Execution Environment

The benchmarks were executed on Uppsala Multidisciplinary Center for Advanced Computational Science’s (UPPMAX) supercomputer cluster Pelle. UPPMAX is part of the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725. Table 4.3 lists the hardware specification of the cluster nodes. Each variant was run using its pre-built JDK binary (see Section 4.1). To prevent file-system I/O from introducing timing variance, the JDK binaries, output log files, and results data were all stored on a RAM-backed file system. Table 4.4 lists the JVM options used in all benchmark runs.

4.2.3 Measurement Methodology

Each benchmark was run for a total of 250 iterations. Jobs were submitted via SLURM and allocated one node running in exclusive mode. Iterations were split across 4 parallel instances. Each instance was pinned via `taskset` to a set of 10 JVM cores and 2 auxiliary cores (12 cores per instance), preventing interference between concurrently running instances. Each instance

Table 4.3. *Cluster node hardware specification*

Property	Value
CPU	AMD EPYC 9454P (Zen 4), 2.75 GHz
Physical cores	48
Threads	96
Memory	768 GiB

Table 4.4. *JVM options used in all benchmark runs*

Option	Purpose
<code>-Xms100g -Xmx100g</code>	Fixed 100 GB heap
<code>-XX:+UseZGC</code>	Enable ZGC
<code>-XX:InitialTenuringThreshold=1</code>	Promote objects to old generation after one cycle
<code>-XX:MaxTenuringThreshold=1</code>	Cap tenuring at 1
<code>-XX:-ZProactive</code>	Disable proactive GC
<code>-XX:ZCollectionIntervalMajor=315360000</code>	Effectively disable timer-driven major collections
<code>-XX:+ZCollectionIntervalOnly</code>	Suppress allocation-pressure-driven major collections
<code>-Xlog:gc+stats,gc+ref</code>	GC statistics and reference processing logs
<code>-XX:NativeMemoryTracking=summary</code>	Native Memory Tracking

ran its share of the 250 iterations preceded by 1 warm-up iteration whose results were discarded. The warm-up executed the full benchmark workload for every variant. Within each measurement iteration, all nine optimisation variants were run in sequence with a 10-second cooldown between each. This interleaved ordering mitigates systematic drift in system conditions affecting one configuration more than others.

During each run, a memory monitoring script samples memory metrics from the process at high frequency. See Table 4.5 for the memory metrics collected.

In addition to the continuous memory samples, GC statistics are parsed from the log files of each run. For each GC metric, the parser extracts the total average and total maximum values reported by ZGC per run. The single-object benchmark concentrates its entire weak-reference load into a single collection cycle, so the per-run *maximum* is the relevant statistic. The multi-object benchmark spreads work across several cycles, so the per-run *average* is used instead. Table 4.6 lists the GC log metrics collected. The rest of the GC log

Table 4.5. *Continuous monitoring metrics used in the measurement pipeline.*

Metric	Units	Description
RSS	kB	Process resident set size, representing total physical memory in use by the JVM process.
GC Committed Memory	kB	GC-related memory currently committed by the JVM.
Java Heap	kB	Java heap usage.

metrics were also collected but are not the focus of this evaluation, they are however available in the published dataset [23].

4.3 Presentation of Results

Results are presented for all nine variants listed in Table 4.1: the unmodified baseline (`none`), the seven `WeakReference` pipeline variants, and `weak_fields`. This enables two complementary comparisons:

- **Intra-model comparison** among the `WeakReference` variants, isolating the effect of each optimisation and their interactions relative to the baseline.
- **Inter-model comparison** between the best-performing `WeakReference` variants and the `weak-field` representation, evaluating whether avoiding `WeakReference` objects altogether offers additional benefit.

Each GC timing figure shows a composite plot: a violin plot (top) with the full per-run distribution and a median percentage-difference histogram (bottom) showing all variants relative to the `none` baseline, including `none` itself at 0%. The single-object benchmark uses per-run maxima and the multi-object benchmark uses per-run averages, following the rationale in Section 4.2.3. The median is used for comparison rather than the mean, as the non-symmetric distributions tend to skew the mean. Note that the violin plot y-axis does not start at zero, it is range-fitted to the observed values so that the shape of each distribution is easier to see. GC timing results are shown for five metrics for each benchmark: Major Collection (Figures 4.1 and 4.2), Old Generation (Figures 4.3 and 4.4), Old Phase: Concurrent Mark (Figures 4.5 and 4.6), Old Phase: Concurrent Process Non-Strong (Figures 4.7 and 4.8), and Young Generation – Promote All (Figures 4.9 and 4.10). Memory results are shown as median percentage-difference histograms of per-run maxima for three metrics: auxiliary GC memory, Java heap, and total process RSS, for both benchmarks (Figures 4.11 and 4.12). Figures for remaining GC metrics from Table 4.6 are shown in Appendix E.

Table 4.6. GC log metrics used in the measurement pipeline. For the single-object benchmark, per-run maximum values are used; for the multi-object benchmark, per-run averages are used.

Metric	Units	Description
Major Collection	ms	End-to-end duration of major collections.
Old Generation	ms	Total time spent in old-generation collection work.
Old Phase: Concurrent Mark	ms	Time spent in old-generation concurrent marking.
Old Phase: Concurrent Process Non-Strong	ms	Time spent processing non-strong references in the old generation.
Old Phase: Concurrent Relocate	ms	Time spent in old-generation concurrent relocation.
Young Generation (Promote All)	ms	Total time in young-generation collection. Because the benchmarks use <code>System.gc()</code> every young-generation collection is a Promote All, which evacuates all surviving objects directly to the old generation.
Young Phase: Concurrent Mark	ms	Time spent in young-generation concurrent marking.
Young Phase: Concurrent Relocate	ms	Time spent in young-generation concurrent relocation.

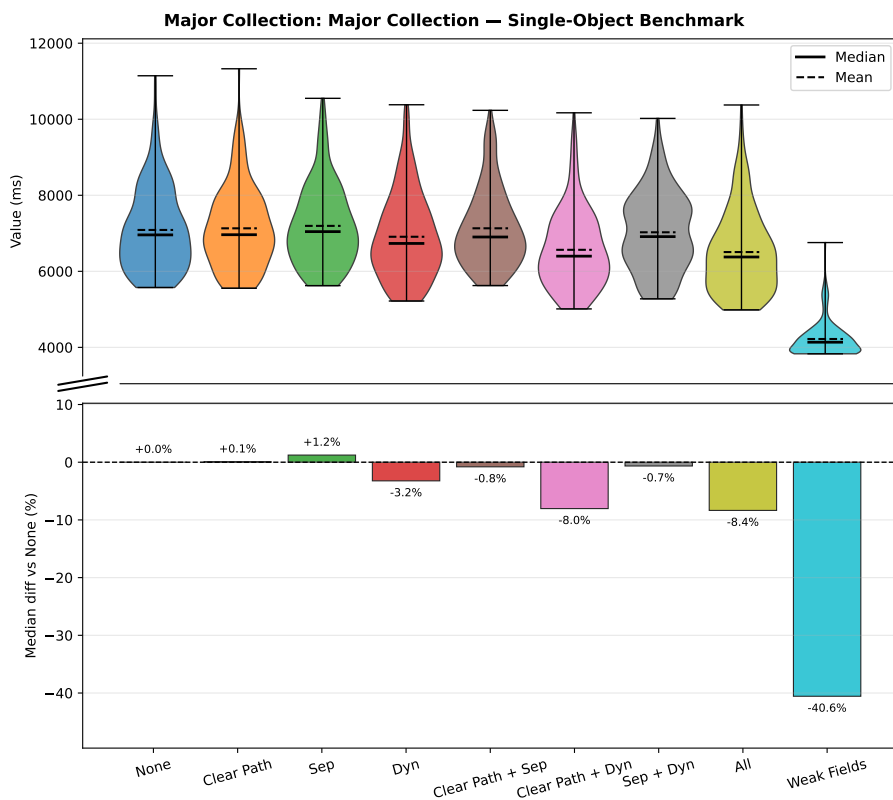


Figure 4.1. Major Collection timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

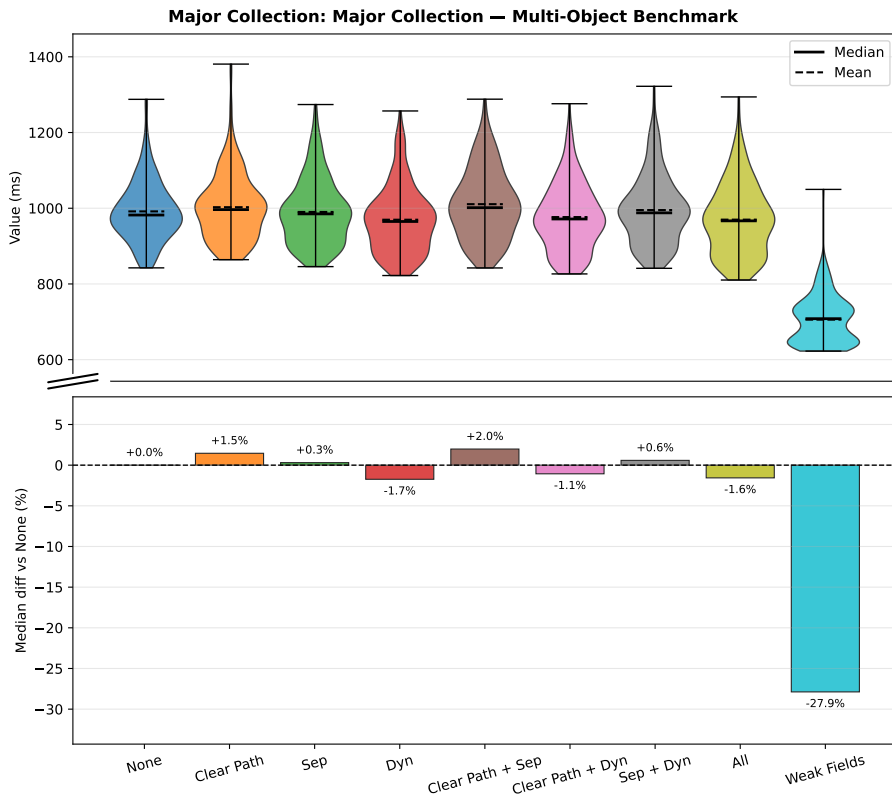


Figure 4.2. Major Collection timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

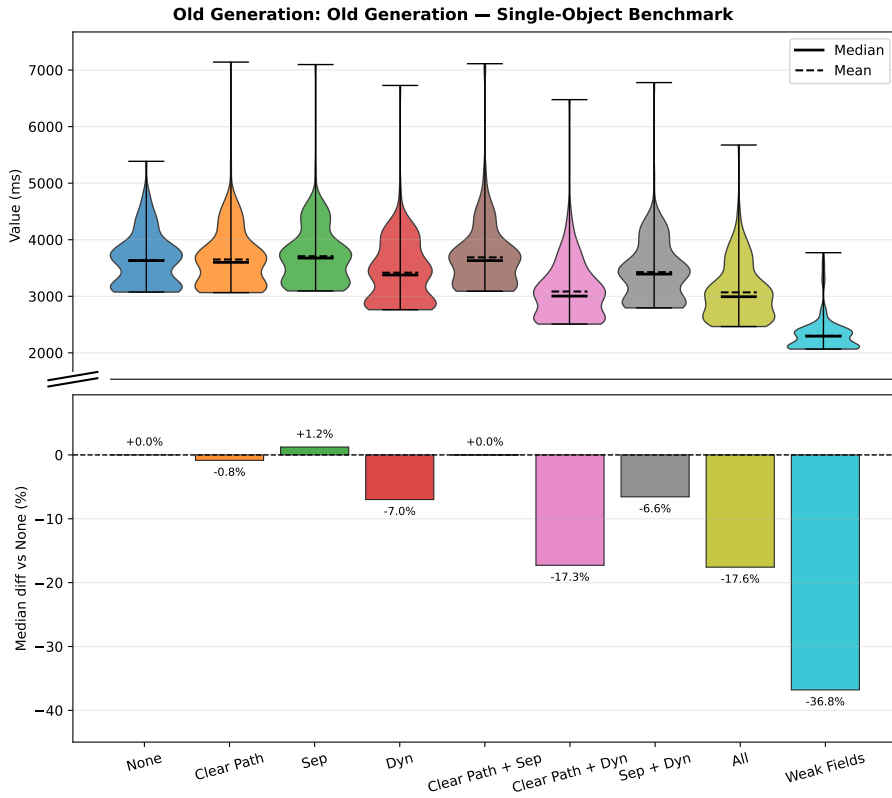


Figure 4.3. Old Generation timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

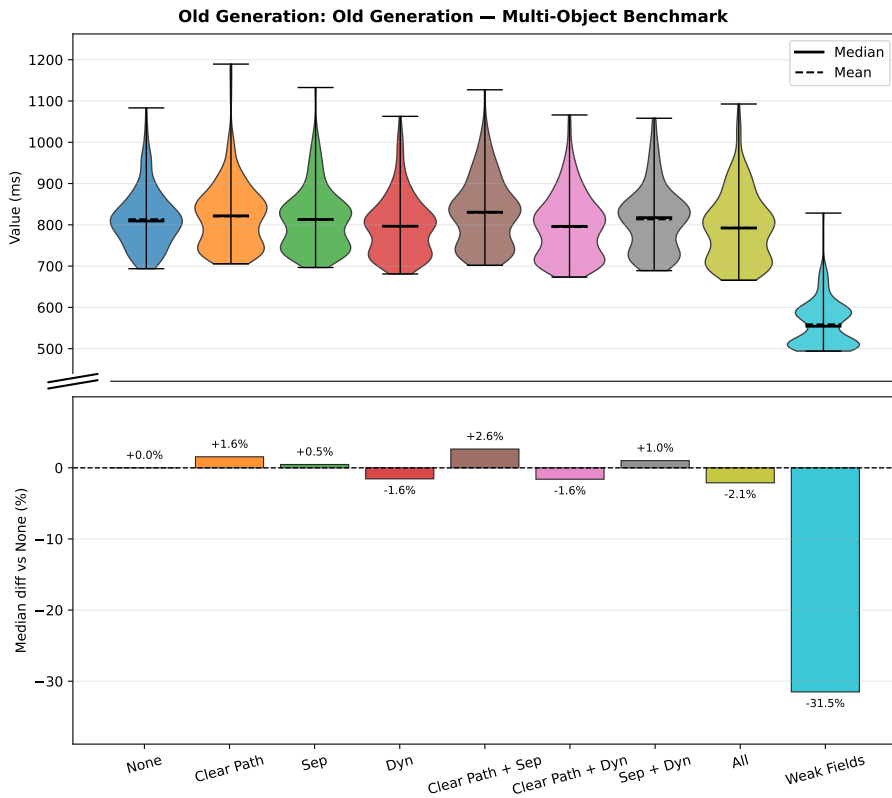


Figure 4.4. Old Generation timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

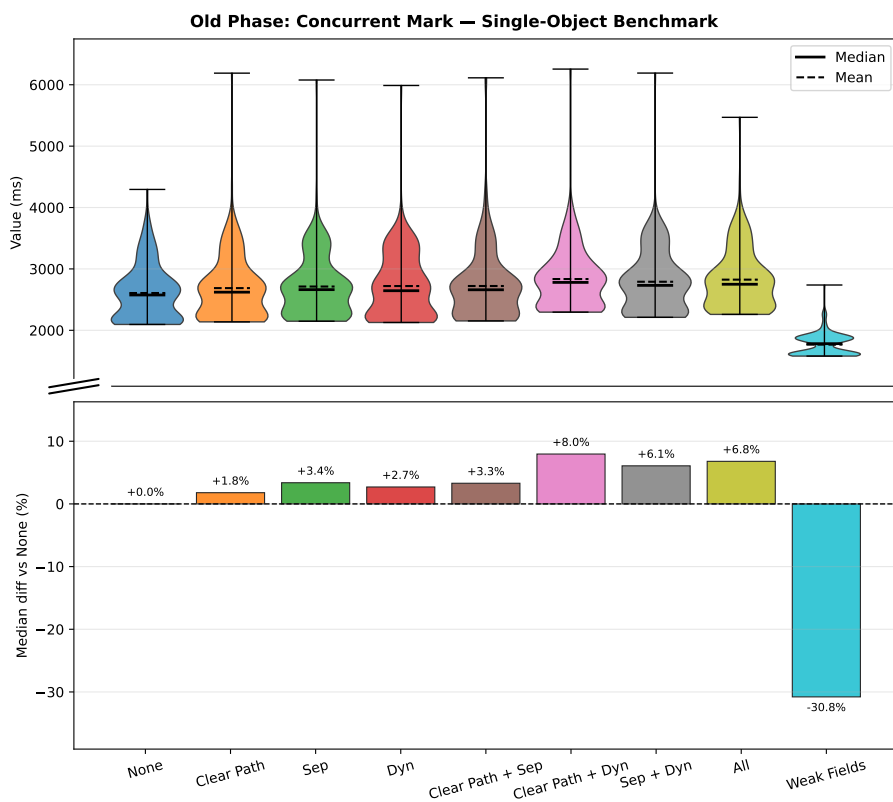


Figure 4.5. Old Phase: Concurrent Mark timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

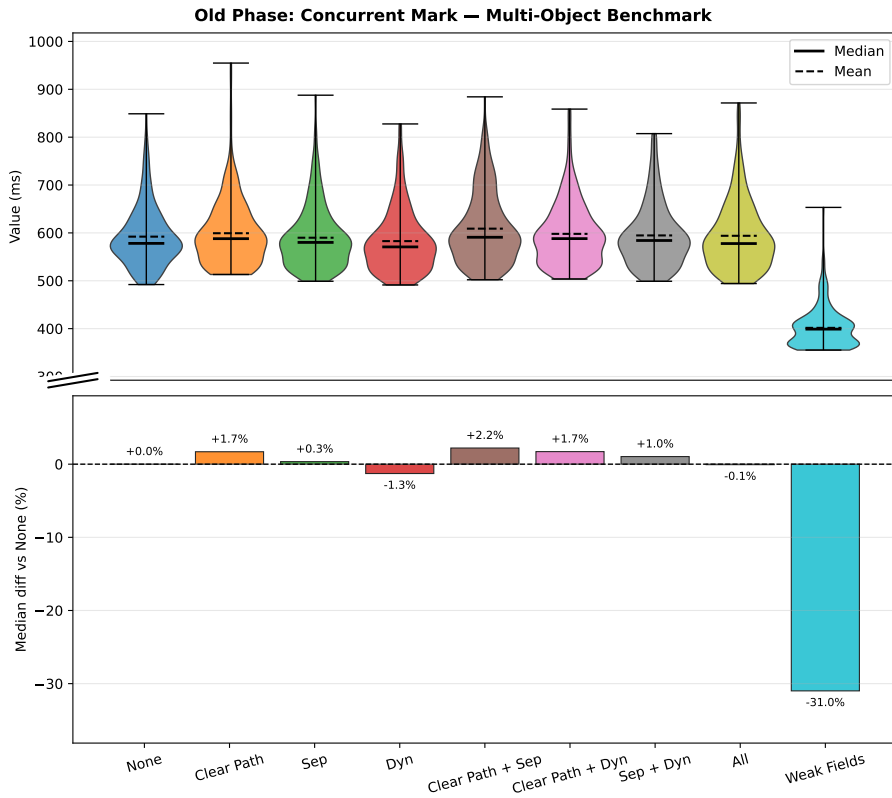


Figure 4.6. Old Phase: Concurrent Mark timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

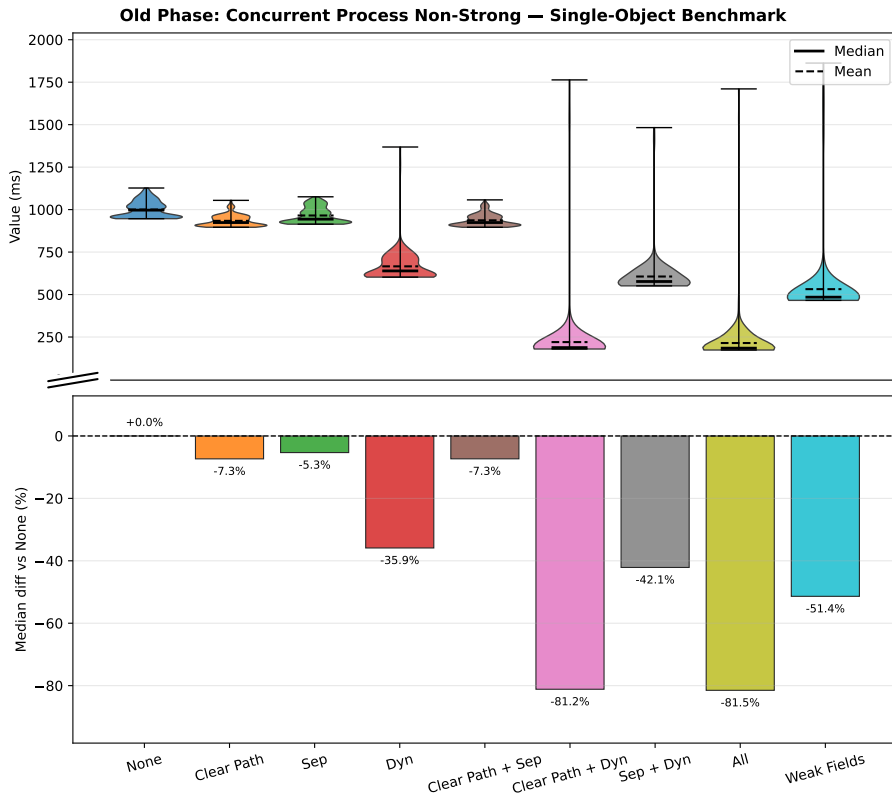


Figure 4.7. Old Phase: Concurrent Process Non-Strong timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

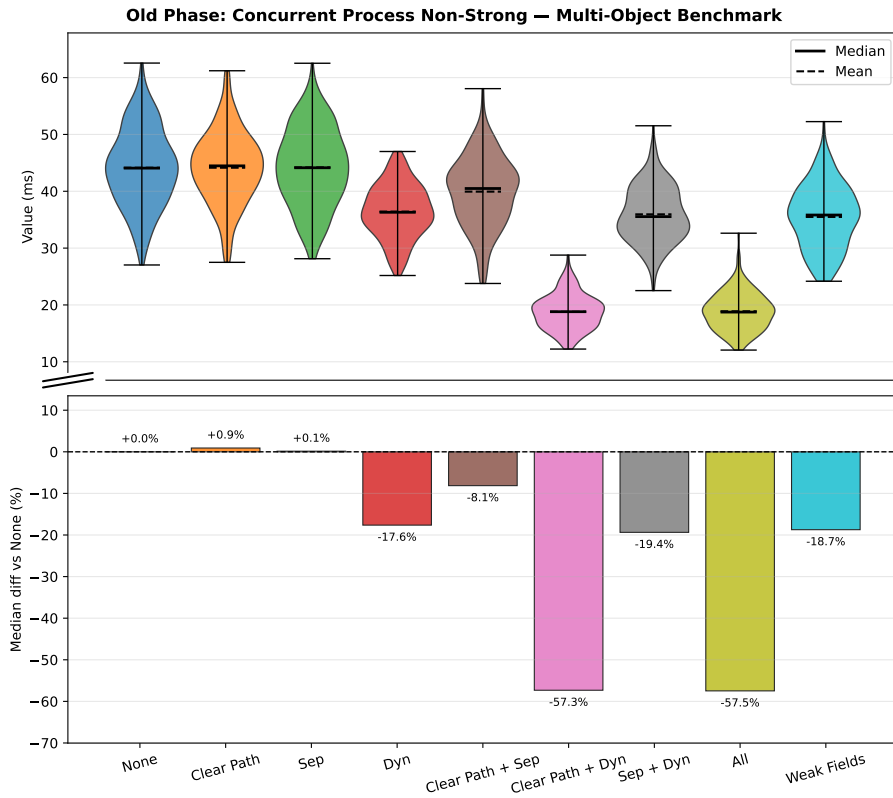


Figure 4.8. Old Phase: Concurrent Process Non-Strong timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

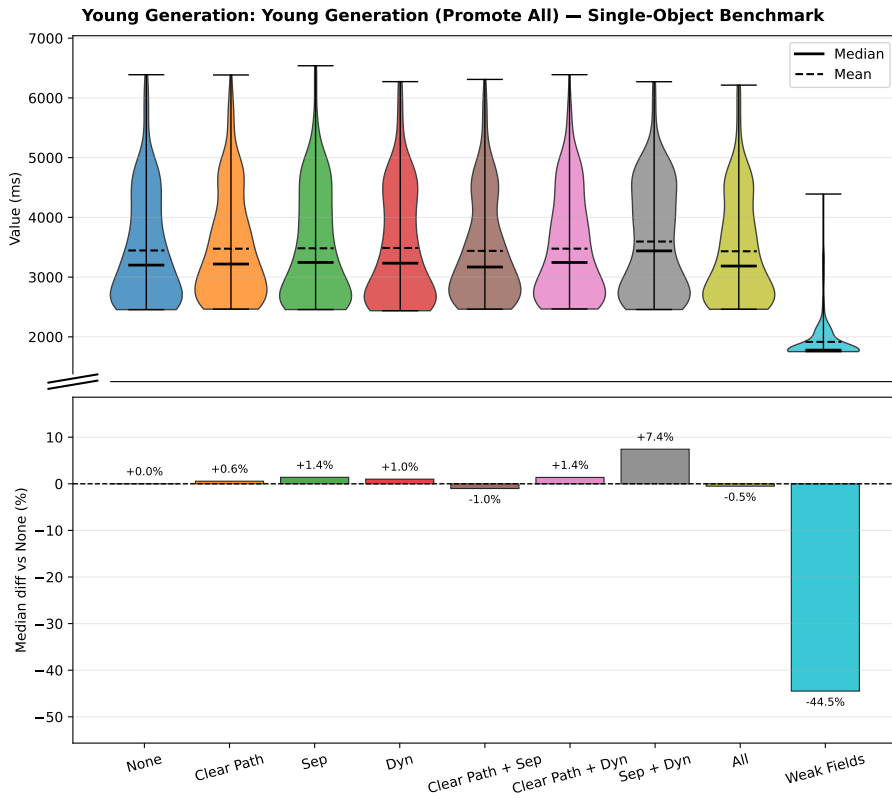


Figure 4.9. Young Generation (Promote All) timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

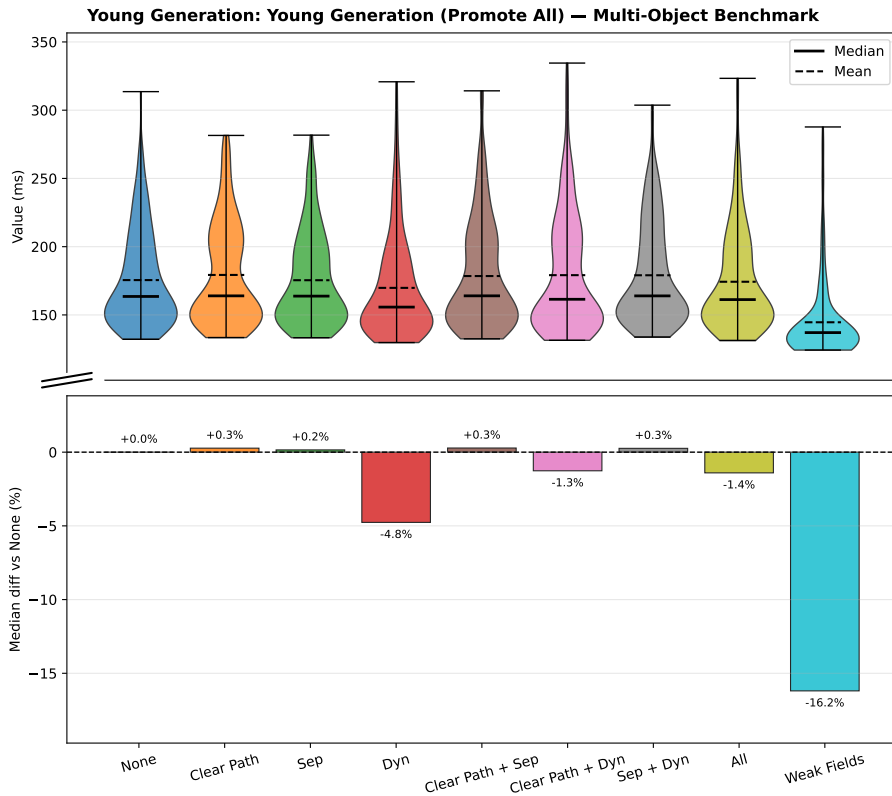


Figure 4.10. Young Generation (Promote All) timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

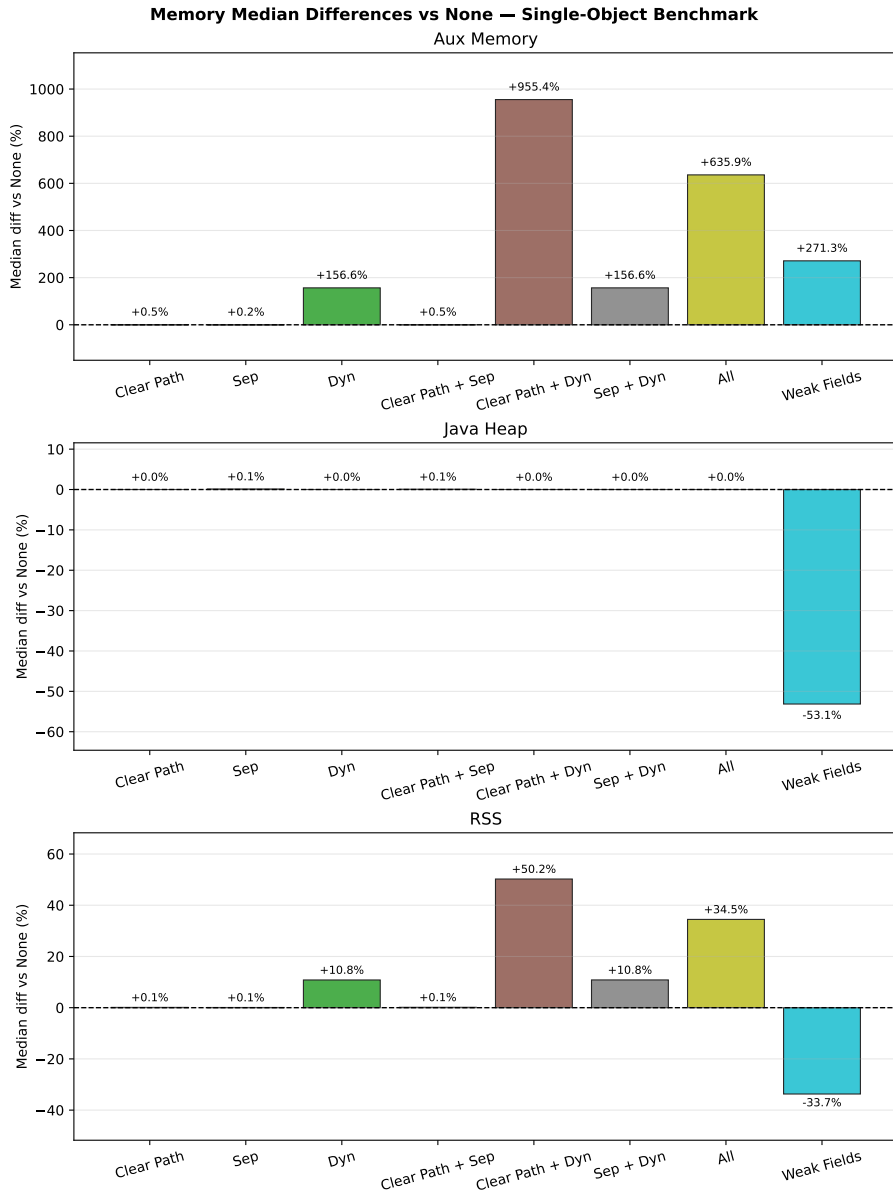


Figure 4.11. Median percentage difference in per-run maximum memory usage relative to the `none` baseline, single-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.

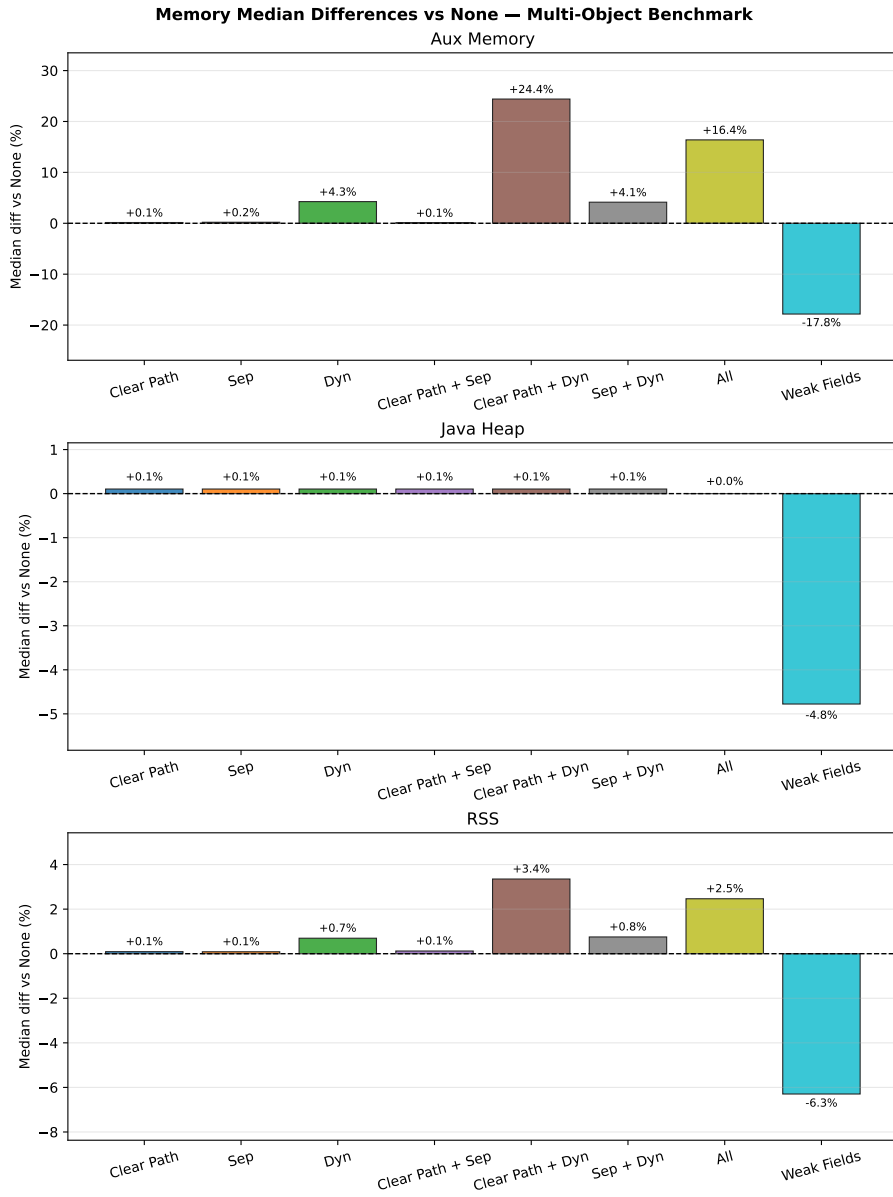


Figure 4.12. Median percentage difference in per-run maximum memory usage relative to the `none` baseline, multi-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.

5. Analysis of Results

The measurements presented in Section 4.3 show clear differences between configurations on some metrics and largely overlapping distributions on others. The analysis proceeds in two parts: a comparison among all variants on both the targeted non-strong processing metric and the broader collection-time metrics, followed by an analysis of memory usage and trade-offs. The `weak_fields` variant is included throughout and compared with the `WeakReference` variants at each stage.

5.1 Non-Strong Reference Processing

The metric most directly targeted by the implemented optimisations is `Concurrent Process Non-Strong`, which captures the total wall-clock time spent in ZGC’s concurrent reference-processing phase per GC cycle. Because this phase is the intended site of all three optimisations, the clearest differences between variants are observed here, making it the natural starting point for the analysis.

Figures 4.7 and 4.8 show the distributions of this metric for the single-object and multi-object benchmarks respectively. Full numerical summaries are given in Table D.4.

5.1.1 Effect of the Clear Path and Dynamic Array

The most striking reduction is achieved by the `clear_path_dyn` variant. In the single-object benchmark its median non-strong processing time is 187.9 ms, compared to 996.9 ms for the unmodified `none` baseline, a reduction of approximately 81%. The `all` variant, which adds the skip-enqueue separation on top of `clear_path_dyn`, reaches a nearly identical median of 184.4 ms (also about an 81% reduction). In the multi-object benchmark the pattern is consistent: `clear_path_dyn` achieves a median of 18.8 ms versus 44.1 ms for `none` (57% reduction), and `all` reaches 18.7 ms (also 57% reduction).

The near-equivalence of `clear_path_dyn` and `all` shows that adding the skip-enqueue separation to a configuration that already has the dynamic array and the specialised clear path yields little extra benefit: `sep_only` reduces the single-object median by only 5%, while `clear_path_dyn` and `all` differ by just 3.5 ms in the single-object benchmark and 0.1 ms in the multi-object benchmark.

The two individual components tell a different story. The `clear_path_only` variant, which enables only the specialised clear path, reduces the single-object median by a modest 7% (to 923.7 ms), while `dyn_only`, which enables only the dynamic array, achieves a 36% reduction (to 639.1 ms). When combined in `clear_path_dyn`, the effect is substantially larger than either isolated contribution. This suggests that the dynamic-array traversal and the specialised clear logic remove different parts of the per-reference cost and therefore interact constructively rather than redundantly.

5.1.2 Effect of the Skip-Enqueue Separation Alone

The `sep_only` variant, which introduces the separate discovered list for queue-less weak references but does not include the dynamic array or the specialised clear path, produces negligible improvement on non-strong processing time. In the single-object benchmark its median is 943.8 ms, only 5% below the `none` baseline, and its IQR of 73.8 ms is comparable to `none`'s 75.9 ms. In the multi-object benchmark the difference is entirely eliminated: 44.1 ms versus 44.1 ms for `none`.

5.1.3 Comparison with Weak Fields

The `weak_fields` variant achieves a median non-strong processing time of 484.6 ms in the single-object benchmark and 35.8 ms in the multi-object benchmark. These figures correspond to reductions of 51% and 19% respectively relative to `none`, positioning `weak_fields` clearly above the `none` baseline but well below the leading `clear_path_dyn` and `all` configurations. Its non-strong phase therefore improves clearly over the baseline, but not enough to match the best specialised `WeakReference` pipeline variants.

Notably, `weak_fields` is closely comparable to `sep_dyn` on this metric, especially in the multi-object benchmark where their medians are effectively indistinguishable: 35.8 ms versus 35.5 ms, a gap of just 0.3 ms. Their distributions are also closely aligned, with `sep_dyn` showing a slightly tighter IQR of 7.1 ms compared to `weak_fields`'s 8.2 ms. In the single-object benchmark `weak_fields`'s median is somewhat lower (484.6 ms versus 576.9 ms for `sep_dyn`, a difference of about 17%) and `weak_fields` has a substantially tighter IQR of 8.4 ms compared to `sep_dyn`'s 75.9 ms. This suggests that the weak-field approach may have a slightly better and significantly more consistent performance profile when the reference load is extremely high as it is in the single-object benchmark.

5.2 Overall GC Collection Times

In contrast to the non-strong processing metric, the overall collection time metrics show broad, heavily overlapping distributions across all `WeakReference` variants, making it difficult to draw firm conclusions from pairwise comparisons within this group. `weak_fields` however shows a clear difference in the distribution compared to all `WeakReference` variants across nearly all overall collection metrics.

5.2.1 Major Collection

For the single-object benchmark, the median major collection time of `none` is 6 958 ms with an interquartile range (IQR) of about 1 596 ms. The `all` variant has a median of 6 377 ms (8% below `none`) and the `clear_path_dyn` variant 6 399 ms (also 8% below `none`), but with IQRs of 1 419 ms and 1 394 ms respectively, both distributions still overlap substantially with the baseline and with each other. Of the other `WeakReference` variants, `dyn_only` has the lowest median at 6 733 ms, which is only about 3% below `none`, while the rest of the variants have medians that are within about 1% of `none` and IQRs that are comparable to `none`'s. In the multi-object benchmark the effect is even smaller in relative terms, with medians ranging from 965 ms (`dyn_only`) to 1 002 ms (`clear_path_sep`) against a baseline of 982 ms; all distributions remain within a narrow IQR band of roughly 93–136 ms, making fine-grained ranking unreliable. See Figures 4.1 and 4.2 for the comparisons and Table D.1 for numerical summaries.

Among the `WeakReference` variants, `all` and `clear_path_dyn` tend to occupy the lower part of the collection-time distribution in both benchmarks, which is directionally consistent with their superior non-strong processing times. However, given the distribution overlap, this observation should be treated as indicative rather than conclusive.

The `weak_fields` variant stands clearly apart on this metric. In the single-object benchmark its median major collection time is 4 136 ms, compared with 6 377–7 044 ms for all `WeakReference` variants, a reduction of approximately 35% against the best `WeakReference` variant and 41% against `none`. Its IQR of 485 ms is also substantially tighter than the 1 354–1 596 ms IQRs of the `WeakReference` group. In the multi-object benchmark the gap is smaller but still unambiguous: 708 ms versus 965–1 002 ms, a reduction of approximately 28% versus `none`. The source of this advantage is structural: by eliminating the `WeakReference` objects entirely, `weak_fields` reduces the amount of GC work that must be performed across all phases of the cycle, not just in the non-strong processing phase.

5.2.2 Old Generation

The old-generation collection time shows a clearer split among the `WeakReference` variants in the single-object benchmark than the major collection metric does. Configurations that include the dynamic array stand noticeably apart: `all` and `clear_path_dyn` both reach a median of approximately 2 993–3 003 ms, roughly 17% below the `none` baseline of 3 631 ms, while `dyn_only` and `sep_dyn` achieve intermediate reductions of around 7% (3 377 ms and 3 393 ms respectively). By contrast, the non-dynamic-array variants (`sep_only`, `clear_path_only`, `clear_path_sep`) remain within 1% of `none` at 3 600–3 676 ms. Given that the IQRs are 600–720 ms for the dynamic-array variants and 600–870 ms for the non-dynamic variants, the distributions of the two groups do overlap; nevertheless, the median separation of roughly 600 ms between `all` and the non-dynamic cluster is comparable to a full IQR, indicating a consistent directional effect. In the multi-object benchmark the picture is completely different: all `WeakReference` variants fall within 792–831 ms against a baseline of 809 ms, an entirely flat picture. See Figures 4.3 and 4.4 for the comparisons and Table D.2 for full numerical summaries.

The `weak_fields` variant again reduces the median substantially: 2 295 ms in the single-object benchmark (37% below `none`'s 3 631 ms) and 554 ms in the multi-object benchmark (31% below `none`'s 809 ms). Its IQR of 282 ms in the single-object benchmark is also considerably tighter than the 600–870 ms range of the `WeakReference` group. As stated in Section 5.2.1, this advantage is attributable to the reduced GC workload across all phases of the cycle, not just the non-strong processing phase.

5.2.3 Old Phase: Concurrent Mark

The concurrent mark phase shows where the `WeakReference` pipeline optimisations suffer. In the single-object benchmark, `WeakReference` variant medians for the old-phase concurrent mark metric fall in the range 2 575–2 780 ms, with `none` at 2 575 ms. The combination variants with dynamic arrays show higher medians (`clear_path_dyn` at 2 780 ms, `all` at 2 750 ms), the `dyn_only` variant is slightly lower at 2 644 ms, which is expected given that less information is stored in the dynamic array entries. However, this pattern is not consistent across benchmarks: in the multi-object benchmark, for example the `dyn_only` variant has a median of 570 ms, which is actually lower than the `none` median of 578 ms. Due to the high IQRs (560–810 ms and 67–99 ms respectively) the differences observed between the `WeakReference` variants on this metric should be treated as indicative rather than conclusive.

In contrast, `weak_fields` reduces the concurrent mark median by approximately 31% in both benchmarks (1 782 ms in single-object, versus 2 575 ms

for `none`; 399 ms versus 578 ms in the multi-object benchmark). This is again consistent with the statements above: by eliminating the `WeakReference` objects, `weak_fields` reduces the amount of GC work that must be performed across most phases of the cycle, including concurrent marking.

5.2.4 Young Generation (Promote All)

The young-generation (Promote All) phase shows no systematic response to the `WeakReference` pipeline optimisations, as expected: all three optimisations operate exclusively on old-generation reference processing and should have no bearing on young-generation work. The majority of `WeakReference` variants in both benchmarks fall within $\pm 1.5\%$ of the `none` median (3 201 ms in the single-object benchmark, 164 ms in the multi-object benchmark). Two isolated deviations stand out but are unlikely to reflect genuine causal effects. In the single-object benchmark, `sep_dyn` has a median of 3 439 ms, some 7.4% above `none`, yet this deviation is well within its IQR of 1 622 ms and disappears entirely in the multi-object benchmark where `sep_dyn` is within 0.3% of baseline. In the multi-object benchmark, `dyn_only` has a median of 156 ms, 4.8% below the `none` median of 164 ms; this too sits within its IQR of 45 ms and has no counterpart in the single-object benchmark where `dyn_only` is within 1% of baseline. See Figures 4.9 and 4.10 for the comparisons and Table D.6 for numerical summaries.

The `weak_fields` variant again stands apart: a -44% reduction in the single-object benchmark and -16% in the multi-object benchmark, attributable to the elimination of `WeakReference` objects that would otherwise be marked and then promoted (moved to the old generation).

5.3 Memory Behaviour

The dynamic-array optimisation improves non-strong processing time at the cost of allocating a contiguous buffer per GC worker thread rather than reusing the intrusive linked-list fields already present in `Reference` objects. This trade-off makes memory usage an important secondary dimension of the evaluation. Auxiliary GC memory is the most direct indicator of the dynamic array's overhead, since the array allocates a contiguous buffer per GC worker thread rather than reusing the existing fields of `Reference` objects. Java heap occupancy reflects the cost of maintaining the `WeakReference` objects themselves, which `weak_fields` eliminates entirely. Total process RSS captures the combined footprint of heap, GC metadata, and native memory, providing the broadest view of the memory trade-off. See Figures 4.11 and 4.12 for the comparisons and Tables D.10 to D.12 for numerical summaries.

5.3.1 Auxiliary Memory Usage and the Dynamic Array

All configurations that include the dynamic array (`dyn_only`, `sep_dyn`, `clear_path_dyn`, and `all`) incur substantially higher auxiliary memory than variants that retain the intrusive linked list. In the single-object benchmark, the median per-run maximum for `dyn_only` and `sep_dyn` is 308 MB of GC-committed memory, compared to approximately 120 MB for the linked-list variants, an increase of 157%. The `clear_path_dyn` variant is the highest of all at 1 268 MB (roughly 10× the non-array baseline), and `all` reaches 884 MB (+637%). In the multi-object benchmark the relative differences are much smaller: `dyn_only` and `sep_dyn` are only 4% above the `none` baseline of 292 MB at 305 MB and 304 MB respectively, `all` is 340 MB (+16%), and `clear_path_dyn` is 363 MB (+24%).

The `weak_fields` variant occupies an intermediate position. Its median auxiliary memory in the single-object benchmark is 446 MB, some 272% above the 120 MB of the non-array `WeakReference` variants but well below `clear_path_dyn`'s 1 268 MB. In the multi-object benchmark it is the lowest of all configurations at 240 MB, 18% below the `none` baseline of 292 MB.

5.3.2 Java Heap Usage

The heap usage figures are essentially identical across all `WeakReference` variants (approximately 1 720 MB per-run median in the single-object benchmark and 1 926–1 928 MB in the multi-object benchmark), with all variants within 0.1% of each other.

The `weak_fields` variant, however, shows markedly lower heap usage: a median of 806 MB in the single-object benchmark, 53% below the `WeakReference` group, and 1 834 MB in the multi-object benchmark, 5% below. This is consistent with the fact that `weak_fields` eliminates the `WeakReference` objects entirely, which constitute a significant portion of the heap occupancy in the `WeakReference` variants.

5.3.3 Process memFootprint

The RSS results follow the same overall direction as auxiliary memory and heap usage. In the single-object benchmark, `clear_path_dyn` reaches the highest median RSS at 2 856 MB (+50% versus the `none` baseline of 1 901 MB), followed by `all` at 2 556 MB (+34%) and `dyn_only` and `sep_dyn` at 2 107 MB each (+11%). The non-array `WeakReference` variants remain within 0.2% of `none`. `weak_fields`, by contrast, has the lowest median RSS of all configurations at 1 260 MB, 34% below `none`. In the multi-object benchmark the spread is much narrower: `clear_path_dyn` is the highest at 2 487 MB (+3% versus `none`'s 2 407 MB), while `weak_fields` is the lowest at 2 255 MB (−6%). This strengthens the interpretation that the

dynamic-array variants trade runtime memory overhead for faster queue-less weak-reference processing, while weak fields reduce both heap occupancy and total process footprint.

6. Discussion

Across both benchmarks, the pipeline optimisations improve the metrics they directly target while leaving broader collection behaviour largely unchanged. The `weak_fields` variant stands apart: by eliminating `WeakReference` objects from the heap entirely, it propagates improvements through every phase of the collection cycle rather than just the reference processing phase. The following sections explain the mechanisms behind these patterns and reflect on the scope of the evaluation.

6.1 Superadditive Interaction of the Clear Path and Dynamic Array

The most striking result is that combining the clear-path optimisation with the dynamic array produces an 81% reduction in non-strong processing time, far larger than the sum of their individual contributions (7% and 36% respectively). This superadditivity arises from the tight coupling of the two mechanisms at the implementation level.

In `dyn_only`, the dynamic array provides improved cache locality during traversal: discovered references are stored contiguously in memory, so iterating through them involves sequential cache-line loads rather than following pointer chains scattered across the heap [14]. However, the per-reference processing logic still uses the standard CAS-based clear path, including a load barrier on the referent field and a CAS operation to atomically set the cleared referent to a coloured null pointer. These operations are not eliminated by the array alone.

In `clear_path_only`, the specialised clear logic eliminates the CAS on the referent field for old-generation referents (replacing it with a direct store) and removes the virtual call to determine the reference type. However, the referent field address and value must still be loaded individually for each reference during processing, and traversal still follows the intrusive linked list with its associated cache misses.

When both optimisations are active in `clear_path_dyn`, the dynamic array enables a further key capability: the referent field address and value are pre-fetched during discovery and stored directly in the array entry (see Listing 3.2). Processing then reads these values from the contiguous array without issuing any additional heap loads. The specialised clear path operates on the pre-fetched data, removing both the load-barrier overhead and the pointer-chasing overhead simultaneously. Each optimisation removes a bottleneck that would persist if the other were applied alone.

6.2 Why the Skip-Enqueue Separation Alone Provides Minimal Benefit

The skip-enqueue optimisation was motivated by the observation that the enqueueing stage performs unnecessary work for queue-less weak references: they are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be discarded [8, 11, 12]. Despite this, `sep_only` reduces the single-object median by only 5% and shows no reduction at all in the multi-object benchmark.

The enqueue stage is not the bottleneck. The dominant costs in the baseline pipeline occur in the processing stage: each reference requires a load barrier on the referent field, a virtual call to determine the reference type, and a CAS operation to clear the referent. These operations are executed in the concurrent GC phase for every discovered reference. The enqueue stage, by contrast, involves linking references together via their `discovered` field, prepending the list to the global pending list under a lock, and then a single thread iterating the pending list. For a workload with 20 million queue-less references, skipping the enqueue step saves comparatively little work relative to what the processing stage already costs.

Furthermore, `sep_only` uses a separate linked list for queue-less references rather than a dynamic array, so per-reference traversal still suffers from the same cache-miss pattern as the baseline. The skip-enqueue separation routes these references away from the enqueue path, but does not reduce the cost of reaching that point.

6.3 Dynamic Array as an Enabler of the Skip-Enqueue Benefit

While `sep_only` provides negligible benefit, `sep_dyn` reduces non-strong processing time by approximately 42% in the single-object benchmark, compared to `dyn_only`'s 36%. The additional reduction comes from two sources. First, `sep_dyn` routes queue-less references to a dedicated dynamic array during discovery; the array traversal therefore operates over a known-homogeneous set where every entry belongs to the no-enqueue path. Second, the processing function for this separate array knows statically that no reference in its collection will be enqueued, allowing it to omit the per-reference queue check and the pending-list linking step, making the per-iteration work cheaper and improving instruction-cache behaviour in the hot loop.

This comparison illustrates that the skip-enqueue separation is not an independent optimisation: its value is contingent on the array providing cache-friendly traversal and a structurally simpler processing function. Without the array, skipping the enqueue step is a minor tail optimisation; with it, the entire hot loop for queue-less references is streamlined.

6.4 The Concurrent-Mark Regression

In the single-object benchmark, variants with the dynamic array show slightly higher concurrent-mark times than `none`: `clear_path_dyn` at 2 780 ms and `all` at 2 750 ms, compared to `none`'s 2 575 ms. This is unexpected because the dynamic array is used exclusively during reference processing, not during marking, and should in principle have no effect on the mark phase.

One plausible explanation involves memory bandwidth pressure. With 20 million references discovered in a single cycle, the retained dynamic-array allocations amount to hundreds of megabytes of committed GC memory. The arrays are kept at peak capacity between cycles (see Section 3.2.1), so this memory remains resident throughout the subsequent concurrent mark phase. Concurrent marking must then trace the heap while these large buffers compete for cache lines and memory bandwidth with the marking data structures (live bitmaps, marking stacks, and remembered sets). Under memory pressure, the marking threads may experience increased cache-miss latency, stretching the concurrent mark phase.

A second possibility is that reference processing, which precedes marking in some generational ZGC cycle configurations, affects the subsequent marking phase through memory subsystem state such as cache eviction or TLB pressure. However, with IQRs of 560–810 ms for this metric in the single-object benchmark and the pattern absent from the multi-object results, a causal attribution cannot be established with confidence. The regression could also be an artefact of sampling variability. A more targeted experiment – running only the concurrent mark phase with and without large retained arrays – would be needed to confirm the cause.

6.5 Memory Overhead of the Pre-Fetched Array Entries

The auxiliary GC memory results show a notable asymmetry among the dynamic-array variants: `dyn_only` and `sep_dyn` use 308 MB in the single-object benchmark (157% above `none`'s 120 MB), while `clear_path_dyn` uses 1 268 MB (roughly 10× the non-array baseline) and `all` uses 884 MB. This asymmetry directly reflects the per-entry struct size.

The `ZWeakRefArray` used in `dyn_only` stores one entry per discovered reference containing the reference's address and its `discovered` field address, which is the minimum needed for processing. The `clear_path_dyn` variant uses an extended entry that additionally stores the referent field address and the pre-fetched referent field value, enabling the barrier-free clear path described in Section 3.3. With 20 million references in the single-object benchmark, this larger entry size directly multiplies the array footprint. The discrepancy between `clear_path_dyn` and `all` further reflects the fact that `all` routes queue-less references through the `sep` path's separate array, and

the two arrays' capacities are maintained independently with slightly different reservation strategies.

In the multi-object benchmark, the same effect is visible but much smaller (24% at worst for `clear_path_dyn`), because the two million references are spread across multiple collection cycles rather than all being discovered in a single cycle.

6.6 Structural Advantage of Weak Fields

The `weak_fields` variant achieves improvements across all measured GC metrics because its advantage is structural rather than algorithmic: eliminating the `WeakReference` wrapper objects reduces the number of live objects the GC must process in every phase of the collection cycle, not just in reference processing.

In the single-object benchmark, 20 million `WeakReference` objects are heap-allocated Java objects that must be allocated in the young generation, survive young collections, and be promoted to the old generation. Each such object carries a header, a referent field, a queue field, a next field, and a discovered field. The concurrent mark phase must trace all of them, contributing to the longer mark times observed for the `WeakReference` variants. Young-generation promotions add to young-generation collection time, explaining the 44% reduction in the Young Generation (Promote All) metric: `WeakReference` variants promote 20 million wrapper objects in addition to the target objects, while `weak_fields` promotes only the target objects.

The heap usage data confirms this directly: `weak_fields` uses 806 MB versus 1 720 MB for all `WeakReference` variants in the single-object benchmark (53% lower). The 900 MB difference corresponds almost exactly to 20 million `WeakReference` objects each occupying approximately 44–48 bytes (header plus four pointer fields on a 64-bit JVM with compressed references disabled), which is consistent with the ZGC heap layout for small Java objects.

In the multi-object benchmark the absolute effect is smaller because the 2 million references represent a smaller fraction of total heap activity: the 5% heap reduction and 31% old-generation reduction reflect a proportionally smaller eliminated object set relative to the benchmark's total GC workload. The tighter IQRs observed for `weak_fields` across virtually all metrics are also consistent with this structural explanation: with fewer objects to trace, the GC cycle duration is less sensitive to variation in the reference processing workload, resulting in more consistent run-to-run timings.

6.7 Processing Mechanics of Weak Fields

On the processing side, `weak_fields` is mechanically similar to `sep_dyn`. Entries discovered during old-generation marking are stored in a per-worker `ZWeakFieldArray`, the direct analogue of the `ZWeakRefArray` used by the dynamic-array `WeakReference` variants. Like queue-less `WeakReference` objects in `sep_dyn`, weak-field entries carry no `ReferenceQueue` and therefore bypass the enqueue stage entirely: once the referent is found to be unreachable, the annotated field is zeroed without creating a pending-list entry.

This structural parallel is reflected in the non-strong processing times. `weak_fields` achieves a 51% reduction in the single-object benchmark, compared to `sep_dyn`'s 42% and `dyn_only`'s 36%. The somewhat better figure for `weak_fields` is consistent with the simpler per-entry structure: annotated fields are stored and cleared directly, without the `WeakReference` object header, queue field, next field, and discovered field.

The result does not reach the 81% reductions of `clear_path_dyn` and `all` for two reasons. First, the `ZWeakFieldArray` does not currently pre-fetch the referent value during discovery, so clearing the field still requires a load-barrier access, analogously to `dyn_only` rather than `clear_path_dyn`. Second, JVM-internal `WeakReference` objects (used for class-loader metadata, interned strings, and internal caches) still go through the standard pipeline regardless of whether application code uses `@weak` fields, providing a floor below which the non-strong metric cannot fall.

6.8 Limitations

Several aspects of the evaluation warrant care in interpreting the results.

Benchmark representativeness. Both benchmarks are custom microbenchmarks designed to isolate reference processing overhead. The single-object benchmark concentrates its entire weak-reference load into a single GC cycle, and the multi-object benchmark uses a fixed reference population with a regular promotion pattern. Neither benchmark captures the full diversity of real-world weak-reference usage, such as dynamic cache eviction, concurrent insertion and removal, or workloads with a mixture of queue-backed and queue-less references. The strong improvements observed on the non-strong processing metric may therefore represent an upper bound on what would be seen in a more heterogeneous production workload.

Single hardware platform. All measurements were collected on a single UPPMAX cluster configuration (see ??). The results may not generalise to different CPU micro-architectures, NUMA configurations, or cache hierarchy sizes. In particular, the memory bandwidth pressure hypothesis for the concurrent-mark regression is architecture-dependent.

Correctness coverage. The correctness of the implementation was verified through the standard OpenJDK test suite and manual inspection, not through formal verification or an exhaustive stress suite for the weak-field annotation. In particular, the compiler and interpreter support for `@weak` annotated fields is complex and spans three execution tiers; corner cases in tier-to-tier deoptimisation paths may not have been exercised.

Unconfirmed causal attributions. The concurrent-mark regression in single-object dynamic-array variants has a plausible explanation in memory-bandwidth pressure (see Section 6.4), but confirming it would require additional instrumentation or targeted experiments to distinguish the proposed mechanism from other confounders.

Weak-field generality. The `weak_fields` implementation does not support young-generation objects or queue-based callbacks, meaning it cannot serve as a drop-in replacement for all uses of `WeakReference`. The evaluation therefore compares it only against the subset of `WeakReference` usage that does not require queue callbacks, which corresponds to the same use case targeted by the `WeakReference` pipeline optimisations.

6.9 Future Work

Several directions emerge naturally from this work.

Production workload evaluation. The most immediate extension would be to evaluate the optimised variants on real-world applications with diverse weak-reference usage patterns, such as application servers or build tools that use `WeakHashMap` extensively. A production trace would reveal whether the 81% non-strong processing improvement translates into meaningful end-to-end latency improvements in a multi-threaded workload where reference processing competes with other GC work.

Investigating the concurrent-mark regression. The elevated concurrent-mark times in dynamic-array variants in the single-object benchmark should be investigated with targeted profiling. Instrumenting ZGC to report cache-miss rates and memory bandwidth consumption during the concurrent mark phase, with and without large retained arrays, would either confirm or refute the memory-bandwidth-pressure hypothesis proposed in Section 6.4. If confirmed, mitigations such as releasing array capacity at the end of a cycle rather than retaining it would be worth exploring.

Extending the clear-path optimisation. The specialised clear path is currently limited to old-generation `WeakReference` objects without a `ReferenceQueue`. Extending it to cover `SoftReference` and `PhantomReference` objects in the queue-less case, or to handle cross-generational references more efficiently, could yield additional reductions in non-strong processing time. The safety argument for omitting the CAS depends on the

constrained update model described in Section 3.3, and it would need to be re-examined for each additional reference type.

Weak-field support for young-generation objects and callbacks. The current weak-field implementation skips young-generation fields and provides no queue-based notification mechanism. Adding young-generation support would require careful integration with the young-generation marking and remembered-set machinery, as weak-field discovery currently runs only during old-generation marking. Adding optional callback support would make `@weak` a closer replacement for the full `WeakReference` API and would allow a broader comparison across more use cases.

Upstream integration. The skip-enqueue separation, the dynamic array, and the clear-path optimisation are all independent changes with well-defined correctness arguments. Submitting one or more of them to the OpenJDK project for review would allow the broader community to evaluate the trade-offs and potentially integrate the improvements that pass the upstream quality bar. The JDK-8029205 enhancement request provides a natural entry point for such a submission.

Array capacity management strategies. The current strategy of retaining array capacity between GC cycles (keeping enough capacity for the previous cycle's count) is simple but may over-allocate on bursty workloads. Alternative strategies, such as exponential decay of reserved capacity or cycle-to-cycle capacity smoothing, could reduce the steady-state auxiliary memory footprint without significantly increasing reallocation frequency.

7. Related Work

Directly published work on optimising ZGC’s weak-reference pipeline is limited; the closest relevant material is therefore a combination of runtime documentation, implementation sources, and cross-language designs for weak semantics. This chapter positions the thesis against those adjacent designs. The aim is not to claim implementation equivalence between runtimes, but to clarify which trade-offs recur across language ecosystems when weak reachability, callback delivery, and runtime overhead have to coexist.

7.1 Java and the JVM

Java’s reference API explicitly combines weak semantics with optional queue-based notification through the `ReferenceQueue` mechanism [6, 8, 9]. This is a richer contract than a bare weak pointer, because it supports both non-owning reachability and callback-style handoff to application code. The design is flexible, but it also means that the runtime has to maintain queue metadata and pending-list machinery even for workloads that only need liveness probing through `WeakReference.get()`.

The OpenJDK enhancement request JDK-8029205 documents this specific inefficiency by observing unnecessary pending-list traversal for references without a `ReferenceQueue` [12]. That issue does not itself present an implementation or an evaluation, but it establishes that the overhead targeted in this thesis is recognised within the OpenJDK ecosystem. In that sense, the present work contributes by taking a documented inefficiency and following it through implementation, benchmarking, and analysis in generational ZGC.

The broader garbage-collection literature also helps frame the design space. Jones et al. emphasise that GC performance depends not only on asymptotic algorithmic behaviour but also on object representation, locality, and the amount of metadata the runtime must maintain for each managed object [24]. That perspective is directly relevant here: the weak-field exploration is not merely an alternative API shape, but a test of whether moving weak semantics into ordinary object layout changes GC cost in a more fundamental way than improving the existing `WeakReference` pipeline.

Finalisation literature provides an additional point of comparison. Boehm argues that lifecycle callbacks and finalisation-style mechanisms introduce semantic and implementation complexity that often exceeds their apparent API simplicity [25]. Although weak references are not the same as finalisers, Java’s queue-based reference handling shares the same broad challenge of coordinating GC decisions with asynchronous callback-style processing.

7.2 Go

Go exposes weak semantics through the standard-library package `weak`, which provides weak pointers that do not by themselves keep referents alive [26]. This design enables non-owning associations in cache-like or metadata-oriented structures without introducing strong retention edges. Crucially, `weak.Pointer` is callback-less by design: the only way to determine whether a referent is still live is through an explicit call to `.Value()`, which returns `nil` once the referent has been collected [26]. Weak reachability and cleanup notification are entirely separate concerns in Go's model.

For lifecycle callbacks, Go documents finalisation support through runtime facilities such as `SetFinalizer` [27]. Importantly, Go's documentation emphasises that finalisers are not prompt resource-management mechanisms and should not be used as the primary correctness path for releasing resources.

Go therefore presents weak semantics as an explicit, callback-less non-owning reference mechanism and treats finalisation as an auxiliary facility with carefully constrained expectations – a deliberate separation rather than an oversight.

7.3 C++ and .NET

In C++, `std::weak_ptr` represents non-owning reachability within the reference-counted `shared_ptr/weak_ptr` ownership model [28]. Its semantics differ from Java's GC-integrated weak references, but the design illustrates an important contrast: weak semantics can be expressed without a queue-based callback mechanism. The caller invokes `.lock()` to obtain a temporary strong pointer and checks whether it is null. There is no notification on clearing, and cleanup of associated resources is handled entirely through separate mechanisms such as RAII destructors.

.NET exposes a `WeakReference<T>` API and also provides `ConditionalWeakTable` for key-associated metadata that should not keep keys alive [29, 30]. Like C++, `WeakReference<T>` is probe-based: callers use `TryGetTarget()` to check liveness; there is no notification callback when the referent is cleared. Compared with Java's single `Reference/ReferenceQueue` framework, .NET's API makes the use-case distinction more explicit by offering separate types for the liveness-probing and side-metadata patterns.

7.4 Lua and Table-Level Weak Semantics

Lua offers weak semantics directly in table representation: a table can be configured so that its keys, values, or both are weak [31]. This is conceptually

close to the weak-field idea explored in this thesis. Rather than allocating a dedicated wrapper object for each weak relation, the language embeds weak semantics into an existing container or field representation. Lua’s design therefore demonstrates that weak behaviour can be attached to ordinary object structure instead of being mediated by explicit weak-reference objects. As with Go’s `weak.Pointer` and C++’s `std::weak_ptr`, there is no callback when a weak entry is cleared: the entry simply stops being present after the next collection cycle.

7.5 Python

Python’s weak-semantics design, standardised in PEP 205, emphasises two use cases: cache construction and reduction of lifecycle coupling in graph-like object structures [32]. The language provides explicit weak-reference objects, weak mappings, and callback hooks on invalidation. A key contribution of the PEP is its explicit treatment of invalidation strategy and the cost implications of global weak-reference management.

At the API level, Python exposes weak references through the `weakref` module and supports optional callbacks when referents become unreachable [33]. This model is widely used for cache-like structures and registry patterns where identity tracking is needed without extending object lifetime.

7.6 Synthesis

Taken together, these systems expose three recurring design patterns. First, weak semantics is consistently treated as a deliberate programming abstraction rather than as an invisible optimisation. Second, in most ecosystems, weak reachability and cleanup notification are treated as entirely separate concerns. Go’s `weak.Pointer`, C++’s `std::weak_ptr`, .NET’s `WeakReference<T>`, and Lua’s weak table entries all provide callback-less weak semantics, where the caller probes for liveness rather than registering for a notification. Cleanup callbacks, where needed at all, are provided through separate and explicitly more complex mechanisms. Java’s `WeakReference` is notable for coupling both concerns into one object and one processing pipeline, making the two use cases architecturally indistinct at runtime. Third, runtimes differ in whether weak reachability is encoded through wrapper objects, integrated container or field semantics, or ownership metadata – a choice with significant consequences for GC pressure.

Against that background, this thesis occupies a specific niche. The queue-less `WeakReference` optimisations aim to reduce the overhead of the callback-less use case while preserving Java’s existing API and object model: for applications that already choose the one-argument `WeakReference` con-

structor, the optimised pipeline makes the runtime cost of weak reachability closer to the lean, probe-only model that Go, C++, and .NET have long provided. The weak-field mechanism explores a more structural alternative, closer in spirit to Lua's embedded weak semantics than to Java's traditional wrapper-object approach. The comparison between these two lines of work is therefore not only an optimisation study, but also a representation study: it asks whether a substantial part of weak-reference cost arises from how weak semantics is encoded in the heap rather than only from how the reference-processing pipeline is implemented.

8. Conclusion

References

- [1] Oracle, “Available collectors.” <https://docs.oracle.com/en/java/javase/25/gctuning/available-collectors.html>, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-02-24.
- [2] Oracle, “Garbage collector implementation.” <https://docs.oracle.com/en/java/javase/25/gctuning/garbage-collector-implementation.html>, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-04-07.
- [3] OpenJDK, “Jep 333: Zgc: A scalable low-latency garbage collector (experimental).” <https://openjdk.org/jeps/333>, 2018. Accessed: 2026-03-12.
- [4] OpenJDK, “Jep 377: Zgc: A scalable low-latency garbage collector (production).” <https://openjdk.org/jeps/377>, 2020. Accessed: 2026-03-12.
- [5] OpenJDK, “Jep 439: Generational zgc.” <https://openjdk.org/jeps/439>, 2023. Accessed: 2026-03-12.
- [6] Oracle, “java.lang.ref (java se 25).” <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html>, 2025. Accessed: 2026-03-12.
- [7] OpenJDK, “Openjdk source: java.lang.ref.weakreference.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/WeakReference.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [8] OpenJDK, “Openjdk source: java.lang.ref.reference.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/Reference.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [9] OpenJDK, “Openjdk source: java.lang.ref.referencequeue.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/lang/ref/ReferenceQueue.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [10] OpenJDK, “Openjdk source: java.util.weakhashmap.java.” <https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/java/util/WeakHashMap.java>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [11] OpenJDK, “Openjdk source: Zgc reference processor (zreferenceprocessor.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zReferenceProcessor.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [12] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.” <https://bugs.openjdk.org/browse/JDK-8029205>, n.d. Accessed: 2026-02-24.
- [13] OpenJDK, “Openjdk source: Shared reference processor (referenceprocessor.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/shared/referenceProcessor.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.

- [14] U. Drepper, “What every programmer should know about memory,” tech. rep., Red Hat, Inc., 2007. Accessed: 2026-04-07.
- [15] OpenJDK, “Openjdk source: Zgc barrier implementation (zbarrier.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zBarrier.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [16] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981. Accessed: 2026-02-24.
- [17] OpenJDK, “Openjdk source: Zgc address representation (zaddress.hpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zAddress.hpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [18] OpenJDK, “Openjdk source: Zgc worker infrastructure (zworkers.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zWorkers.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [19] OpenJDK, “Openjdk source: Access decorators (accessdecorators.hpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/oops/accessDecorators.hpp>, 2026. GitHub code blob. Accessed: 2026-04-16.
- [20] OpenJDK, “Openjdk source: Reference policy (referencepolicy.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/shared/referencePolicy.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [21] OpenJDK, “Openjdk source: Generational zgc implementation (zgeneration.cpp).” <https://github.com/openjdk/jdk/blob/master/src/hotspot/share/gc/z/zGeneration.cpp>, 2026. GitHub code blob. Accessed: 2026-03-24.
- [22] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd ed., 1997. Section 3.4.2, Algorithm P.
- [23] F. Hammarberg, “Benchmark data: Optimising weak reference processing in the jvm z garbage collector.” Zenodo, 2026. Dataset. DOI to be assigned upon publication.
- [24] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.
- [25] H.-J. Boehm, “Destructors, finalizers, and synchronization,” in *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2003)*, pp. 262–272, ACM, 2003. Accessed: 2026-04-22.
- [26] The Go Authors, “weak package – go standard library documentation.” <https://pkg.go.dev/weak>, 2026. Accessed: 2026-04-22.
- [27] The Go Authors, “runtime.setfinalizer – go standard library documentation.” <https://pkg.go.dev/runtime#SetFinalizer>, 2026. Accessed: 2026-04-22.
- [28] cppreference.com contributors, “std::weak_ptr – cppreference.com.” https://en.cppreference.com/w/cpp/memory/weak_ptr, 2026. Accessed: 2026-04-22.
- [29] Microsoft, “Weakreference class (.net).” <https://learn.microsoft.com/en-us/dotnet/api/system.weakreference?view=net-9.0>, 2026. Accessed: 2026-04-22.
- [30] Microsoft, “Conditionalweaktble<tkey, tvalue> class (.net).” <https://learn.microsoft.com/en-us/dotnet/api/system.runtime.compilerservices.conditionalweaktble-2?view=net-9.0>, 2026. Accessed: 2026-04-22.

- [31] R. Ierusalimsky, L. H. de Figueiredo, and W. Celes, “Lua 5.4 reference manual.” <https://www.lua.org/manual/5.4/>, 2025. Sections 2.5.3 and 2.5.4. Accessed: 2026-04-22.
- [32] F. L. Drake, “Pep 205 – weak references.” <https://peps.python.org/pep-0205/>, 2000. Python Enhancement Proposal. Accessed: 2026-04-22.
- [33] Python Software Foundation, “weakref – weak references (python 3 documentation).” <https://docs.python.org/3/library/weakref.html>, 2026. Accessed: 2026-04-22.

Appendix A.

Usage of Generative Artificial Intelligence Tools

The generative artificial intelligence (GenAI) tool GitHub Copilot was used as support during this thesis. It was used under a student licence via Uppsala University. The tool was used to accelerate practical work and improve writing quality, while all final technical decisions, implementation choices, and evaluations remained under the author's control.

A.1 Use During Implementation

GenAI was used as support during coding of the implementation, primarily for discussing alternative implementation approaches, suggesting code edits through auto-complete, and implementing refactorings of already functioning code.

A.2 Use for Tests and Scripts

GenAI was used to completely generate several test files and automation scripts used in this project, including scripts for benchmarking workflows and result-processing utilities. These generated artefacts were reviewed and validated through repeated execution.

A.3 Use During Writing

GenAI was used as writing support in three main ways:

- Suggesting rephrasing for clarity, tone, and structure.
- Suggesting and correcting LaTeX formatting and fixing compilation or syntax errors.
- Generating first drafts of information-dense sections, especially where the text was based directly on OpenJDK source code and implementation details.

All GenAI-generated text was manually edited and verified against primary sources before inclusion in the final manuscript.

Appendix B.

Source Code

This appendix provides the key source code for the implemented optimisations. The code for the three core optimisations is from the combined (all) configuration variant, which includes all three optimisations. The code for the weak-field mechanism is from the weak_fields configuration variant. Complete sources for all variants are available in the patches/ directory of the project repository.

B.1 Dynamic-array Data Structure

File: zWeakRefArray.hpp

```
1  /*
2   * Copyright (c) 2025, Oracle and/or its affiliates. All rights
   * reserved.
3   * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4   *
5   * This code is free software; you can redistribute it and/or
   * modify it
6   * under the terms of the GNU General Public License version 2
   * only, as
7   * published by the Free Software Foundation.
8   *
9   * This code is distributed in the hope that it will be useful,
   * but WITHOUT
10  * ANY WARRANTY; without even the implied warranty of
   * MERCHANTABILITY or
11  * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
   * License
12  * version 2 for more details (a copy is included in the
   * LICENSE file that
13  * accompanied this code).
14  *
15  * You should have received a copy of the GNU General Public
   * License version
16  * 2 along with this work; if not, write to the Free Software
   * Foundation,
17  * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
```

```

18  *
19  * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
    CA 94065 USA
20  * or visit www.oracle.com if you need additional information
    or have any
21  * questions.
22  */
23
24  #ifndef SHARE_GC_Z_ZWEAKREFARRAY_HPP
25  #define SHARE_GC_Z_ZWEAKREFARRAY_HPP
26
27  #include "gc/z/zAddress.hpp"
28  #include "memory/allocation.hpp"
29  #include "utilities/debug.hpp"
30  #include "utilities/powerOfTwo.hpp"
31  #include "utilities/globalDefinitions.hpp"
32  #include <string.h>
33
34  struct ZWeakRefData {
35      zpointer* referent_field_addr;
36      zaddress* discovered_field_addr;
37      zaddress referent_addr;
38      zpointer referent_field_value;
39  };
40
41
42  class ZWeakRefArray : public AnyObj {
43  private:
44      ZWeakRefData* _data;
45      const size_t _ZWeakRefData_size = sizeof(ZWeakRefData);
46      size_t _length;
47      size_t _capacity;
48
49      void expand_to(size_t new_capacity) {
50          assert(new_capacity > _capacity, "expected growth but %ld <=
            %ld", new_capacity, _capacity);
51
52          ZWeakRefData* new_data = (ZWeakRefData*)AllocateHeap(
              new_capacity * _ZWeakRefData_size, mtGC);
53
54          if (_length > 0) {
55              // Use memcpy for trivially copyable types - much faster
                than element-by-element copy
56              memcpy(new_data, _data, _length * _ZWeakRefData_size);
57          }
58
59          if (_data != nullptr) {
60              FreeHeap(_data);
61          }
62
63          _data = new_data;

```

```

64     _capacity = new_capacity;
65 }
66
67 void grow(size_t min_capacity) {
68     size_t new_capacity = MAX2((size_t) 8, next_power_of_2(
69         min_capacity - 1));
70     expand_to(new_capacity);
71 }
72
73 public:
74     ZWeakRefArray() : _data(nullptr), _length(0), _capacity(0) {}
75
76     ~ZWeakRefArray() {
77         if (_data != nullptr) {
78             FreeHeap(_data);
79             _data = nullptr;
80         }
81
82     void append(const ZWeakRefData& entry) {
83         if (_length >= _capacity) {
84             grow(_length + 1);
85         }
86         _data[_length] = entry;
87         _length++;
88     }
89
90     // Get entry at index
91     const ZWeakRefData& at(size_t index) const {
92         assert(index < _length, "index out of bounds: %ld (length: %
93             ld)", index, _length);
94         return _data[index];
95     }
96
97     // Current length
98     size_t length() const {
99         return _length;
100     }
101
102     // Whether the array is empty
103     bool is_empty() const {
104         return _length == 0;
105     }
106
107     // Current capacity
108     size_t capacity() const {
109         return _capacity;
110     }
111
112     void clear_and_reserve(size_t new_capacity) {
113         if (new_capacity == 0) {

```

```

113 // Special case for clearing to empty - free memory and
    reset
114 if (_data != nullptr) {
115     FreeHeap(_data);
116     _data = nullptr;
117 }
118 _length = 0;
119 _capacity = 0;
120 return;
121 }
122 _length = 0;
123 if (_data != nullptr) {
124     FreeHeap(_data);
125 }
126 _capacity = MAX2((size_t) 8, next_power_of_2(new_capacity -
    1));
127 _data = (ZWeakRefData*)AllocateHeap(_capacity *
    _ZWeakRefData_size, mtGC);
128 }
129
130 // Clear without deallocating
131 void clear() {
132     _length = 0;
133 }
134
135 // Reserve capacity without clearing
136 void reserve(size_t new_capacity) {
137     if (new_capacity > _capacity) {
138         grow(new_capacity);
139     }
140 }
141 };
142
143 #endif // SHARE_GC_Z_ZWEAKREFARRAY_HPP

```

B.2 Reference Processor

B.2.1 Interface

File: zReferenceProcessor.hpp

```

1 /*
2  * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All
    rights reserved.
3  * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4  *
5  * This code is free software; you can redistribute it and/or
    modify it

```



```

6  * under the terms of the GNU General Public License version 2
   * only, as
7  * published by the Free Software Foundation.
8  *
9  * This code is distributed in the hope that it will be useful,
   * but WITHOUT
10 * ANY WARRANTY; without even the implied warranty of
   * MERCHANTABILITY or
11 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
   * License
12 * version 2 for more details (a copy is included in the
   * LICENSE file that
13 * accompanied this code).
14 *
15 * You should have received a copy of the GNU General Public
   * License version
16 * 2 along with this work; if not, write to the Free Software
   * Foundation,
17 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
18 *
19 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
   * CA 94065 USA
20 * or visit www.oracle.com if you need additional information
   * or have any
21 * questions.
22 */
23
24 #ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
25 #define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
26
27 #include "gc/shared/referenceDiscoverer.hpp"
28 #include "gc/z/zAddress.hpp"
29 #include "gc/z/zWeakRefArray.hpp"
30 #include "gc/z/zValue.hpp"
31
32 class ConcurrentGCTimer;
33 class ReferencePolicy;
34 class ZWorkers;
35
36 class ZReferenceProcessor : public ReferenceDiscoverer {
37     friend class ZReferenceProcessorTask;
38
39 private:
40     static const size_t ReferenceTypeCount = REF_PHANTOM + 1;
41     typedef size_t Counters[ReferenceTypeCount];
42
43     ZWorkers* const _workers;
44     ReferencePolicy* _soft_reference_policy;
45     bool _uses_clear_all_soft_reference_policy;
46     ZPerWorker<Counters> _encountered_count;

```

```

47 ZPerWorker<Counters> _discovered_count;
48 ZPerWorker<Counters> _enqueued_count;
49 ZPerWorker<size_t> _encountered_weak_refs_without_queue_count
   ;
50 ZPerWorker<size_t> _discovered_weak_refs_without_queue_count;
51 ZPerWorker<size_t> _cleared_weak_refs_without_queue_count;
52 ZPerWorker<zaddress> _discovered_list;
53 ZContended<zaddress> _pending_list;
54 zaddress _pending_list_tail;
55 ZPerWorker<ZWeakRefArray>
   _discovered_weak_refs_without_queue_arr;
56 ZPerWorker<bool> _array_empty;
57 OopHandle _null_queue_handle;
58
59 bool is_inactive(zaddress reference, oop referent,
   ReferenceType type) const;
60 bool is_strongly_live(oop referent) const;
61 bool is_softly_live(zaddress reference, ReferenceType type)
   const;
62
63 bool should_discover(zaddress reference, ReferenceType type,
   oop referent) const;
64 bool try_make_inactive(zaddress reference, ReferenceType type
   ) const;
65 bool try_make_inactive_fast(const ZWeakRefData& data);
66
67 void discover(zaddress reference, ReferenceType type,
   zaddress referent);
68
69 void verify_empty() const;
70
71 void process_worker_discovered_list(zaddress discovered_list);
72
73 void process_worker_discovered_weak_refs_without_queue(
   ZWeakRefArray& weak_refs_without_queue);
74 void work();
75 void collect_statistics();
76
77 zaddress swap_pending_list(zaddress pending_list);
78
79 inline bool has_reference_queue(zaddress reference);
80
81 void initialize_null_queue_handle();
82 public:
83 ZReferenceProcessor(ZWorkers* workers);
84
85 void set_soft_reference_policy(bool clear_all_soft_references
   );
86 bool uses_clear_all_soft_reference_policy() const;
87

```

```

88 void reset_statistics();
89
90 virtual bool discover_reference(oop reference, ReferenceType
    type);
91 void process_references();
92 void enqueue_references();
93
94 void verify_pending_references();
95
96 void initializeResources();
97 };
98
99 #endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP

```

B.2.2 Implementation Excerpts

The following excerpts from `zReferenceProcessor.cpp` show the constructor and per-worker storage setup, the discovery path with queue-less weak-reference routing, the processing paths, and the queue sentinel initialisation.

Constructor and per-worker storage:

```

1  _soft_reference_policy(nullptr),
2  _uses_clear_all_soft_reference_policy(false),
3  _encountered_count(),
4  _discovered_count(),
5  _enqueued_count(),
6  _encountered_weak_refs_without_queue_count(),
7  _discovered_weak_refs_without_queue_count(),
8  _cleared_weak_refs_without_queue_count(),
9  _discovered_list(zaddress::null),
10 _pending_list(zaddress::null),
11 _pending_list_tail(zaddress::null),
12 _discovered_weak_refs_without_queue_arr(),
13 _array_empty(),
14 _null_queue_handle() {
15
16 _array_empty.set_all(true);
17 log_info(gc, ref) ("Reference Processor created, Config: All
    optimisations");
18 }
19
20 void ZReferenceProcessor::set_soft_reference_policy(bool
    clear_all_soft_references) {
21     static AlwaysClearPolicy always_clear_policy;
22     static LRUMaxHeapPolicy lru_max_heap_policy;
23
24     _uses_clear_all_soft_reference_policy =
        clear_all_soft_references;
25

```

```

26  if (clear_all_soft_references) {
27      _soft_reference_policy = &always_clear_policy;
28  } else {
29      _soft_reference_policy = &lru_max_heap_policy;
30  }
31
32  _soft_reference_policy->setup();
33  }
34
35  bool ZReferenceProcessor::uses_clear_all_soft_reference_policy
36      () const {
37      return _uses_clear_all_soft_reference_policy;
38  }
39
40  bool ZReferenceProcessor::is_inactive(zaddress reference, oop
41      referent, ReferenceType type) const {

```

Discovery path with queue-less routing:

```

1
2  assert (ZHeap::heap()->is_old(reference), "Must be old");
3  assert (is_null(reference_discovered(reference)), "Already
4      discovered");
5
6  // Update statistics
7  if (type == REF_WEAK && !has_reference_queue(reference)) {
8      _discovered_weak_refs_without_queue_count.get()++;
9  }
10 else {
11     _discovered_count.get()[type]++;
12 }
13
14 if (type == REF_WEAK && !has_reference_queue(reference)) {
15     zpointer* const referent_addr =
16         reference_referent_addr_non_vol(reference);
17     zaddress* const discovered_addr = reference_discovered_addr(
18         reference);
19     const zpointer referent_value = *referent_addr;
20
21     // WeakReference with null ReferenceQueue - remember for
22     // special processing
23     ZWeakRefArray& weak_refs_without_queue =
24         _discovered_weak_refs_without_queue_arr.get();
25     ZWeakRefData data = {referent_addr, discovered_addr,
26         referent, referent_value};
27     weak_refs_without_queue.append(data);
28     _array_empty.set(false);
29     reference_set_discovered(reference, reference); // mark as
30         discovered
31 } else {
32     if (type == REF_FINAL) {

```

```

27 // Mark referent (and its reachable subgraph) finalizable.
    This avoids
28 // the problem of later having to mark those objects if
    the referent is
29 // still final reachable during processing.
30 volatile zpointer* const referent_addr =
    reference_referent_addr(reference);
31 ZBarrier::mark_barrier_on_old_oop_field(referent_addr, true
    /* finalizable */);
32 }
33
34 // Add reference to discovered list
35 zaddress* const list = _discovered_list.addr();
36 reference_set_discovered(reference, *list);
37 *list = reference;
38 }
39 }
40
41 bool ZReferenceProcessor::discover_reference(oop reference_obj,
    ReferenceType type) {
42 if (!RegisterReferences) {
43 // Reference processing disabled
44 return false;
45 }
46
47 log_trace(gc, ref) ("Encountered Reference: " PTR_FORMAT " (%s
    )", p2i(reference_obj), reference_type_name(type));
48
49 const zaddress reference = to_zaddress(reference_obj);
50
51 // Update statistics
52 if (type == REF_WEAK && !has_reference_queue(reference)) {
53 _encountered_weak_refs_without_queue_count.get()++;
54 }
55 else {
56 _encountered_count.get()[type]++;
57 }
58
59 if (ZHeap::heap()->is_young(reference)) {
60 // Don't discover young references. Young gen reference
    processing is scary and therefore not supported.
61 return false;
62 }
63
64 volatile zpointer* const referent_addr =
    reference_referent_addr(reference);
65 const zaddress ref_raw_addr = ZBarrier::
    load_barrier_on_oop_field(referent_addr);
66 const oop referent = to_oop(ref_raw_addr);
67
68 if (!should_discover(reference, type, referent)) {

```

```

69     // Not discovered
70     return false;
71 }
72
73 discover(reference, type, ref_raw_addr);
74
75 // Discovered
76 return true;
77 }
78
79 void ZReferenceProcessor::process_worker_discovered_list(
    zaddress discovered_list) {
80     zaddress keep_head = zaddress::null;
81     zaddress keep_tail = zaddress::null;
82
83     // Iterate over the discovered list and unlink them as we go,
        potentially

```

Processing paths for separate discovered lists and dynamic arrays:

```

1     *((zpointer*)data.discovered_field_addr) = color_null(); //
        Mark as dropped
2     if (try_make_inactive_fast(data)) {
3         log_trace(gc, ref) ("\"Enqueued\" Weak Reference without
        Queue");
4         // Update statistics
5         _cleared_weak_refs_without_queue_count.get()++;
6     } else {
7         log_trace(gc, ref) ("Dropped Weak Reference without Queue");
8
9         dropped++;
10    }
11    SuspendibleThreadSet::yield();
12    weak_refs_without_queue.clear_and_reserve(dropped);
13 }
14
15 void ZReferenceProcessor::work() {
16     SuspendibleThreadSetJoiner sts_joiner;
17
18     ZPerWorkerIterator<zaddress> iter(&_discovered_list);
19     ZPerWorkerIterator<ZWeakRefArray> iter_weak_refs(&
        _discovered_weak_refs_without_queue_arr);
20     ZPerWorkerIterator<bool> iter_array_empty(&_array_empty);
21
22     zaddress* list_addr = nullptr;
23     ZWeakRefArray* array_addr = nullptr;
24     bool* array_empty = nullptr;
25
26     for (; iter.next(&list_addr) && iter_weak_refs.next(&
        array_addr) && iter_array_empty.next(&array_empty);) {
27

```

```

28     const address discovered_list = AtomicAccess::xchg(
        list_addr, zaddress::null);
29     const bool has_array = !AtomicAccess::xchg(array_empty, true
        );
30     const bool has_discovered = discovered_list != zaddress::
        null;
31
32     if (has_array) {
33         // Process discovered weak references without queue
34         process_worker_discovered_weak_refs_without_queue(*
            array_addr);
35     }
36     if (has_discovered) {
37         // Process discovered references
38         process_worker_discovered_list(discovered_list);
39     }
40 }
41 }
42
43 void ZReferenceProcessor::verify_empty() const {
44 #ifdef ASSERT
45     ZPerWorkerConstIterator<zaddress> iter(&_discovered_list);
46     for (const zaddress* list; iter.next(&list);) {
47         assert(is_null(*list), "Discovered list not empty");
48     }
49
50     ZPerWorkerConstIterator<ZWeakRefArray> iter_weak_refs(&
        _discovered_weak_refs_without_queue_arr);
51     for (const ZWeakRefArray* array; iter_weak_refs.next(&array)
        ;) {
52         assert(array->is_empty(), "Discovered weak refs without
            queue not empty");
53     }
54
55     assert(is_null(_pending_list.get()), "Pending list not empty"
        );
56 #endif
57 }

```

Queue sentinel initialisation and queue checks:

```

1     return true;
2 }
3
4     const address ref_queue_addr_value = ZBarrier::
        load_barrier_on_oop_field(reference_queue_addr(reference));
5     const oop ref_queue = to_oop(ref_queue_addr_value);
6     bool result = ref_queue != _null_queue_handle.resolve();
7     return result;
8 }
9
10 void ZReferenceProcessor::initialize_null_queue_handle() {

```

```

11 EXCEPTION_MARK;
12 TempNewSymbol class_name = SymbolTable::new_symbol("java/lang
   /ref/ReferenceQueue");
13 Klass* k = SystemDictionary::resolve_or_fail(class_name, true,
   CHECK);
14 InstanceKlass* ik = InstanceKlass::cast(k);
15 ik->initialize(CHECK);
16 fieldDescriptor fd;
17 bool found = ik->find_local_field(SymbolTable::new_symbol("
   NULL_QUEUE"),
18                                     vmSymbols::referencequeue_signature
   (), &fd);
19 assert(found && fd.is_static(), "ReferenceQueue.NULL_QUEUE
   missing");
20 oop null_q = ik->java_mirror()->obj_field(fd.offset());
21 _null_queue_handle = OopHandle(Universe::vm_global(), null_q);
22 }
23
24 void ZReferenceProcessor::initializeResources() {
25   Thread* const current = Thread::current();
26   guarantee(current->is_Java_thread(), "Must be called by a
   JavaThread");
27   guarantee(!current->is_ConcurrentGC_thread(), "Must not be
   called by a GC thread");
28
29   initialize_null_queue_handle();
30 }

```

B.3 Weak Fields

B.3.1 Annotation

File: java/lang/ref/weak.java

```

1  /*
2   * Copyright (c) 2026, Oracle and/or its affiliates. All rights
   reserved.
3   * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4
5   *
6   * This code is free software; you can redistribute it and/or
   modify it
7   * under the terms of the GNU General Public License version 2
   only, as
8   * published by the Free Software Foundation. Oracle designates
   this
9   * particular file as subject to the "Classpath" exception as
   provided
   * by Oracle in the LICENSE file that accompanied this code.

```



```

10  *
11  * This code is distributed in the hope that it will be useful,
    * but WITHOUT
12  * ANY WARRANTY; without even the implied warranty of
    * MERCHANTABILITY or
13  * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
    * License
14  * version 2 for more details (a copy is included in the
    * LICENSE file that
15  * accompanied this code).
16  *
17  * You should have received a copy of the GNU General Public
    * License version
18  * 2 along with this work; if not, write to the Free Software
    * Foundation,
19  * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
20  *
21  * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
    * CA 94065 USA
22  * or visit www.oracle.com if you need additional information
    * or have any
23  * questions.
24  */
25
26 package java.lang.ref;
27
28 import java.lang.annotation.*;
29
30 /**
31  * A field annotation for references that should be treated as
    * weak by the VM.
32  */
33 @Retention(RetentionPolicy.RUNTIME)
34 @Target(ElementType.FIELD)
35 public @interface weak {}

```

B.3.2 Data Structure

File: zWeakFieldArray.hpp

```

1  /*
2  * Copyright (c) 2025, Oracle and/or its affiliates. All rights
    * reserved.
3  * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
4  *
5  * This code is free software; you can redistribute it and/or
    * modify it

```

```

6  * under the terms of the GNU General Public License version 2
   * only, as
7  * published by the Free Software Foundation.
8  *
9  * This code is distributed in the hope that it will be useful,
   * but WITHOUT
10 * ANY WARRANTY; without even the implied warranty of
   * MERCHANTABILITY or
11 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
   * License
12 * version 2 for more details (a copy is included in the
   * LICENSE file that
13 * accompanied this code).
14 *
15 * You should have received a copy of the GNU General Public
   * License version
16 * 2 along with this work; if not, write to the Free Software
   * Foundation,
17 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
18 *
19 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
   * CA 94065 USA
20 * or visit www.oracle.com if you need additional information
   * or have any
21 * questions.
22 */
23
24 #ifndef SHARE_GC_Z_ZWEAKFIELDARRAY_HPP
25 #define SHARE_GC_Z_ZWEAKFIELDARRAY_HPP
26
27 #include "gc/z/zAddress.hpp"
28 #include "memory/allocation.hpp"
29 #include "utilities/debug.hpp"
30 #include "utilities/powerOfTwo.hpp"
31 #include "utilities/globalDefinitions.hpp"
32 #include <string.h>
33
34 struct ZWeakFieldData {
35     volatile zpointer* field_addr;
36     zpointer field_value;
37 };
38
39 // High-performance growable array specifically for storing
   * discovered reference data.
40 // Uses array-of-structs (AoS) layout for better cache locality
   * when processing sequentially.
41 // The stored entry type is configured by the template parameter
   * .
42 //
43 // Performance optimizations:

```

```

44 // - Uses memcpy for bulk copying instead of element-by-element
    loops
45 // - Does not zero newly allocated memory (caller responsible
    for initialization)
46 // - Single clear_and_reserve operation to avoid separate clear
    /reserve calls
47 // - Array-of-structs layout improves cache efficiency for
    sequential processing
48 class ZWeakFieldArray : public AnyObj {
49 private:
50     ZWeakFieldData* _data;
51     const size_t _ZWeakFieldData_size = sizeof(ZWeakFieldData);
52     size_t _length;
53     size_t _capacity;
54
55     void expand_to(size_t new_capacity) {
56         assert(new_capacity > _capacity, "expected growth but %ld <=
            %ld", new_capacity, _capacity);
57
58         ZWeakFieldData* new_data = (ZWeakFieldData*)AllocateHeap(
            new_capacity * _ZWeakFieldData_size, mtGC);
59
60         if (_length > 0) {
61             // Use memcpy for trivially copyable types - much faster
            than element-by-element copy
62             memcpy(new_data, _data, _length * _ZWeakFieldData_size);
63         }
64
65         if (_data != nullptr) {
66             FreeHeap(_data);
67         }
68
69         _data = new_data;
70         _capacity = new_capacity;
71     }
72
73     void grow(size_t min_capacity) {
74         size_t new_capacity = MAX2((size_t) 8, next_power_of_2(
            min_capacity - 1));
75         expand_to(new_capacity);
76     }
77
78 public:
79     ZWeakFieldArray() : _data(nullptr), _length(0), _capacity(0)
        {}
80
81     ~ZWeakFieldArray() {
82         if (_data != nullptr) {
83             FreeHeap(_data);
84             _data = nullptr;
85         }

```

```

86     }
87
88     void append(const ZWeakFieldData& entry) {
89         if (_length >= _capacity) {
90             grow(_length + 1);
91         }
92         _data[_length] = entry;
93         _length++;
94     }
95
96     // Get entry at index
97     const ZWeakFieldData& at(size_t index) const {
98         assert(index < _length, "index out of bounds: %ld (length: %
99             ld)", index, _length);
100         return _data[index];
101     }
102
103     // Current length
104     size_t length() const {
105         return _length;
106     }
107
108     // Whether the array is empty
109     bool is_empty() const {
110         return _length == 0;
111     }
112
113     // Current capacity
114     size_t capacity() const {
115         return _capacity;
116     }
117
118     void clear_and_reserve(size_t new_capacity) {
119         if (new_capacity == 0) {
120             // Special case for clearing to empty - free memory and
121             reset
122             if (_data != nullptr) {
123                 FreeHeap(_data);
124                 _data = nullptr;
125             }
126             _length = 0;
127             _capacity = 0;
128             return;
129         }
130         if (_data != nullptr) {
131             FreeHeap(_data);
132         }
133         _capacity = MAX2((size_t) 8, next_power_of_2(new_capacity -
134             1));

```

```

133     _data = (ZWeakFieldData*)AllocateHeap(_capacity *
134         _ZWeakFieldData_size, mtGC);
135 }
136
137 // Clear without deallocating
138 void clear() {
139     _length = 0;
140 }
141
142 // Reserve capacity without clearing
143 void reserve(size_t new_capacity) {
144     if (new_capacity > _capacity) {
145         grow(new_capacity);
146     }
147 };
148
149 #endif // SHARE_GC_Z_ZWEAKFIELDARRAY_HPP

```

B.3.3 Reference Processor

File: zReferenceProcessor.hpp

```

1  /*
2   * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All
3   * rights reserved.
4   *
5   * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
6   *
7   * This code is free software; you can redistribute it and/or
8   * modify it
9   * under the terms of the GNU General Public License version 2
10  * only, as
11  * published by the Free Software Foundation.
12  *
13  * This code is distributed in the hope that it will be useful,
14  * but WITHOUT
15  * ANY WARRANTY; without even the implied warranty of
16  * MERCHANTABILITY or
17  * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public
18  * License
19  * version 2 for more details (a copy is included in the
20  * LICENSE file that
21  * accompanied this code).
22  *
23  * You should have received a copy of the GNU General Public
24  * License version
25  * 2 along with this work; if not, write to the Free Software
26  * Foundation,

```

```

17  * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
18
19  * Please contact Oracle, 500 Oracle Parkway, Redwood Shores,
    CA 94065 USA
20  * or visit www.oracle.com if you need additional information
    or have any
21  * questions.
22  */
23
24  #ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
25  #define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
26
27  #include "gc/shared/referenceDiscoverer.hpp"
28  #include "gc/z/zAddress.hpp"
29  #include "gc/z/zWeakFieldArray.hpp"
30  #include "gc/z/zValue.hpp"
31
32  class ConcurrentGCTimer;
33  class ReferencePolicy;
34  class ZWorkers;
35
36  class ZReferenceProcessor : public ReferenceDiscoverer {
37  friend class ZReferenceProcessorTask;
38
39 private:
40     static const size_t ReferenceTypeCount = REF_PHANTOM + 1;
41     typedef size_t Counters[ReferenceTypeCount];
42
43     ZWorkers* const      _workers;
44     ReferencePolicy*     _soft_reference_policy;
45     bool                 _uses_clear_all_soft_reference_policy;
46     ZPerWorker<Counters> _encountered_count;
47     ZPerWorker<Counters> _discovered_count;
48     ZPerWorker<Counters> _enqueued_count;
49     ZPerWorker<size_t>   _encountered_weak_fields_count;
50     ZPerWorker<size_t>   _discovered_weak_fields_count;
51     ZPerWorker<size_t>   _cleared_weak_fields_count;
52     ZPerWorker<zaddress> _discovered_list;
53     ZContended<zaddress> _pending_list;
54     zaddress             _pending_list_tail;
55     ZPerWorker<ZWeakFieldArray> _discovered_weak_fields;
56     ZPerWorker<bool>      _discovered_weak_fields_empty;
57
58     bool is_inactive(zaddress reference, oop referent,
59                     ReferenceType type) const;
60     bool is_strongly_live(oop referent) const;
61     bool is_softly_live(zaddress reference, ReferenceType type)
        const;

```

```

62  bool should_discover(zaddress reference, ReferenceType type)
        const;
63  bool should_discover_weak_field(zpointer field_value, volatile
        zpointer* field_addr) const;
64  bool try_make_inactive(zaddress reference, ReferenceType type
        ) const;
65  bool try_make_inactive_fast(const ZWeakFieldData& data);
66
67  void discover(zaddress reference, ReferenceType type);
68
69  void verify_empty() const;
70
71  void process_worker_discovered_list(zaddress discovered_list);
72
73  void process_worker_discovered_weak_fields(ZWeakFieldArray&
        weak_fields);
74  void work();
75  void collect_statistics();
76
77  zaddress swap_pending_list(zaddress pending_list);
78 public:
79  ZReferenceProcessor(ZWorkers* workers);
80
81  bool discover_weak_field(volatile zpointer* field_addr);
82
83  void set_soft_reference_policy(bool clear_all_soft_references
        );
84  bool uses_clear_all_soft_reference_policy() const;
85
86  void reset_statistics();
87
88  virtual bool discover_reference(oop reference, ReferenceType
        type);
89  void process_references();
90  void enqueue_references();
91
92  void verify_pending_references();
93 };
94
95 #endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP

```

The following excerpts from `zReferenceProcessor.cpp` show the discovery filtering, the discovery function, and the processing function.

Discovery filtering:

```

1  return true;
2  }
3
4  bool ZReferenceProcessor::should_discover_weak_field(zpointer
        referent_ptr, volatile zpointer* field_addr) const {

```

```

5  if (ZPointer::is_mark_good(referent_ptr)) {
6      // Field points to a strongly live object, no need to
        discover
7      return false;
8  }
9
10 if (is_null_any(referent_ptr)) {
11     // Field is null but not yet marked
12     AtomicAccess::cmpxchg(field_addr, referent_ptr, color_null(),
        memory_order_relaxed);
13     return false;
14 }

```

Weak-field discovery:

```

1  return false;
2  }
3
4  log_trace(gc, ref) ("Encountered Reference: " PTR_FORMAT " (%s
    )", p2i(reference_obj), reference_type_name(type));
5
6  const zaddress reference = to_zaddress(reference_obj);
7
8  // Update statistics
9  _encountered_count.get()[type]++;
10
11 if (!should_discover(reference, type)) {
12     // Not discovered
13     return false;
14 }
15
16 discover(reference, type);
17
18 // Discovered
19 return true;
20 }
21
22 bool ZReferenceProcessor::discover_weak_field(volatile zpointer*
    field_addr) {
23     if (!RegisterReferences) {
24         // Reference processing disabled
25         return false;
26     }
27
28     log_trace(gc, ref) ("Encountered Weak Reference Field: "
        PTR_FORMAT, p2i(field_addr));
29
30     // Update statistics
31     _encountered_weak_fields_count.get()++;
32
33     if (ZHeap::heap()->is_young(to_zaddress(p2i(field_addr)))) {

```



```

34 // Don't discover young references. Young gen reference
    processing is scary and therefore not supported.
35 return false;

```

Weak-field processing:

```

1  const ReferenceType type = reference_type(reference);
2  const zaddress next = reference_discovered(reference);
3  reference_set_discovered(reference, zaddress::null);
4
5  if (try_make_inactive(reference, type)) {
6      // Keep reference
7      log_trace(gc, ref) ("Enqueued Reference: " PTR_FORMAT " (%s
          ), untype(reference), reference_type_name(type));
8
9      // Update statistics
10     _enqueued_count.get()[type]++;
11
12     list_append(keep_head, keep_tail, reference);
13 } else {
14     // Drop reference
15     log_trace(gc, ref) ("Dropped Reference: " PTR_FORMAT " (%s
        ), untype(reference), reference_type_name(type));
16 }
17
18 reference = next;
19 SuspendibleThreadSet::yield();
20 }
21
22 // Prepend discovered references to internal pending list
23
24 // Anything kept on the list?
25 if (!is_null(keep_head)) {
26     const zaddress old_pending_list = AtomicAccess::xchg(
        _pending_list.addr(), keep_head);
27
28     // Concatenate the old list
29     reference_set_discovered(keep_tail, old_pending_list);
30
31     if (is_null(old_pending_list)) {
32         // Old list was empty. First to prepend to list, record
            tail
33         _pending_list_tail = keep_tail;
34     } else {
35         assert(ZHeap::heap()->is_old(old_pending_list), "Must be
            old");
36     }
37 }
38 }
39
40 void ZReferenceProcessor::process_worker_discovered_weak_fields(
    ZWeakFieldArray& weak_fields) {

```

```
41 | size_t dropped = 0;
```

B.3.4 Marking Closure

The following excerpt from `zMark.cpp` shows how weak-annotated fields are detected and diverted to the weak-field discovery path during object marking.

Field annotation check and discovery diversion:

```
1 | bool is_weak_annotated_field(oop* p) const {
2 |     if (_current_instance_klass == nullptr) {
3 |         return false;
4 |     }
5 |
6 |     const uintptr_t addr = (uintptr_t)p;
7 |     if (addr < _current_obj_start || addr >= _current_obj_end) {
8 |         return false;
9 |     }
10 |
11 |     const int offset = (int)(addr - _current_obj_start);
12 |     fieldDescriptor fd;
13 |     if (!_current_instance_klass->find_field_from_offset(offset,
14 |         false /* is_static */, &fd)) {
15 |         return false;
16 |     }
17 |     return fd.is_weak();
18 | }
19 |
20 | const bool _visit_metadata;
21 |
22 | public:
23 | ZMarkBarrierFollowOopClosure()
24 |     : OopIterateClosure(discoverer()),
25 |       _current_instance_klass(nullptr),
26 |       _current_obj_start(0),
27 |       _current_obj_end(0),
28 |       _visit_metadata(visit_metadata()) {}
29 |
30 | void set_iteration_object(oop obj) {
31 |     if (obj->klass()->is_instance_klass()) {
32 |         _current_instance_klass = InstanceKlass::cast(obj->klass())
33 |             );
34 |         _current_obj_start = cast_from_oop<uintptr_t>(obj);
35 |         _current_obj_end = _current_obj_start + (obj->size() *
36 |             HeapWordSize);
37 |     } else {
38 |         _current_instance_klass = nullptr;
39 |         _current_obj_start = 0;
40 |         _current_obj_end = 0;
```

```

39     }
40 }
41
42 void clear_iteration_object() {
43     _current_instance_klass = nullptr;
44     _current_obj_start = 0;
45     _current_obj_end = 0;
46 }
47
48 virtual void do_oop(oop* p) {
49     if (!finalizable && is_weak_annotated_field(p)) {
50         if (ZGeneration::old()->discover_weak_field((volatile
51             zpointer*)p)) {
52             return;
53         }
54
55         switch (generation) {
56             case ZGenerationIdOptional::young:
57                 ZBarrier::mark_barrier_on_young_oop_field((volatile
58                     zpointer*)p);
59                 break;
60             case ZGenerationIdOptional::old:
61                 ZBarrier::mark_barrier_on_old_oop_field((volatile zpointer
62                     *)p, finalizable);
63                 break;
64             case ZGenerationIdOptional::none:
65                 ZBarrier::mark_barrier_on_oop_field((volatile zpointer*)p,
66                     finalizable);
67                 break;
68         }
69     }
70 }

```

B.3.5 Compiler and Interpreter Support

The following excerpts show how the @weak decorator is applied in each execution tier. In all three tiers, the pattern is the same: if the field carries the weak flag and ZGC is active, ON_WEAK_OOP_REF is added to the decorator set before the access is performed.

C2 (opto/parse3.cpp) – field load and store:

```

1 void Parse::do_get_xxx(Node* obj, ciField* field, bool is_field)
2 {
3     BasicType bt = field->layout_type();
4
5     // Does this field have a constant value? If so, just push the
6     // value.
7     if (field->is_constant() &&
8         // Keep consistent with types found by ciTypeFlow: for an

```

```

7 // unloaded field type, ciTypeFlow::StateVector::
  do_getstatic()
8 // speculates the field is null. The code in the rest of
  this
9 // method does the same. We must not bypass it and use a
  non
10 // null constant here.
11 (bt != T_OBJECT || field->type()->is_loaded())) {
12 // final or stable field
13 Node* con = make_constant_from_field(field, obj);
14 if (con != nullptr) {
15     push_node(field->layout_type(), con);
16     return;
17 }
18 }
19
20 ciType* field_klass = field->type();
21 bool is_vol = field->is_volatile();
22
23 // Compute address and memory type.
24 int offset = field->offset_in_bytes();
25 const TypePtr* adr_type = C->alias_type(field)->adr_type();
26 Node *adr = basic_plus_adr(obj, obj, offset);
27 assert(C->get_alias_index(adr_type) == C->get_alias_index(
  _gvn.type(adr)->isa_ptr()),
28     "slice of address and input slice don't match");
29
30 // Build the resultant type of the load
31 const Type *type;
32
33 bool must_assert_null = false;
34
35 DecoratorSet decorators = IN_HEAP;
36 decorators |= is_vol ? MO_SEQ_CST : MO_UNORDERED;
37
38 bool is_obj = is_reference_type(bt);
39
40 if (is_obj) {
41     if (UseZGC && field->is_weak()) {
42         decorators |= ON_WEAK_OOP_REF;
43     }
44     if (!field->type()->is_loaded()) {
45         type = TypeInstPtr::BOTTOM;
46         must_assert_null = true;
47     } else if (field->is_static_constant()) {
48         // This can happen if the constant oop is non-perm.
49         ciObject* con = field->constant_value().as_object();
50         // Do not "join" in the previous type; it doesn't add
          value,
51         // and may yield a vacuous result if the field is of
          interface type.

```

```

52     if (con->is_null_object()) {
53         type = TypePtr::NULL_PTR;
54     } else {
55         type = TypeOopPtr::make_from_constant(con->isa_oopptr());
56     }
57     assert(type != nullptr, "field singleton type must be
        consistent");
58 } else {
59     type = TypeOopPtr::make_from_klass(field_klass->as_klass());
60 }
61 } else {
62     type = Type::get_const_basic_type(bt);
63 }
64
65 Node* ld = access_load_at(obj, adr, adr_type, type, bt,
        decorators);
66 if (is_obj && UseZGC && field->is_weak()) {
67     insert_mem_bar(Op_MemBarCPUOrder);
68 }
69
70 // Adjust Java stack
71 if (type2size[bt] == 1)
72     push(ld);
73 else
74     push_pair(ld);
75
76 if (must_assert_null) {
77     // Do not take a trap here. It's possible that the program
78     // will never load the field's class, and will happily see
79     // null values in this field forever. Don't stumble into a
80     // trap for such a program, or we might get a long series
81     // of useless recompilations. (Or, we might load a class
82     // which should not be loaded.) If we ever see a non-null
83     // value, we will then trap and recompile. (The trap will
84     // not need to mention the class index, since the class will
85     // already have been loaded if we ever see a non-null value
86     //.)
87     // uncommon_trap(iter().get_field_signature_index());
88     if (PrintOpto && (Verbose || WizardMode)) {
89         method()->print_name(); tty->print_cr(" asserting nullness
            of field at bci: %d", bci());
90     }
91     if (C->log() != nullptr) {
92         C->log()->elem("assert_null reason='field' klass='%d'",
            C->log()->identify(field->type()));
93     }
94     // If there is going to be a trap, put it at the next
95     // bytecode:
96     set_bci(iter().next_bci());
97     null_assert(peek());

```

```

97     set_bci(iter().cur_bci()); // put it back
98 }
99 }
100
101 void Parse::do_put_xxx(Node* obj, ciField* field, bool is_field)
102 {
103     bool is_vol = field->is_volatile();
104
105     // Compute address and memory type.
106     int offset = field->offset_in_bytes();
107     const TypePtr* adr_type = C->alias_type(field)->adr_type();
108     Node* adr = basic_plus_adr(obj, obj, offset);
109     assert(C->get_alias_index(adr_type) == C->get_alias_index(
110         _gvn.type(adr)->isa_ptr()),
111         "slice of address and input slice don't match");
112     BasicType bt = field->layout_type();
113     // Value to be stored
114     Node* val = type2size[bt] == 1 ? pop() : pop_pair();
115
116     DecoratorSet decorators = IN_HEAP;
117     decorators |= is_vol ? MO_SEQ_CST : MO_UNORDERED;
118
119     bool is_obj = is_reference_type(bt);
120
121     if (is_obj) {
122         if (UseZGC && field->is_weak()) {
123             decorators |= ON_WEAK_OOP_REF;
124         }
125     }
126
127     // Store the value.
128     const Type* field_type;
129     if (!field->type()->is_loaded()) {
130         field_type = TypeInstPtr::BOTTOM;
131     } else {
132         if (is_obj) {
133             field_type = TypeOopPtr::make_from_klass(field->type()->
134                 as_klass());
135         } else {
136             field_type = Type::BOTTOM;
137         }
138     }
139     access_store_at(obj, adr, adr_type, val, field_type, bt,
140         decorators);
141
142     if (is_field) {
143         // Remember we wrote a volatile field.
144         // For not multiple copy atomic cpu (ppc64) a barrier should
145         be issued
146         // in constructors which have such stores. See do_exits() in
147         parse1.cpp.

```

```

142     if (is_vol) {
143         set_wrote_volatile(true);
144     }
145     set_wrote_fields(true);
146
147     // If the field is final, the rules of Java say we are in <
148     init> or <clinit>.
149     // If the field is @Stable, we can be in any method, but we
150     only care about
151     // constructors at this point.
152     //
153     // Note the presence of writes to final/@Stable non-static
154     fields, so that we
155     // can insert a memory barrier later on to keep the writes
156     from floating
157     // out of the constructor.
158     if (field->is_final() || field->is_stable()) {

```

C1 (c1/c1_LIRGenerator.cpp) – field load decorator:

```

1     }
2
3     DecoratorSet decorators = IN_HEAP;
4     if (is_volatile) {
5         decorators |= MO_SEQ_CST;
6     }
7     if (needs_patching) {
8         decorators |= C1_NEEDS_PATCHING;
9     } else if (UseZGC && field_type == T_OBJECT && x->field()->
10         is_weak()) {
11         decorators |= ON_WEAK_OOP_REF;
12     }
13
14     LIR_Opr result = rlock_result(x, field_type);
15     access_load_at(decorators, field_type,
16         object, LIR_OprFact::intConst(x->offset()), result,
17         info ? new CodeEmitInfo(info) : nullptr, info);

```

C1 (c1/c1_LIRGenerator.cpp) – field store decorator:

```

1
2     DecoratorSet decorators = IN_HEAP;
3     if (is_volatile) {
4         decorators |= MO_SEQ_CST;
5     }
6     if (needs_patching) {
7         decorators |= C1_NEEDS_PATCHING;
8     } else if (UseZGC && field_type == T_OBJECT && x->field()->
9         is_weak()) {
10         decorators |= ON_WEAK_OOP_REF;

```

```

10     }
11
12     access_store_at(decorators, field_type, object, LIR_OprFact::
        intConst(x->offset()),
13         value.result(), info != nullptr ? new
            CodeEmitInfo(info) : nullptr, info);
14 }
15
16 void LIRGenerator::do_StoreIndexed(StoreIndexed* x) {
17     assert(x->is_pinned(), "");
18     bool needs_range_check = x->compute_needs_range_check();

```

Template Interpreter (cpu/x86/templateTable_x86.cpp) – getfield weak path:

```

1  __ bind(notBool);
2  __ cmpl(tos_state, atos);
3  __ jcc(Assembler::notEqual, notObj);
4  // atos
5  if (UseZGC) {
6      __ testl(flags, (1 << ResolvedFieldEntry::is_weak_shift));
7      __ jcc(Assembler::zero, strongOopLoad);
8      // Weak load path
9      do_oop_load(_masm, field, rax, ON_WEAK_OOP_REF);
10     __ push(atos);
11     // Don't rewrite bytecode for weak references, the fast path
        is only for strong loads
12     __ jmp(Done);
13     __ bind(strongOopLoad);
14 }
15 do_oop_load(_masm, field, rax);
16 __ push(atos);
17 if (!is_static && rc == may_rewrite) {
18     patch_bytecode(Bytecodes::_fast_agetfield, bc, rbx);
19 }
20 __ jmp(Done);

```

Template Interpreter (cpu/x86/templateTable_x86.cpp) – putfield weak path:

```

1  // atos
2  {
3      if (UseZGC) {
4          // Test weak flag before pop(atos) overwrites flags (rax)
5          __ testl(flags, (1 << ResolvedFieldEntry::is_weak_shift));
6          __ jcc(Assembler::notZero, strongOopStore);
7          // Weak store path
8          __ pop(atos);
9          if (!is_static) pop_and_check_object(obj);
10         do_oop_store(_masm, field, rax, ON_WEAK_OOP_REF);
11         // Don't rewrite bytecode for weak references, the fast
            path is only for strong stores

```



```
12     __ jmp(Done);
13     // Strong store path
14     __ bind(strongOopStore);
15 }
16
17 __ pop(atos);
18 if (!is_static) pop_and_check_object(obj);
19 do_oop_store(_masm, field, rax);
20 if (!is_static && rc == may_rewrite) {
21     patch_bytecode(Bytecodes::_fast_aputfield, bc, rbx, true,
22                     byte_no);
23 }
24 __ jmp(Done);
25 }
```

Appendix C.

Per-Variant Patch Details

The following tables list every modified or added file in each patch variant, together with its insertion and deletion counts relative to the upstream OpenJDK master branch. New files (not present in the upstream source) are listed with their total line count and no deletion count. For variants that use the shared base patch, its files are listed first under a “Base patch” heading within each table. The `dyn_only` and `weak_fields` variants do not include the base patch.

C.1 all

Table C.1: Files changed in the *all* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+223	–32
gc/z/zReferenceProcessor.hpp	+31	–16
gc/z/zWeakRefArray.hpp (new)	+143	—
Total	+436	–65

C.2 clear_path_dyn

Table C.2: Files changed in the *clear_path_dyn* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+224	–53
gc/z/zReferenceProcessor.hpp	+30	–17
gc/z/zWeakRefArray.hpp (new)	+155	—
Total	+448	–87

C.3 clear_path_only

Table C.3: Files changed in the *clear_path_only* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+172	–22
gc/z/zReferenceProcessor.hpp	+22	–5
Total	+233	–44

C.4 clear_path_sep

Table C.4: Files changed in the *clear_path_sep* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+193	–21
gc/z/zReferenceProcessor.hpp	+24	–5
Total	+256	–43

C.5 dyn_only

Table C.5: Files changed in the *dyn_only* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
gc/z/zReferenceProcessor.cpp	+44	–28
gc/z/zReferenceProcessor.hpp	+17	–15
gc/z/zWeakRefArray.hpp (new)	+149	—
Total	+210	–43

C.6 sep_dyn

Table C.6: Files changed in the *sep_dyn* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0

File	+	–
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+182	–27
gc/z/zReferenceProcessor.hpp	+28	–14
gc/z/zWeakRefArray.hpp (new)	+149	—
Total	+398	–58

C.7 sep_only

Table C.7: Files changed in the *sep_only* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
<i>Base patch</i>		
gc/shared/collectedHeap.hpp	+4	–0
gc/z/zCollectedHeap.cpp	+9	–0
gc/z/zCollectedHeap.hpp	+1	–0
gc/z/zGeneration.hpp	+1	–0
gc/z/zGeneration.inline.hpp	+4	–0
gc/z/zStat.cpp	+14	–8
gc/z/zStat.hpp	+2	–1
runtime/threads.cpp	+4	–8
<i>Variant-specific patch</i>		
gc/z/zReferenceProcessor.cpp	+157	–20
gc/z/zReferenceProcessor.hpp	+17	–5
Total	+213	–42

C.8 weak_fields

Table C.8: Files changed in the *weak_fields* variant, with insertion and deletion counts relative to upstream OpenJDK.

File	+	–
cpu/x86/templateTable_x86.cpp	+30	–9
cpu/x86/templateTable_x86.hpp	+1	–1
cl/cl_LIRGenerator.cpp	+5	–0

File	+	−
cl/cl_Runtime1.cpp	+12	−3
ci/ciField.cpp	+1	−1
ci/ciField.hpp	+1	−0
ci/ciFlags.hpp	+5	−3
classfile/classFileParser.cpp	+36	−17
classfile/vmSymbols.hpp	+1	−1
gc/shared/accessBarrierSupport.cpp	+25	−6
gc/z/zBarrier.cpp	+55	−6
gc/z/zBarrier.hpp	+2	−1
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zMark.cpp	+70	−20
gc/z/zReferenceProcessor.cpp	+172	−2
gc/z/zReferenceProcessor.hpp	+20	−9
gc/z/zStat.cpp	+13	−7
gc/z/zStat.hpp	+2	−1
gc/z/zWeakFieldArray.hpp (new)	+149	—
oops/fieldInfo.hpp	+3	−0
oops/resolvedFieldEntry.cpp	+2	−1
oops/resolvedFieldEntry.hpp	+10	−5
opto/library_call.cpp	+216	−146
opto/parse3.cpp	+12	−0
prims/jvmtiTagMap.cpp	+41	−10
runtime/fieldDescriptor.hpp	+2	−2
services/heapDumper.cpp	+14	−6
java.base/.../ref/weak.java (new)	+35	—
Total	+940	−257

Appendix D.

Summary Statistics

This appendix provides numerical summary statistics (median, mean, and interquartile range) for all configuration variants and the key metrics discussed in Chapter 5. All tables are produced automatically by the `parse_gc_stats.py` script from the benchmark CSV data. The single-object benchmark values are per-run maxima; the multi-object benchmark values are per-run averages. Each cell is based on $n = 250$ independent benchmark runs.

D.1 Major Collection Time

Table D.1: *Summary statistics for Major Collection across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	6958.1	7088.1	1595.6	982.0	991.7	92.6
clear_path_only	6963.0	7131.0	1493.4	996.4	1002.8	112.5
sep_only	7044.1	7195.2	1486.3	985.1	990.2	116.2
dyn_only	6733.4	6909.2	1495.2	965.0	969.7	109.1
clear_path_sep	6902.6	7131.8	1353.9	1001.5	1010.8	134.4
clear_path_dyn	6399.0	6565.7	1393.5	971.6	976.7	115.5
sep_dyn	6912.6	7026.7	1458.7	987.8	995.2	117.7
all	6376.9	6506.5	1419.3	966.7	969.9	136.7
weak_fields	4135.9	4220.0	484.9	708.3	705.7	93.2

D.2 Old-Generation Collection Time

Table D.2: Summary statistics for Old Generation across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	3630.8	3637.7	683.2	809.2	813.4	80.3
clear_path_only	3600.1	3650.8	626.7	821.8	820.7	110.7
sep_only	3675.5	3708.3	603.9	813.0	812.0	100.4
dyn_only	3377.0	3417.1	871.5	796.7	797.1	97.0
clear_path_sep	3631.1	3688.2	599.3	830.6	829.6	122.7
clear_path_dyn	3003.3	3084.6	719.5	796.3	794.7	117.3
sep_dyn	3392.5	3426.8	761.9	817.4	813.4	106.6
all	2992.7	3069.4	669.7	792.2	792.8	122.5
weak_fields	2295.0	2301.0	281.6	554.3	558.8	82.0

D.3 Old Phase: Concurrent Mark

Table D.3: Summary statistics for Old Phase: Concurrent Mark across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	2575.3	2607.4	607.7	578.1	592.2	66.6
clear_path_only	2621.4	2687.4	584.9	588.0	599.3	89.4
sep_only	2662.3	2713.2	564.9	580.1	589.9	83.1
dyn_only	2644.8	2721.0	810.5	570.7	582.8	81.1
clear_path_sep	2660.1	2721.7	560.3	590.9	608.9	99.1
clear_path_dyn	2780.2	2834.9	661.4	588.1	598.0	96.2
sep_dyn	2731.7	2791.0	666.2	584.1	594.7	76.1
all	2750.2	2825.3	624.0	577.8	594.0	94.3
weak_fields	1782.4	1766.7	272.6	399.0	401.7	46.7

D.4 Non-Strong Processing Time

Table D.4: Summary statistics for Non-Strong Processing across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	996.9	1000.4	75.9	44.1	44.1	9.4
clear_path_only	923.7	933.8	53.1	44.5	44.1	8.0
sep_only	943.8	965.4	73.8	44.1	44.2	9.6
dyn_only	639.1	666.0	100.7	36.3	36.4	6.4
clear_path_sep	923.9	936.9	52.6	40.5	39.9	8.0
clear_path_dyn	187.9	220.2	11.6	18.8	18.8	4.2
sep_dyn	576.9	606.1	75.9	35.5	35.9	7.1
all	184.4	214.6	22.8	18.7	18.9	4.5
weak_fields	484.6	531.8	8.4	35.8	35.5	8.2

D.5 Old Phase: Concurrent Relocate

Table D.5: Summary statistics for Old Phase: Concurrent Relocate across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	0.0	0.0	0.0	155.6	164.6	36.3
clear_path_only	0.0	0.1	0.0	148.2	164.3	46.9
sep_only	0.0	0.1	0.0	148.6	165.0	45.5
dyn_only	0.0	0.1	0.0	155.3	165.0	34.3
clear_path_sep	0.0	0.1	0.0	149.6	168.0	46.7
clear_path_dyn	0.0	0.1	0.0	148.5	164.9	46.0
sep_dyn	0.0	0.1	0.0	160.8	169.7	46.0
all	0.0	0.0	0.0	148.8	166.7	46.3
weak_fields	1.6	1.2	1.9	103.7	116.5	33.8

D.6 Young Generation (Promote All)

Table D.6: *Summary statistics for Young Generation (Promote All) across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	3201.3	3446.1	1516.6	163.5	175.5	50.7
clear_path_only	3219.0	3475.7	1210.2	164.0	179.3	57.5
sep_only	3245.8	3482.6	1571.1	163.8	175.4	48.7
dyn_only	3233.1	3487.7	1688.8	155.7	169.8	45.0
clear_path_sep	3168.9	3439.2	1312.2	164.0	178.5	55.9
clear_path_dyn	3245.0	3476.6	1404.0	161.5	179.1	62.0
sep_dyn	3438.9	3595.6	1622.4	163.9	179.1	53.0
all	3185.2	3432.8	1395.7	161.2	174.3	47.9
weak_fields	1777.3	1915.7	241.2	137.0	144.6	16.4

D.7 Young Phase: Concurrent Mark

Table D.7: *Summary statistics for Young Phase: Concurrent Mark across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	2605.4	2851.6	1523.8	56.1	62.0	25.5
clear_path_only	2623.1	2874.9	1198.1	56.6	64.1	28.7
sep_only	2645.0	2883.8	1574.1	56.2	62.0	24.2
dyn_only	2631.1	2888.7	1691.1	52.5	59.2	22.2
clear_path_sep	2566.5	2837.2	1317.8	56.7	63.7	28.0
clear_path_dyn	2642.4	2873.8	1394.3	55.1	63.9	31.0
sep_dyn	2837.9	2997.3	1623.3	56.3	63.8	26.8
all	2583.7	2831.1	1409.3	54.9	61.4	24.1
weak_fields	1493.5	1631.2	238.6	47.9	51.4	8.3

D.8 Young Phase: Concurrent Relocate

Table D.8: *Summary statistics for Young Phase: Concurrent Relocate across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	395.7	396.8	1.6	13.9	13.9	0.1
clear_path_only	400.7	404.5	7.9	13.7	13.7	0.2
sep_only	394.5	396.2	2.3	13.8	13.8	0.2
dyn_only	399.6	402.0	4.3	13.9	13.9	0.2
clear_path_sep	397.8	399.0	5.6	13.6	13.6	0.2
clear_path_dyn	405.4	406.4	8.9	13.7	13.7	0.2
sep_dyn	394.4	396.2	4.3	13.8	13.8	0.2
all	404.2	405.2	10.2	13.9	13.9	0.1
weak_fields	198.2	199.0	1.2	11.4	11.4	0.1

D.9 Concurrent Mark Follow Time

Table D.9: *Summary statistics for Concurrent Mark Follow across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 100$ per variant).*

Variant	Single (ms)			Multi (ms)		
	Median	Mean	IQR	Median	Mean	IQR
none	2575.2	2607.3	607.7	578.1	592.1	66.6
clear_path_only	2621.3	2687.3	584.8	587.9	599.3	89.4
sep_only	2662.2	2713.1	564.9	580.1	589.9	83.1
dyn_only	2644.7	2721.0	810.5	570.7	582.8	81.1
clear_path_sep	2660.0	2721.6	560.3	590.8	608.8	99.1
clear_path_dyn	2780.1	2834.8	661.4	588.0	597.9	96.2
sep_dyn	2731.7	2790.9	666.2	584.1	594.7	76.2
all	2750.1	2825.2	624.0	577.7	593.9	94.3
weak_fields	1782.3	1766.7	272.6	398.9	401.6	46.7

D.10 Auxiliary GC Memory

Table D.10: Summary statistics for Aux Memory per-run maximum across all variants. Values in MB ($n = 100$ per variant).

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
none	120	120	2	292	294	4
clear_path_only	121	121	2	293	294	2
sep_only	120	120	2	293	294	3
dyn_only	308	310	0	305	305	5
clear_path_sep	121	121	2	292	294	2
clear_path_dyn	1268	1303	3	363	357	23
sep_dyn	308	310	0	304	305	6
all	884	901	3	340	337	14
weak_fields	446	448	5	240	239	6

D.11 Java Heap

Table D.11: Summary statistics for Java Heap per-run maximum across all variants. Values in MB ($n = 100$ per variant).

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
none	1720	1721	2	1926	1927	2
clear_path_only	1720	1721	2	1928	1929	2
sep_only	1722	1721	2	1928	1928	2
dyn_only	1720	1721	2	1928	1928	2
clear_path_sep	1721	1721	2	1928	1929	2
clear_path_dyn	1720	1721	2	1928	1928	2
sep_dyn	1720	1721	2	1928	1928	2
all	1720	1720	2	1926	1927	2
weak_fields	806	805	2	1834	1834	2

D.12 Total Process RSS

Table D.12: Summary statistics for RSS per-run maximum across all variants. Values in MB ($n = 100$ per variant).

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
none	1901	1903	5	2407	2408	6

Variant	Single (MB)			Multi (MB)		
	Median	Mean	IQR	Median	Mean	IQR
clear_path_only	1903	1905	6	2409	2410	8
sep_only	1902	1904	5	2409	2409	8
dyn_only	2107	2108	4	2423	2424	11
clear_path_sep	1904	1905	7	2410	2410	8
clear_path_dyn	2856	2861	43	2487	2492	30
sep_dyn	2107	2109	3	2425	2425	10
all	2556	2552	16	2466	2468	27
weak_fields	1260	1260	3	2255	2256	13

Appendix E.

Additional Benchmark Plots

This appendix contains the remaining GC timing and memory plots from the benchmark evaluation. The metrics shown here (Old Phase: Concurrent Relocate, Young Phase: Concurrent Mark, and Young Phase: Concurrent Relocate) were not included in the main evaluation chapter as they are less central to the optimised reference processing pipeline. Memory maximum distributions are included here to complement the median percentage-difference histograms shown in the main chapter.

Additional GC timing plots are shown for Old Phase: Concurrent Relocate (Figures E.1 and E.2), Young Phase: Concurrent Mark (Figures E.3 and E.4), and Young Phase: Concurrent Relocate (Figures E.5 and E.6) for both benchmark families. Memory per-run maximum distributions are shown for auxiliary GC memory, Java heap, and total process RSS for both the single-object and multi-object benchmarks (Figures E.7 and E.8).

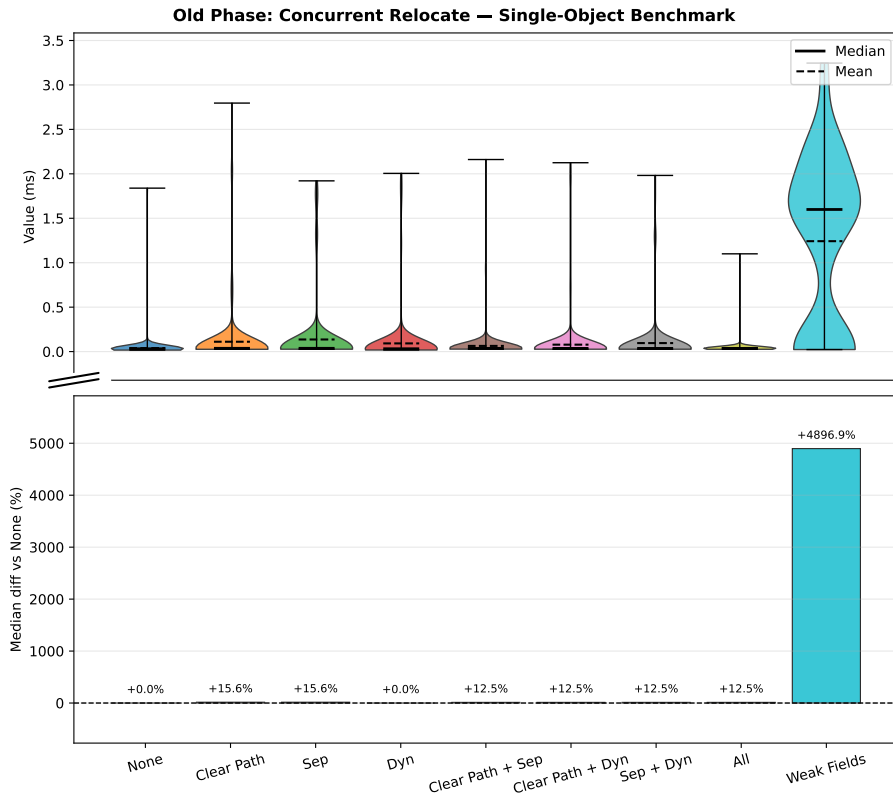


Figure E.1. Old Phase: Concurrent Relocate timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

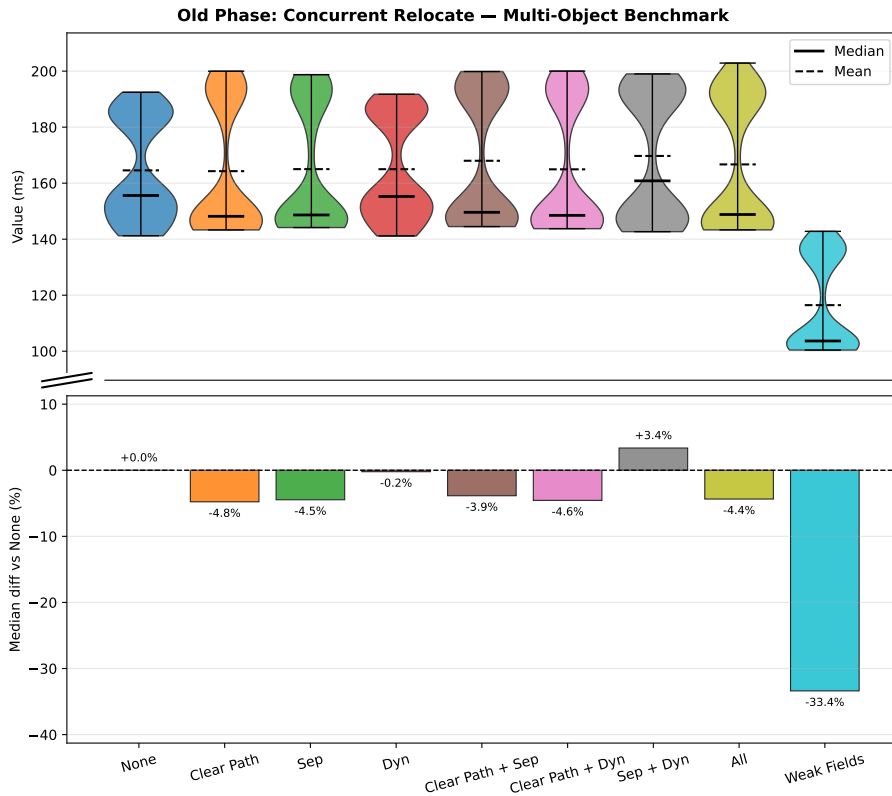


Figure E.2. Old Phase: Concurrent Relocate timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

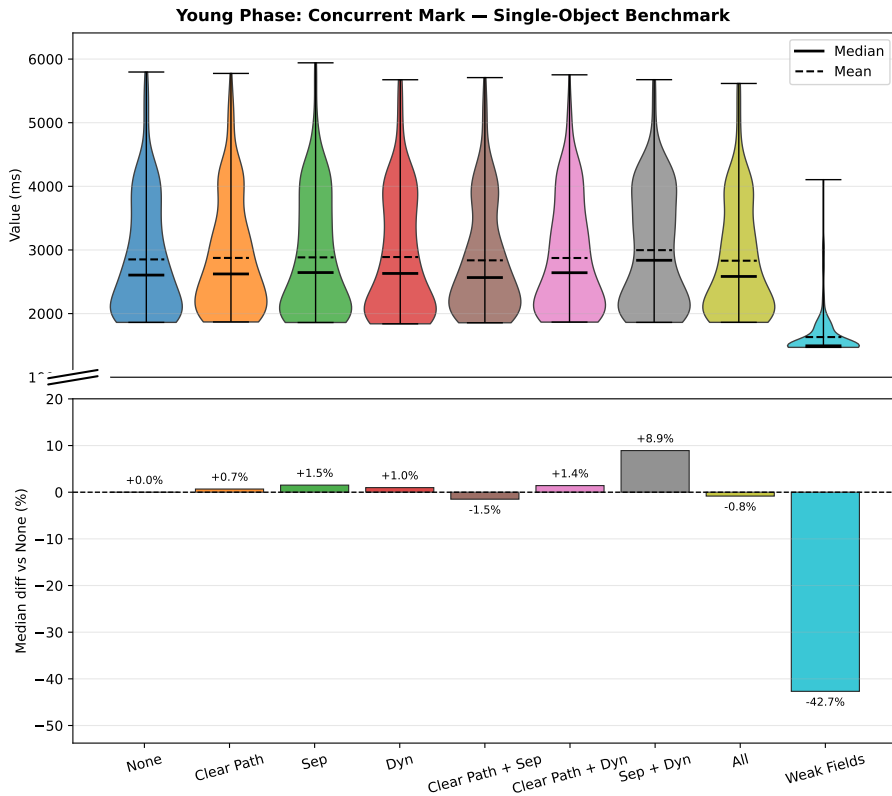


Figure E.3. Young Phase: Concurrent Mark timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

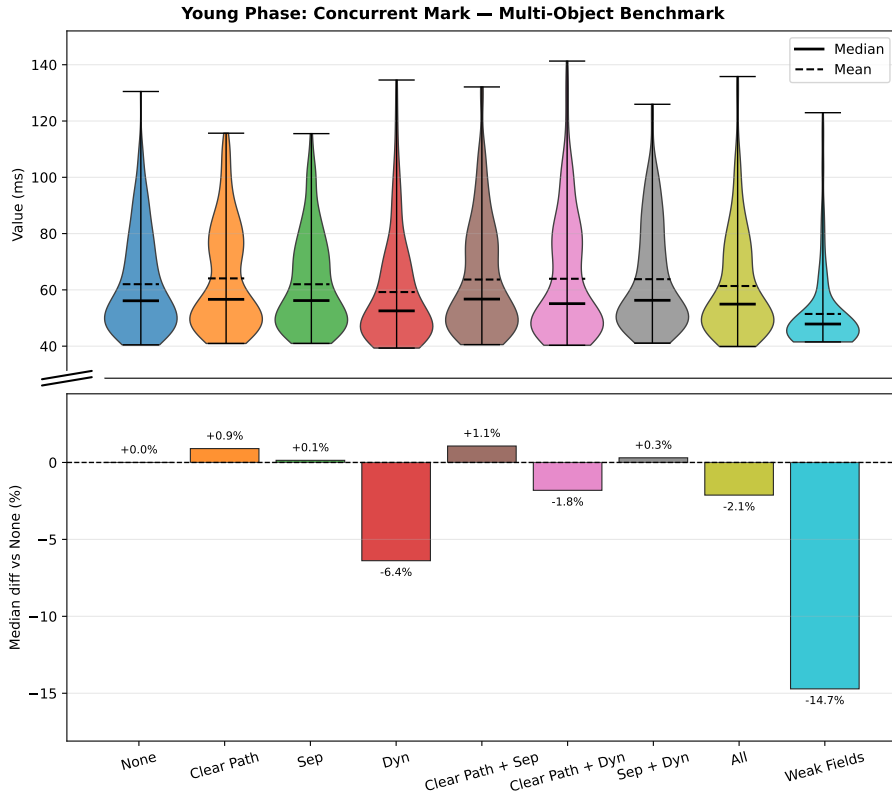


Figure E.4. Young Phase: Concurrent Mark timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

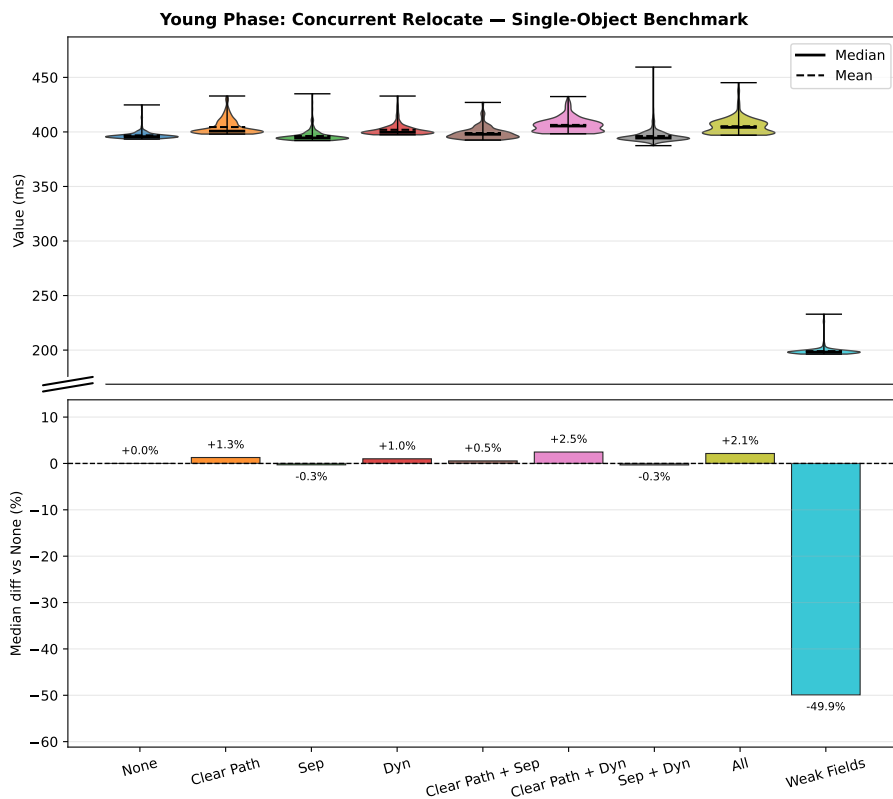


Figure E.5. Young Phase: Concurrent Relocate timing, single-object benchmark (per-run maxima). Top: distribution; bottom: median % difference vs baseline.

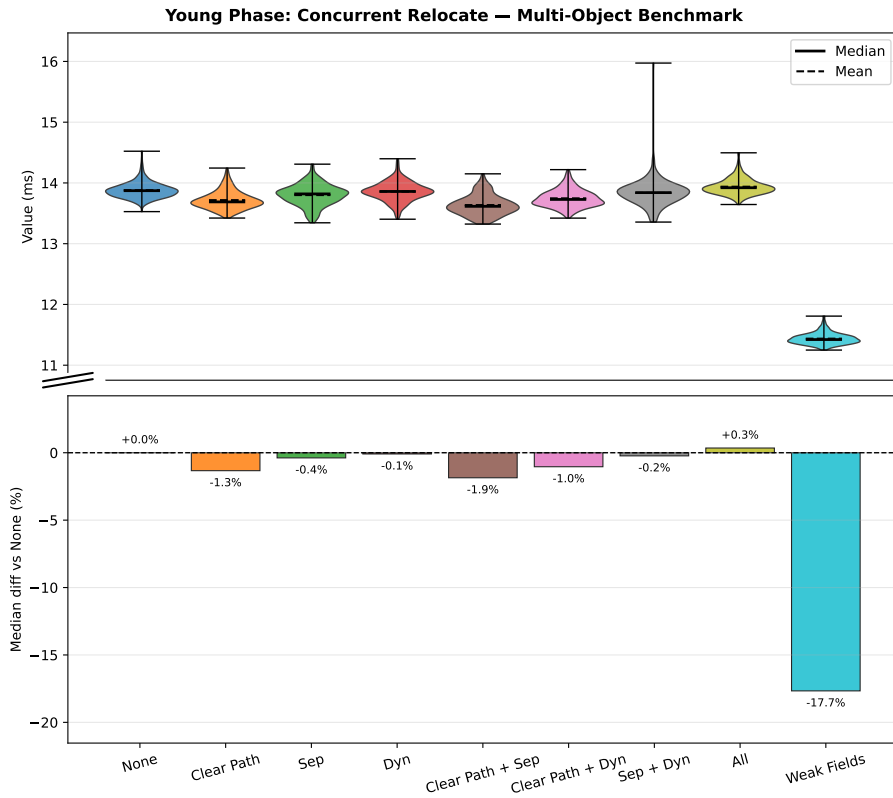


Figure E.6. Young Phase: Concurrent Relocate timing, multi-object benchmark (per-run averages). Top: distribution; bottom: median % difference vs baseline.

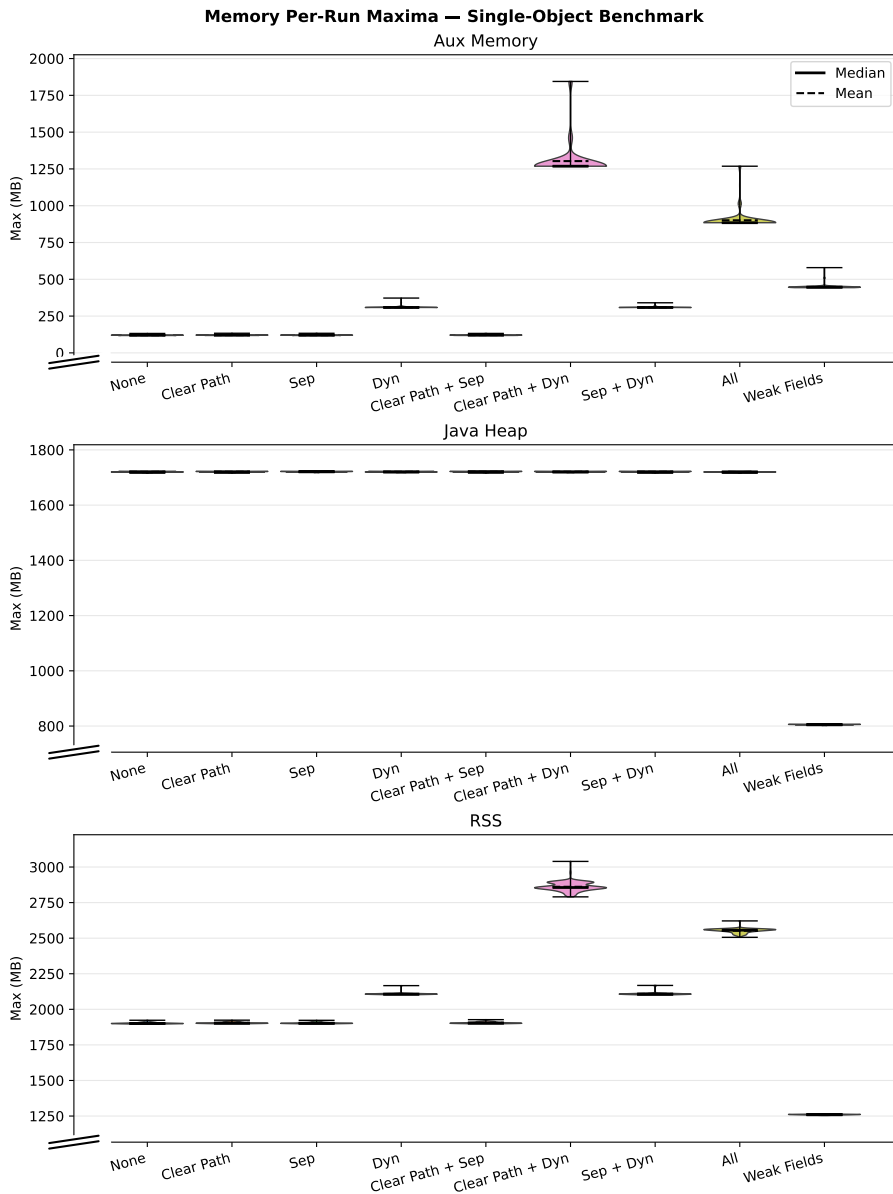


Figure E.7. Distribution of per-run maximum memory across variants, single-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.

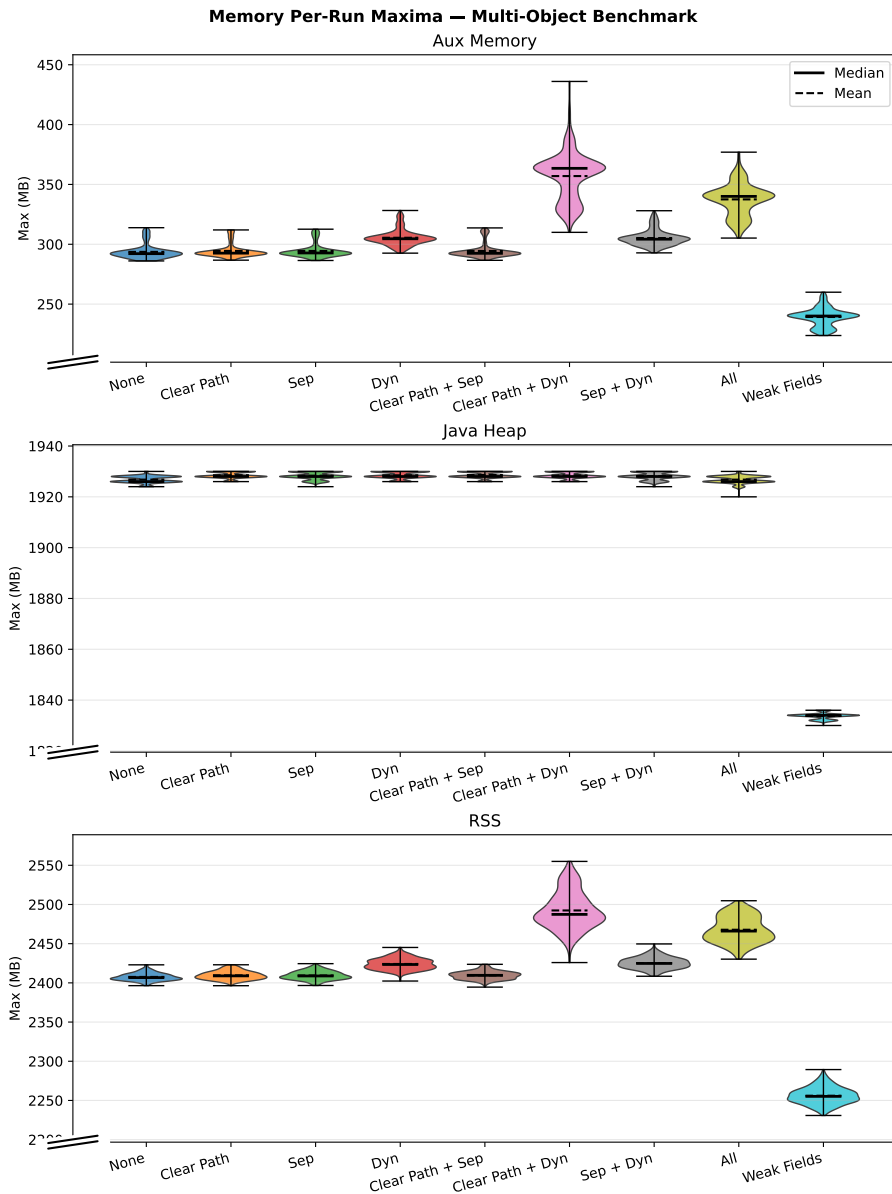


Figure E.8. Distribution of per-run maximum memory across variants, multi-object benchmark. Top to bottom: auxiliary GC memory, Java heap, total process RSS.